学生検索画面 基本設計書

文書属性	内容
文書ID	CSHARP_ASPNET-BAS-STU-01
種別	basic-design／基本設計書
版	1.0
正本の相対パス	基本設計書/01_学生検索画面.md
対象責務	学生の条件検索、候補更新、一覧・ページ操作、検索状態の保持、登録・編集・参照との往復
記述基準	ソースで観測した実装。DDLの宣言、初期投入SQLの定義、コメントの意図は区別して記載する。
根拠表記	原ソースのPOSIX相対パスと開始行・終了行。対応するindexed原文の行番号を使用する。
対応文書	CSHARP_ASPNET-DET-CSharp_ASPNET-SCR-STU-001／detailed-specification／詳細仕様書/CSharp_ASPNET/01_学生検索画面.md。計画時点では未生成。検索項目、一覧項目、制御値、入口、各イベントを同じ原ソースへ対応させる。

概要

1.1 目的と利用者

学生検索画面は、論理削除されていない学生を条件で絞り込み、氏名、生年月日、所属、在籍区分、住所、本人電話、更新日時を一覧で確認する画面である。検索SQLには常に s.is_deleted=0 を設定する。電話番号の条件は本人電話と緊急連絡先電話を検索するが、一覧の電話列は本人電話だけを表示する。 [根拠：Views/Students/Search.cshtml:5-8、Core/SqlServerStudentService.cs:77-90・137-153]

宣言された役割はViewer、Staff、Administratorである。認証済み利用者が検索・既存学生の読込みを行い、Viewerの行リンクは「参照」、その他の役割の行リンクは「編集」となる。両者の学生読込入口は同じである。Viewerには新規登録リンクを表示せず、遷移先の登録・更新もサーバー側で拒否する。論理削除はAdministratorに限定する。 [根拠：Core/Contracts.cs:4-6、Views/Students/Search.cshtml:6-8、Infrastructure/RequestIdentity.cs:13-16、Infrastructure/WebStudentCoordinator.cs:17-19]

1.2 画面と処理の受渡し

画面は共有ヘッダー、画面名、検証メッセージ・通知、検索条件、検索・候補更新、クリア・新規登録、結果説明、件数・並び順、11列の一覧、ページ操作の順に配置する。ヘッダーの利用者名は User.Identity.Name、業務日は現在時刻をUTCオフセット+9へ変換した日付である。利用者名の表示と、検索状態の所有者照合に使う認証主体は別の値である。［根拠：Views/Shared/_Layout.cshtml:1-1、Views/Shared/_MessagePanel.cshtml:2-3、Views/Students/Search.cshtml:3-8、Infrastructure/WebPresentation.cs:5-7、Infrastructure/RequestIdentity.cs:8-10］

ブラウザーはフォーム値を送信し、MVCがバインド・変換した値をコントローラーが受ける。検索では StudentSearchInput を SearchCriteria へ移送し、サービスが検査・正規化した局所値をSQLパラメータにする。取得行は StudentRow、表示用オブジェクト、JsonElement を経てRazorが表示する。このJSON変換は画面モデル内部の処理であり、接続されたフォーム入口の成功応答はRazor画面である。［根拠：Web/Controllers/StudentsController.cs:13-22・39-40、Web/Inputs/StudentSearchInput.cs:19-22、Core/SqlServerStudentService.cs:77-90、Infrastructure/WebPresentation.cs:20-20、Web/Models/StudentSearchPageModel.cs:15-15］

入力中の条件、適用済み条件、直近結果は別々に扱う。候補更新は入力中の条件と候補を再表示する操作であり、適用済み条件と一覧を更新しない。検索・ページ変更が成功すると条件と結果をメモリへ反映する。検索復帰は保持条件で一覧を取り直す。［根拠：Web/Controllers/StudentsController.cs:22-22、Infrastructure/WebStudentCoordinator.cs:7-15］

1.3 静的画面投影

次のHTMLは、Viewer以外の利用者について、初期条件で検索した通常結果の1ページ目に次ページが存在する状態を表す。角括弧の値は動的値の差込み位置であり、実際の学生、利用者、件数を示すものではない。在籍区分の名称はDB候補の名称を置く。01〜04はseedに定義されたコードを投影しており、実画面では取得した候補だけを列挙する。実画面の撮影ではない。

共有レイアウトと検索画面に適用されるCSSを一体で示す。検索条件の .fields は2列、幅760px以下では1列である。一覧の .grid は横スクロールを持ち、表は最小幅1050px、セルは任意位置で折り返す。操作領域の .actions は折返し可能なflex配置である。項目の id と配置を決める class は別属性である。元の送信先とHTTP methodは非実行のdata属性に記録し、リンクは文書内fragment、ボタンはすべて type=button とする。［根拠：Views/_ViewStart.cshtml:1-1、Views/_ViewImports.cshtml:1-3、Views/Shared/_Layout.cshtml:1-1、Views/Students/Search.cshtml:3-8、wwwroot/css/site.css:1-1］

学生検索

検索条件

学籍番号

氏名

氏名カナ

年度

すべて

学年

すべて

クラス

すべて

在籍区分

[01のDB名称] [02のDB名称] [03のDB名称]

[04のDB名称] 全解除はすべて

未所属のみ

都道府県

すべて

電話番号

入学日・開始

年 /月/日

入学日・終了

年 /月/日

検索 候補を更新

クリア 新規登録

検索結果 [総件数] 件 [検索日時]

表示件数 並び順

20 学籍番号

昇順

並び順・件数を反映

学籍番号	氏名	氏名カナ	生年月日	年度／学年	クラス	在籍区分	住所	本人電話	更新日時	操作
［学籍番号］	［氏名］	［氏名カナ］	［和暦生年月日］	［年度］ ［学年名］	［クラスコード］ ［クラス名］	［在籍区分名］	［都道府県名＋住所1］	［本人電話］	［更新日時］	編集

前へ　1 / ［最終ページ］　次へ

互いに異なる状態は、上の投影に重ねて同時成立させず、次の差分として表す。［根拠：Views/Students/Search.cshtml:5-8、Views/Shared/_MessagePanel.cshtml:2-3、wwwroot/js/site.js:7-7］

状態・役割	表示・操作の差分
Viewer	新規登録のリンクを表示しない。各行の「編集」を「参照」にする。元の学生読込routeは同じ。
初期表示・クリア後	結果説明を「未検索」、tbodyを空、ページを1 / 1とする。反映・前へ・次へを無効化する。条件は在籍01・02、その他未指定、未所属false。
未所属trueの再描画	SRC-007をchecked、SRC-004・005・006をdisabledにする。候補更新で所属3値を解除する。チェック変更時のJavaScriptは値を空にするだけで、その場ではdisabledを切り替えない。
0件	結果説明を「該当する学生はありません。」、tbodyを空にする。反映・前後ボタンは無効。件数・検索日時の数値表示は行わない。
10,000件超過	warningクラスの段落で「10,000件を超えました。条件を追加してください。」を表示する。tbodyを空にし、反映・前後ボタンを無効化する。件数と検索日時を表示しない。
通常結果のページ境界	前へはpageが1以下なら無効。次へはpage×表示件数が総件数以上なら無効。反映ボタンは有効。

状態・役割	表示・操作の差分
通常通知	h1の直後にmessageとinfoの両classを持つdivを表示し、aria-liveをpoliteにする。info単独に対応するCSS規則はない。
検証エラー	h1の直後にmessageとerrorの両classを持つAll検証サマリーを表示し、aria-liveをpoliteにする。通常通知もある場合は検証サマリーの後に表示する。
停止クラス候補	「年度 クラスコード 名称」の後ろへ「（停止）」を追加する。期間外の注記はない。一覧クラス列には停止注記を追加しない。

1.4 結果状態

検証失敗、未検索、成功0件、通常結果、上限超過を区別する。上限判定の内部件数10001は打切り値であり、実総件数ではない。表示件数・並び順・方向のselect自体は、未検索・0件・上限超過でも無効化しない。［根拠：Core/SqlServerStudentService.cs:156-170、Infrastructure/WebStudentCoordinator.cs:20-21、Views/Students/Search.cshtml:7-8］

表示状態	データ側の値	結果説明	反映・ページ操作
unsearched	totalCount=NULL、items空、searchedAt=NULL	未検索	反映・前後とも無効
empty	totalCount=0、items空、searchedAtあり	該当する学生はありません。	反映・前後とも無効
found	totalCount=検索件数、itemsはページ分、searchedAtあり	件数と検索日時	反映は有効。前後はページ境界による。
tooMany	totalCount=NULL、items空、searchedAtあり	条件追加の警告	反映・前後とも無効。時刻は画面に表示しない。

ページ分母は最低1で、画面モデルの表示件数と件数から計算する。未検索・上限超過の件数NULLはビュー内で0に置き換えて分母を計算する。検索失敗の再表示では過去の適用済みページが残る場合があり、「未検索」の表示だけから保持状態まで初期化されたと判断しない。［根拠：Views/Students/Search.cshtml:2-2・8-8、Infrastructure/WebStudentCoordinator.cs:21-21、Web/Controllers/StudentsController.cs:14-16］

項目一覧

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
SRC-001	学籍番号で学生を絞る。	input、Criteria.StudentNo。入力DTO string →検索引数StudentNo:string?。	初期空、maxlength=30。サービスは原入力・Trim後・NFKCと英大文字化後を15スカラー以内に検査。先頭英数字、以降英数字・ハイフン。NULL・空文字は条件なし。空白だけの非空入力は必須検査エラー。	student_no の前方一致、@no:VarChar(31)。保存も同じ加工で VarChar(15)、列は m_student.student_no:varchar(15) NOT NULL。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:7-22、Core/StudentNormalizer.cs:47-64・124-124、Core/SqlServerStudentService.cs:30-31・113-140、database/001_schema.sql:9-9
SRC-002	氏名の照合値で部分一致検索する。	input、Criteria.Name:string → StudentName:string?。	初期空、maxlength=16。原入力・Trim後各8スカラー以内。NULL・空白のみは条件なし。照合キーはNFKC、英大文字化、全Unicode空白除去後160スカラー以内。	name_key の部分一致、@name:NVarChar(642)。保存は表示値 student_name:nvarchar(16) NOT NULL とキー name_key:nvarchar(320) NOT NULL に分ける。保存パラメータ16・320、DDLスカラーCHECK8・160。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:8-22、Core/StudentNormalizer.cs:47-70・125-145、Core/SqlServerStudentService.cs:31-32・114-141、database/001_schema.sql:10-13・44-46
SRC-003	氏名カナの照合値で部分一致検索する。	input、Criteria.Kana:string → StudentNameKana:string?。	初期空、maxlength=72。原入力36スカラー以内。Trim、NFKC、ひらがなのカタカナ化、単語間空白の単一半角空白化を行い、許可カタカナ・長音・中点・繰返し記号に限定。正規化後は空白込み18スカラー以内。NULL・空文字は条件なし。空白だけの非空入力は空キーのLIKEになる。	半角空白除去後の kana_key を部分一致検索、@kana:NVarChar(74)。保存は表示カナとキーを別列へ設定。両列 nvarchar(36) NOT NULL、保存 NVarChar(36)、DDLスカラーCHECK各18。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:9-22、Core/StudentNormalizer.cs:72-85・126-145、Core/SqlServerStudentService.cs:31-32・116-142、database/001_schema.sql:11-13・45-47

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
SRC-004	所属クラスの年度で絞る。	select、Criteria.SchoolYear:int? → SchoolYear:int?。	初期「すべて」。全クラス候補の年度を重複除去し降順。指定時1900～9998と年度候補の存在を確認。未所属の再描画時disabled。	m_class.school_year:smallint NOT NULL に一致、@year:SmallInt。学生保存ではクラス整合用の補助引数。学生へ保存する所属値はclass_id。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:10-22、Core/SqlServerStudentService.cs:109-129・143-143・363-374、database/001_schema.sql:64-64
SRC-005	所属クラスの学年で絞る。	select、Criteria.GradeCode:string? → GradeCode:string?。	初期「すべて」。GRADEのコード値を選び名称を表示。原入力・Trim後2スカラー以内、NFKC後半角数字2桁。空白のみは条件なし。コード存在を確認。未所属の再描画時disabled。	m_class.grade_code:varchar(2) NOT NULL に一致、@grade:VarChar(2)。保存時はクラス整合用。学年名はm_code.code_name。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:11-22、Core/StudentNormalizer.cs:113-117、Core/SqlServerStudentService.cs:118-135・144-144、database/001_schema.sql:65-66
SRC-006	特定クラスへの所属で絞る。	select、Criteria.ClassId:string? → ClassId:long?。	初期「すべて」。年度・学年に合う候補を列挙。空値は未指定。それ以外は19コード単位以内の半角数字かつ正のlongへ変換。サービスで存在・上位条件整合を確認。未所属の再描画時disabled。	キーはclass_id、表示は年度・クラスコード・名称。検索・学生保存列はm_student.class_id:bigint NULL、@class:BigInt。参照先m_class.class_id:bigint NOT NULL はPK。名称保存側の制約は補足説明4.2による。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:12-22、Web/Inputs/StudentSaveInput.cs:57-60、Core/SqlServerStudentService.cs:131-145、database/001_schema.sql:28-28・63-68

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
SRC-007	クラス未所属の学生だけを検索する。	checkbox、Criteria.UnassignedOnly:bool → UnassignedOnly:bool。	初期false。所属3値の非NULLとtrueの同時指定を拒否。候補更新では所属3値を解除。ブラウザーのチェック変更は値のクリアのみ。	非保存項目。SQLにs.class_id IS NULL を追加する。クラス条件の未指定とは異なる。	Views/Students/Search.cshtml:5-5、Web/Controllers/StudentsController.cs:22-22、Core/SqlServerStudentService.cs:110-110・146-146、wwwroot/js/site.js:7-7
STATUS	在籍区分を複数選択して絞る。文書整理キーで、原ビューに個別DOM IDはない。	候補別checkbox、送信名Criteria.StatusCodes。入力DTO string[] → EnrollmentStatusCodes:string[]。	初期01・02。seedの名称は在学・休学、画面名称はDB候補。キー欠落は入口で空配列へ補正。全解除は絞込みなし。最大4値、重複なし、01～04を検査。NULL配列は空配列と同じ分岐ではない。	m_student.enrollment_status_code:varchar(2) NOT NULL のIN条件。各@statusN:VarChar(2)。保存は単一の必須2桁コードをVarChar(2) で設定。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:14-22、Web/Controllers/StudentsController.cs:14-14・22-22、Core/SqlServerStudentService.cs:111-112・151-153、Core/StudentNormalizer.cs:136-136、database/001_schema.sql:25-26、database/002_seed.sql:19-22
SRC-009	住所の都道府県で絞る。	select、Criteria.PrefectureCode:string? → PrefectureCode:string?。	初期「すべて」。PREFECTUREのコード値・名称を使用。原入力・Trim後2スカラー以内、NFKC後半角数字2桁。空白のみは条件なし。分類内の存在を確認。	m_student.prefecture_code:varchar(2) NOT NULL に一致、@prefecture:VarChar(2)。保存では必須コード。住所の都道府県名はコード表とのLEFT JOIN結果。	Views/Students/Search.cshtml:5-5、Core/StudentNormalizer.cs:113-117・130-130、Core/SqlServerStudentService.cs:80-90・119-125・147-147、database/001_schema.sql:18-19

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
SRC-010	本人電話または緊急連絡先電話の一致で絞る。	input、Criteria.PhoneNumber:string → PhoneNumber:string?。	初期空、maxlength=40。サービス原入力上限20スカラー。Trim・NFKC後、全空白、半角ハイフン、U+2010・U+2011・U+2212、丸括弧を除去。国内は0開始10〜11桁、国際は+に続く非0開始7〜15桁。NULL・空白のみは条件なし。ただし原入力上限の検査が先。	本人または緊急電話へ完全一致、@phone:VarChar(16)。保存も同じ加工で両列varchar(16) NULL、保存VarChar(16)。一覧は本人電話のみ。	Views/Students/Search.cshtml:5-8、Core/StudentNormalizer.cs:47-57・87-94・133-141、Core/SqlServerStudentService.cs:34-37・117-148、database/001_schema.sql:22-32
SRC-011	入学日の下限で絞る。	date input、Criteria.AdmissionFrom:string? →同名検索引数、局所値 DateOnly?。	初期未指定。NULL・空文字は下限なし。非空はNFKC後yyyy-MM-dd の10文字を厳密解析。1900-01-01以上かつ日本時間の業務日以下。前後空白をTrimしない。両端指定時は開始≦終了。	m_student.admission_date:date NOT NULL に開始日以上、@from:Date。学生保存では単一入学日を同じ解析で検査する。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:17-22、Core/JapaneseDateService.cs:52-59、Core/SqlServerStudentService.cs:120-122・149-149、Core/StudentNormalizer.cs:135-135、database/001_schema.sql:24-24
SRC-012	入学日の上限で絞る。	date input、Criteria.AdmissionTo:string? →同名検索引数、局所値 DateOnly?。	初期未指定。NULL・空文字は上限なし。解析・範囲はSRC-011と同じ。終了日を含む。	admission_date に終了日以下、@to:Date。別の終了日列へ保存する値ではない。	Views/Students/Search.cshtml:5-5、Web/Inputs/StudentSearchInput.cs:18-22、Core/JapaneseDateService.cs:52-59、Core/SqlServerStudentService.cs:121-122・150-150

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
PAGE-SIZE	1ページの最大表示行数。文書整理キー。	selectの PageSize:int。検索フォームにも同名hiddenがあるが別フォーム。	初期20、許可20・50・100。適用済み件数からの変更反映は先頭ページへ移動。select変更だけでは検索フォームhiddenを更新しない。	非DB保存項目。適用済み状態へ保持し、取得SQLの @size:Int に使用。	Views/Students/Search.cshtml:4-4・8-8、Web/Inputs/StudentSearchInput.cs:27-29、Infrastructure/WebStudentCoordinator.cs:12-12、Core/SqlServerStudentService.cs:107-107・164-164
SORT-KEY	一覧の比較対象。文書整理キー。	selectの SortKey:string、検索フォームのhidden。	初期studentNo。許可 studentNo・name・admissionDate・updatedAt。変更反映は先頭ページへ移動。	student_no・name_key・admission_date・updated_at に対応。氏名順は照合キー順。各ORDER BYの末尾に student_id ASC を付ける。	Views/Students/Search.cshtml:4-4・8-8、Core/SqlServerStudentService.cs:108-108・162-164、Infrastructure/WebStudentCoordinator.cs:12-12
SORT-DIR	比較方向。文書整理キー。	selectの SortDirection:string、aria-labelは並び順方向。検索フォームにもhidden。	初期asc。許可asc・desc。方向変更の反映は先頭ページへ移動。	非DB保存項目。許可済みの方向だけをORDER BYへ使用。	Views/Students/Search.cshtml:4-4・8-8、Core/SqlServerStudentService.cs:107-108・162-162、Infrastructure/WebStudentCoordinator.cs:12-12
PAGE	一覧のページ位置。文書整理キー。	ボタン送信名 Page、PageRequest.Page:int。	初期1、サービスは1以上を要求。反映は1、前後は表示pageの±1。ページ操作と検索復帰では保持ページの補正が異なる。	非DB保存項目。取得位置へ換算し @offset:Int に使用。	Views/Students/Search.cshtml:8-8、Web/Inputs/StudentSearchInput.cs:29-29、Infrastructure/WebStudentCoordinator.cs:12-14、Core/SqlServerStudentService.cs:160-164

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
ROW-01	一覧の学籍番号。文書整理キー。	第1列、StudentRow.StudentNo:string → studentNo。	保存済み学籍番号を表示。	m_student.student_no:varchar(15) NOT NULL。	Views/Students/Search.cshtml:8-8、Core/Contracts.cs:50-50、Core/SqlServerStudentService.cs:78-83、database/001_schema.sql:9-9
ROW-02	一覧の氏名。文書整理キー。	第2列、StudentName:string → studentName。	照合キーではなく表示氏名を表示。保存時8スカラー上限。	m_student.student_name:nvarchar(16) NOT NULL、保存 NVarChar(16)。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:31-31・78-78、Core/StudentNormalizer.cs:125-125、database/001_schema.sql:10-10
ROW-03	一覧の氏名カナ。文書整理キー。	第3列、StudentNameKana:string → studentNameKana。	空白除去キーではなく正規化済み表示カナ。保存時空白込み18スカラー上限。	m_student.student_name_kana:nvarchar(36) NOT NULL、保存 NVarChar(36)。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:31-31・78-78、Core/StudentNormalizer.cs:72-85・126-126、database/001_schema.sql:11-11
ROW-04	生年月日を和暦で確認する。文書整理キー。	第4列、BirthDate:DateOnly → BirthDateWareki:string。	明治・大正・昭和・平成・令和の明示境界を使用。「元号名＋元号年＋年＋MM月DD日」。元号年1は元。明治の実装下限は1900-01-01。	m_student.birth_date:date NOT NULL。検索条件には使用しない。	Views/Students/Search.cshtml:8-8、Core/Contracts.cs:50-55、Core/JapaneseDateService.cs:10-16・41-50、database/001_schema.sql:14-14

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
ROW-05	所属年度と学年を確認する。文書整理キー。	第5列、SchoolYear:int? と GradeName:string?。	年度と学年名を並べる。未所属・結合不成立時はNULLになり得る。	年度は m_class.school_year:smallint NOT NULL、学年名は m_code.code_name:nvarchar(24) NOT NULL。結合結果のNULLと保存列のNOT NULLは別。	Views/Students/Search.cshtml:8-8、Core/Contracts.cs:51-51、Core/SqlServerStudentService.cs:79-89、database/001_schema.sql:64-66・115-115
ROW-06	所属クラスを確認する。文書整理キー。	第6列、ClassCode:string? と ClassName:string?。	コードと名称を並べる。一覧には停止注記を付けない。	m_class.class_code:varchar(9) NOT NULL、class_name:nvarchar(36) NOT NULL。保守検査はコード9・名称18スカラー、保存は VarChar(9)・NVarChar(36)。結合結果はNULL可能。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:79-87、Core/MasterMaintenanceService.cs:33-33・73-76、database/001_schema.sql:67-81
ROW-07	在籍区分を名称で確認する。文書整理キー。	第7列、EnrollmentStatusName:string? → enrollmentName。	学生の在籍コードに対応するDB名称を表示。	分類・コードで m_code へ LEFT JOIN。名称保存列 nvarchar(24) NOT NULL、保守検査12スカラー、保存 NVarChar(24)。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:80-89、Infrastructure/WebPresentation.cs:20-20、Core/MasterMaintenanceService.cs:82-82・113-120、database/001_schema.sql:115-128
ROW-08	住所を確認する。文書整理キー。	第8列、Address:string → addressDisplay。	都道府県名と住所1を直接連結。名称NULLなら先頭に[不整合]。住所2は取得・表示しない。	都道府県名はコード名称。住所1は m_student.address_line1:nvarchar(90) NOT NULL、保存時原入力・Trim後各45スカラー、保存 NVarChar(90)。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:34-34・80-90、Core/StudentNormalizer.cs:131-132、database/001_schema.sql:20-21

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
ROW-09	本人電話を確認する。文書整理キー。	第9列、PhoneNumber:string?。	NULLだけを「ー」にする。緊急電話で検索が一致した学生も本人電話を表示。	m_student.phone_number:varchar(16) NULL。加工・保存条件はSRC-010と同じ。	Views/Students/Search.cshtml:8-8、Core/SqlServerStudentService.cs:80-85・148-148、database/001_schema.sql:22-22
ROW-10	学生の最終更新日時を確認する。文書整理キー。	第10列、UpdatedAt:DateTimeOffset → updatedAt:string。	UTCオフセット+9へ変換しyyyy/MM/dd HH:mm:ss。検索実施日時とは別。	m_student.updated_at:datetimeoffset(7) NOT NULL。保存・削除で処理開始時のnowを設定し、更新者と行版も更新。	Views/Students/Search.cshtml:8-8、Infrastructure/WebPresentation.cs:7-7・20-20、Core/SqlServerStudentService.cs:229-229・297-301・394-410、database/001_schema.sql:40-42
ROW-11	選択学生を編集・参照する。文書整理キー。	第11列のa要素。StudentId:long を10進文字列化してURLに使用。	Viewerは参照、その他は編集。検索状態IDを復帰先として渡す。	m_student.student_id:bigint IDENTITY NOT NULL、PK。行リンクは更新しない。RowVersion:long は行に取得するが検索画面に表示しない。	Views/Students/Search.cshtml:8-8、Infrastructure/WebPresentation.cs:20-20、Web/Controllers/StudentsController.cs:24-25、Core/Contracts.cs:50-53、database/001_schema.sql:8-8・42-43
CTL-CONTEXT	検索状態を指定する。文書整理キー。	各POSTフォームのhidden ContextId。要求 string、Get引数 string?。	生成値は32ランダムバイトの小文字16進64文字。POSTのNULL・空・不在・失効は409。GET初期表示のNULL・空は新規作成。	DB列ではなくメモリ辞書のキー。学生IDと異なる。	Views/Students/Search.cshtml:4-8、Web/Inputs/StudentSearchInput.cs:25-29、Infrastructure/SearchContextStore.cs:9-14、Web/Controllers/StudentsController.cs:9-10

項目ID	論理項目・用途	実コントロール／内部値	初期値・検査・加工	検索・表示・保存列との対応	根拠
CTL-REVISION	古い画面からの状態更新を検知する。文書整理キー。	hidden ContextRevision:string、内部 Revision:long。	初期1。受信文字列と現在版のInvariantCulture文字列を直接比較。相違・版上限は409。候補更新では比較しない。	メモリの検索状態版。学生DBのrow_versionとは別。	Views/Students/Search.cshtml:4-8、Web/Inputs/StudentSearchInput.cs:25-29、Infrastructure/SearchContextStore.cs:9-14、Web/Controllers/StudentsController.cs:22-22
CTL-TAB	検索画面を識別する。文書整理キー。	hidden TabId:string、Entryの Tab:string。	新規未指定時は64文字生成値。非空受信Tabだけを比較し、不一致は403。NULL・空では比較省略。	メモリ制御値。検索POSTで渡す。GET復帰・復帰先確認では渡さない。	Views/Students/Search.cshtml:4-8、Infrastructure/SearchContextStore.cs:11-14、Web/Controllers/StudentsController.cs:9-22、Infrastructure/WebStudentCoordinator.cs:6-6
CTL-CSRF	フォーム要求を検証する。文書整理キー。	各POSTフォームの Html.AntiForgeryToken() と全体の検証filter。	検証値は業務入力ではない。拒否は通常の入力エラー再表示へ合流しない。	非DBの要求検証情報。検索routeは /api 用JSON失敗変換の対象外。	Views/Students/Search.cshtml:4-8、Infrastructure/WebHostSetup.cs:15-16、Infrastructure/AntiforgeryResultFilter.cs:7-8

イベント仕様

ProgramはWeb設定後にコントローラーを接続し、IStudentService を SqlServerStudentService へ登録する。コントローラー全体にAuthorizeを指定する。共通のフォーム検査・検索状態・最終例外応答は補足説明4.3〜4.5に示し、以下ではイベント固有の検査位置、引数、結果を記載する。 [根拠：Program.cs:2-8、Infrastructure/WebHostSetup.cs:13-19、Web/Controllers/StudentsController.cs:5-6]

操作	接続されたHTTP入口	主要な入力
初期表示・検索復帰	GET /students	contextId、contextRevision

操作	接続されたHTTP入口	主要な入力
検索	POST /students/search	Criteria、ContextId、ContextRevision、TabId、PageSize、SortKey、SortDirection
候補更新	POST /students/options	検索フォームと同じ項目
ページ・並び順・件数変更	POST /students/page	PageRequest、TabId。検索条件フォームの未適用入力は送らない。
クリア	POST /students/clear	ContextId、ContextRevision、TabId
新規登録へ	GET /students/new	returnContextId
編集・参照へ	GET /students/{id:long}	学生ID、returnContextId
遷移先での保存	POST /students/create または /students/{id:long}/update	Input、SnapshotId
遷移先での論理削除	POST /students/{id:long}/delete	削除要求、SnapshotId

[根拠：Web/Controllers/StudentsController.cs:8-27・37-38]

3.1 初期表示

処理概要

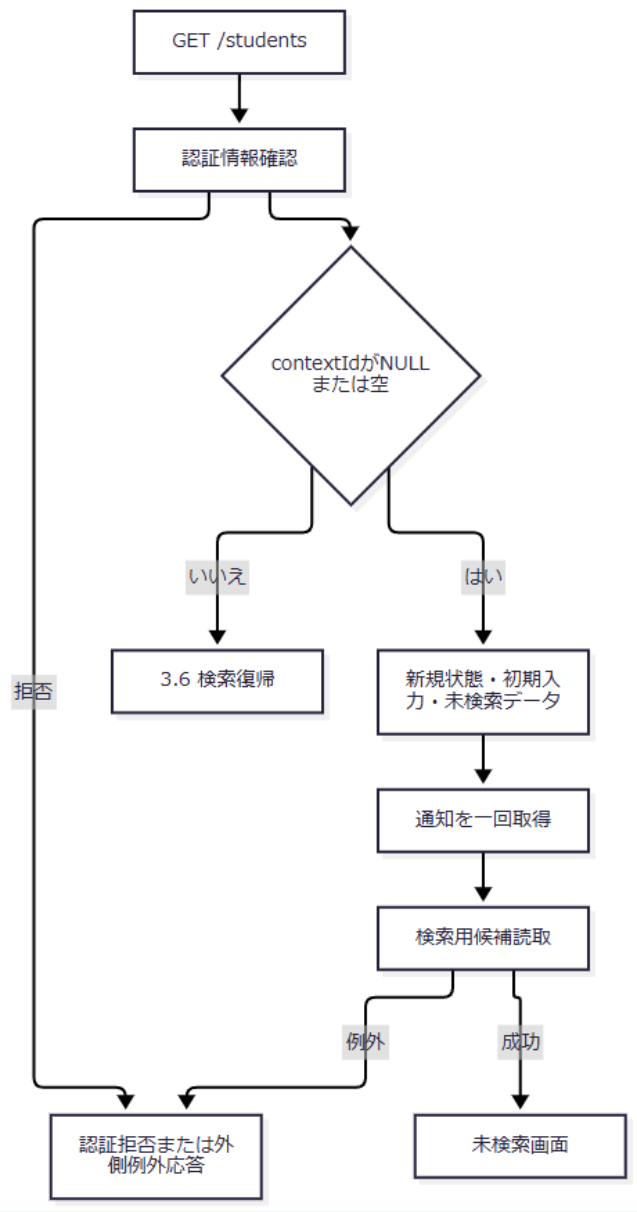
状態IDを指定しないGETは、新しい検索状態と初期条件を作り、学生検索を行わず候補を表示する。contextIdがNULLまたは空文字の場合が対象である。 [根拠：Web/Controllers/StudentsController.cs:8-11]

処理手順

1. 認証情報を確認する。
2. 所有者と認証セッションを付けた検索状態を作る。初期版は1、期限は作成時から30分、Applied・LastResultはNULLとする。
3. Present(entry,null) で未検索の表示データを作る。
4. 初期入力を用意する。在籍区分01・02、未所属false、その他の条件は空または未指定とする。表示件数20、学籍番号昇順、ページ1となる。
5. 現在利用者・認証セッション・検索状態IDに対応する通知を一回取得する。
6. 検索用候補を読み、Search画面を共有レイアウトで表示する。

[根拠：Web/Controllers/StudentsController.cs:8-11・39-39、Web/Inputs/StudentSearchInput.cs:7-22、Infrastructure/SearchContextStore.cs:9-12、Infrastructure/WebStudentCoordinator.cs:21-21]

処理フロー



```
flowchart TD
  A[GET /students] --> B[認証情報確認]
  B -->|拒否| X[認証拒否または外側例外応答]
  B --> C{contextIdがNULLまたは空}
  C -->|いいえ| R[3.6 検索復帰]
  C -->|はい| D[新規状態・初期入力・未検索データ]
  D --> E[通知を一回取得]
  E --> F[検索用候補読取]
  F -->|成功| G[未検索画面]
  F -->|例外| X
```

SQL・テーブル関連（ER図）

m_code、m_code_category、m_class を参照する。候補間の関係は4.1の関連図、候補SQLの結果形・順序は4.2による。m_student の件数判定・一覧取得は行わない。

DB操作の詳細

LookupAsync(false) が別のSnapshotトランザクションでコードとクラスの2結果セットを読む。検索用なので所属人数を集計せず0を設定し、正常読取後にcommitする。検索状態の作成先はメモリであり、DBへのINSERTではない。［根拠：Web/Controllers/StudentsController.cs:39-39、Core/SqlServerStudentService.cs:197-216、Infrastructure/SearchContextStore.cs:10-12］

結果・例外時の扱い

正常時は未検索画面を返す。候補読取等に失敗しても、作成済みメモリ状態を取り消す処理はない。個別catchはなく、例外は4.5の外側処理へ進む。通知を取得した後に表示が失敗した場合、通知は消費済みとなり得る。［根拠：Web/Controllers/StudentsController.cs:8-11、Infrastructure/FlashMessageStore.cs:9-9］

3.2 検索

処理概要

入力中の条件を適用して先頭ページを検索する。検証を通った通常結果、0件、上限超過は検索成功として保持する。サービス検証失敗を0件の成功へ変換しない。［根拠：Web/Inputs/StudentSearchInput.cs:19-22、Infrastructure/WebStudentCoordinator.cs:7-9・20-21］

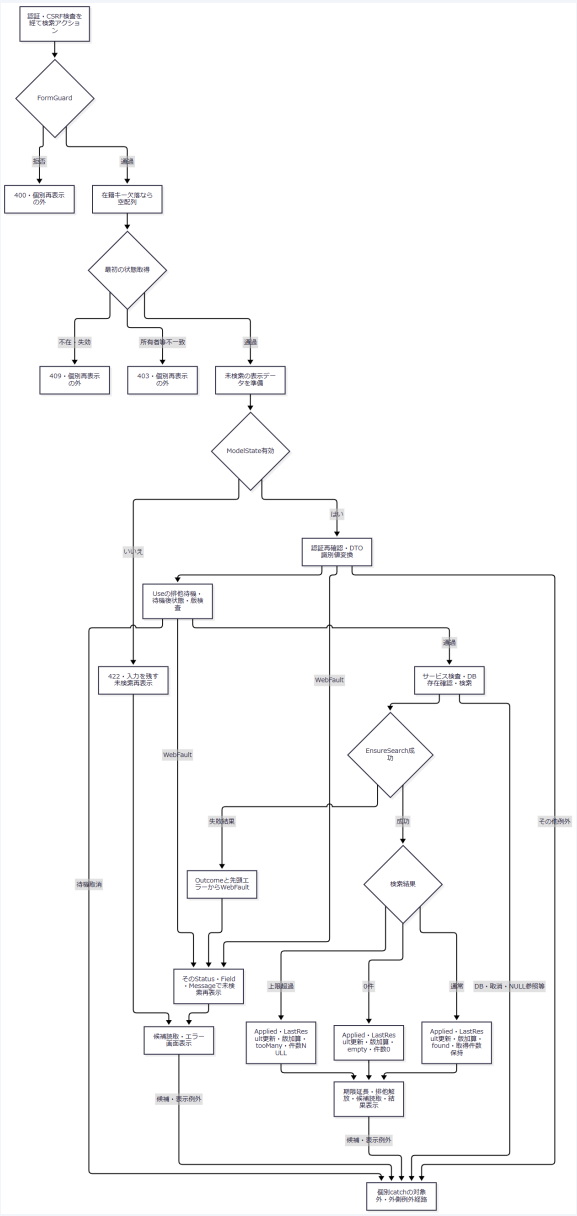
処理手順

1. アクション到達後、FormGuardで許可項目・重複値・書式メタ情報を検査する。在籍区分キーがない場合は空配列へ補正する。
2. 現在利用者と受信TabIdで検索状態を取得し、失敗時用の未検索データを作る。この取得は個別catchの外にある。
3. ModelStateを確認する。不正なら422とし、Coordinatorを呼ばない。
4. Coordinatorで認証を再確認し、入力DTOをSearchCriteriaへ移送する。ClassIdの正のlongへの変換はUseより前に行う。Pageは1とする。
5. Useで状態取得、排他待機、待機後の再確認、受信版の一致、版上限を確認する。
6. サービスでページ・件数・方向、並び順キー、年度範囲・正のクラスID、未所属と所属条件の同時指定、在籍区分の数・重複・許可値を確認する。未所属の同時指定は学年コードを空白正規化する前に判定する。

7. 学籍番号・氏名・カナ・電話・学年・都道府県を正規化し、入学日期間を解析して開始≤終了を確認する。
8. DB接続とSnapshot開始後、学年・都道府県コードの存在、年度候補の存在、クラスの存在と上位条件整合を確認する。検索では停止・期間外を拒否しない。
9. 件数上限を判定し、10,000件以下ならページ分を取得する。commit後に検索結果を返す。
10. EnsureSearchが成功を確認してから、入力由来条件のコピーをApplied、結果をLastResultへ設定し、版を1加算する。正常にUseが戻ると期限を30分延長する。
11. 候補を再読取りして表示する。検索入口は受信PageSize・SortKey・SortDirectionを画面モデルへ残す。

[根拠：Web/Controllers/StudentsController.cs:13-16・39-39、Infrastructure/WebStudentCoordinator.cs:7-9・20-21、Infrastructure/SearchContextStore.cs:14-14、Core/SqlServerStudentService.cs:103-170]

処理フロー




```
flowchart TD
  A[認証・CSRF検査を経て検索アクション] --> B{FormGuard}
  B -->|拒否| X[400・個別再表示の外]
```

```
        B -->|通過| C[在籍キー欠落なら空配列]
        C --> D{最初の状態取得}
        D -->|不在・失効| Y[409・個別再表示の外]
        D -->|所有者等不一致| Z[403・個別再表示の外]
        D -->|通過| E[未検索の表示データを準備]
        E --> F{ModelState有効}
        F -->|いいえ| V[422・入力を残す未検索再表示]
        F -->|はい| G[認証再確認・DTO識別値変換]
        G --> H[Useの排他待機・待機後状態・版検査]
        H -->|待機取消| O[個別catchの対象外・外側例外経路]
        H -->|通過| I[サービス検査・DB存在確認・検索]
        I --> J{EnsureSearch成功}
        J -->|失敗結果| W[Outcomeと先頭エラーからWebFault]
        J -->|成功| K{検索結果}
        K -->|上限超過| K1[Applied・LastResult更新・版加算・tooMany・件数NULL]
        K -->|0件| K2[Applied・LastResult更新・版加算・empty・件数0]
        K -->|通常| K3[Applied・LastResult更新・版加算・found・取得件数保持]
        G -->|WebFault| W1[そのStatus・Field・Messageで未検索再表示]
        H -->|WebFault| W1
        W --> W1
        K1 --> L[期限延長・排他解放・候補読取・結果表示]
        K2 --> L
        K3 --> L
        V --> M[候補読取・エラー画面表示]
        W1 --> M
        I -->|DB・取消・NULL参照等| O
        G -->|その他例外| O
        L -->|候補・表示例外| O
        M -->|候補・表示例外| O
```

Use取得後の例外ではfinallyで排他を解放する。解放は応答やエラーコードを固定する合流点ではなく、元の例外を引き継ぐ。WebFault再表示の枝も固定422にはせず、識別値不正422、状態・版の理由別403・409等を保持する。［根拠：Infrastructure/SearchContextStore.cs:14-14、Web/Inputs/StudentSaveInput.cs:59-60、Web/Controllers/StudentsController.cs:15-15］

SQL・テーブル関連（ER図）

学生をクラスへLEFT JOINし、一覧ではさらに学年・在籍区分・都道府県コードへLEFT JOINする。関連は4.1の図に示す。件数SQLのJOINはクラスだけで、名称取得用JOINは一覧SQLに追加する。［根拠：Core/SqlServerStudentService.cs:82-90・156-162］

DB操作の詳細

次の述語をANDで結合する。未指定条件は追加しない。電話比較だけは一つの括弧内のORである。

条件	SQL比較・加工	SQLパラメータ
常時	s.is_deleted=0	なし
学籍番号あり	s.student_no LIKE @no ESCAPE '!'。加工値末尾に%	@no:VarChar(31)
氏名キーあり	s.name_key LIKE @name ESCAPE '!'。前後に%	@name:NVarChar(642)
カナキーあり	s.kana_key LIKE @kana ESCAPE '!'。前後に%	@kana:NVarChar(74)
年度あり	c.school_year=@year	@year:SmallInt
学年あり	c.grade_code=@grade	@grade:VarChar(2)
クラスあり	s.class_id=@class	@class:BigInt
未所属true	s.class_id IS NULL	なし
都道府県あり	s.prefecture_code=@prefecture	@prefecture:VarChar(2)
電話あり	(s.phone_number=@phone OR s.emergency_phone_number=@phone)	@phone:VarChar(16)
開始日あり	s.admission_date>=@from	@from:Date
終了日あり	s.admission_date<=@to	@to:Date
在籍配列が非空	s.enrollment_status_code IN (...)	各 @statusN:VarChar(2)

LIKE値内の !、%、_、[はEscapeLikeでエスケープする。パラメータhelperはNULLをDBNull.Valueへ、DateOnlyを日付の午前0時のDateTimeへ変換する。 [根拠：Core/SqlServerStudentService.cs:67-73・137-155、Core/StudentNormalizer.cs:161-161]

存在確認では、コードは m_code の分類・値のCOUNT_BIG=1を要求し、@category:VarChar(24)・@value:VarChar(10) を使用する。年度は m_class.school_year=@year のCOUNT_BIGが0でないことを要求する。クラスは class_id=@id、@id:BigInt で年度・学年を読み、指定上位条件との一致を確認する。 [根拠：Core/SqlServerStudentService.cs:125-135・218-222]

件数判定は全検索条件を適用した学生IDのTOP(10001)をCOUNT_BIGする。10001なら一覧SQLを実行せずcommitし、NoChange・空一覧・LimitExceeded=trueを返す。10,000以下なら最終ページを最低1とし、サービス内部の取得ページを最終ページ以下へ補正する。許可した4列・方向と student_id ASC によるORDER BY、OFFSET/FETCH、@offset:Int・@size:Int で最大表示件数分を取得する。読取後commitし、NoChange・件数・StudentRow一覧・検索開始時刻・LimitExceeded=false・空Errorsを返す。 [根拠：Core/SqlServerStudentService.cs:156-168]

存在確認・件数判定・一覧取得は一つのSnapshotトランザクション内で行い、DB更新は行わない。接続・タイムアウト・取消・資源破棄の共通契約は4.5による。候補の再読取は、この検索トランザクションとは別である。

結果・例外時の扱い

サービスはtry内でInputValidationExceptionを先に捕捉し、そのErrorsをValidationFailedへ引き継ぐ。続くArgumentExceptionはArgumentErrorでFieldErrorへ変換する。規定形式の先頭コードは保持し、それ以外はE-STU-002にする。検証失敗のCount=0・Items空は成功0件ではなく、EnsureSearchでWebFaultへ変換する。User確認はtry前であり、DB・取消・NULL参照等をこの検証結果へ一般化しない。［根拠：Core/SqlServerStudentService.cs:105-106・169-170、Core/StudentNormalizer.cs:162-168、Infrastructure/WebStudentCoordinator.cs:20-20］

ModelState不正またはCoordinator内のWebFaultでは、入力を残して未検索再表示する。過去のApplied・LastResultを削除しないため、未検索表示でも以前の検索条件・結果がメモリに残る場合がある。表示データのページ等は既存Applied由来で、受信値を残した画面モデルと一致するとは限らない。［根拠：Web/Controllers/StudentsController.cs:14-16、Infrastructure/WebStudentCoordinator.cs:21-21］

FormGuardと最初の状態取得の拒否は個別catchへ到達しない。サービス成功後の候補読取・表示変換失敗も個別catchの外であり、更新済みApplied・LastResult・Revisionを取り消さない。応答未開始の場合だけ4.5の外側例外応答へ進む。［根拠：Web/Controllers/StudentsController.cs:14-16・39-39、Infrastructure/WebHostSetup.cs:23-24］

3.3 候補更新・所属条件の連動

処理概要

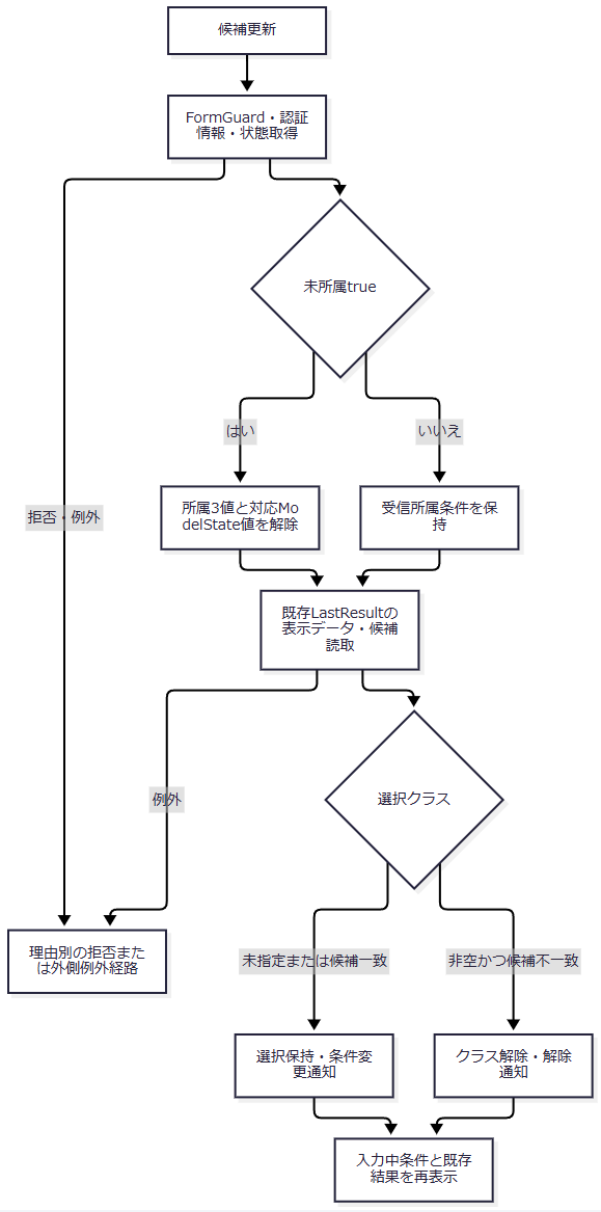
年度・学年に合わせてクラス候補を表示し、上位条件に合わない選択を解除する。入力中の条件を一覧へ適用する操作ではない。［根拠：Web/Controllers/StudentsController.cs:22-22］

処理手順

1. FormGuardを実行し、在籍区分キー欠落を空配列にする。
2. 現在利用者・セッション・非空Tabで検索状態を取得する。
3. 未所属trueなら年度・学年・クラスをNULLへ変更し、対応するModelState値も解除する。
4. 既存LastResultから表示データを作り、検索用候補を読む。
5. 非空の選択クラスが、受信年度・学年に合う候補に存在するか確認する。候補IDの文字列表現と受信値を比較し、IdParserによる数値化は行わない。
6. 不一致ならClassIdとそのModelState値を解除する。
7. 条件と既存一覧を再表示する。通常は「条件を変更しました。検索を押すと一覧に反映します。」、クラス解除時は「上位条件に合わないクラスを解除しました。」を表示する。

［根拠：Web/Controllers/StudentsController.cs:22-22・39-39］

処理フロー




```
flowchart TD
  A[候補更新] --> B[FormGuard・認証情報・状態取得]
  B -->|拒否・例外| X[理由別の拒否または外側例外経路]
```

```
        B --> C{未所属true}
        C -->|はい| D[所属3値と対応ModelState値を解除]
        C -->|いいえ| E[受信所属条件を保持]
        D --> F[既存LastResultの表示データ・候補読取]
        E --> F
        F -->|例外| X
        F --> G{選択クラス}
        G -->|未指定または候補一致| H[選択保持・条件変更通知]
        G -->|非空かつ候補不一致| I[クラス解除・解除通知]
        H --> J[入力中条件と既存結果を再表示]
        I --> J
```

SQL・テーブル関連（ER図）

4.1の候補関連、4.2の候補SQLを使用する。学生の検索SQLは行わない。停止・期間外の候補も検索用に取得する。

DB操作の詳細

LookupAsync(false) の2結果セットを読み、取得後のビューとコントローラーで年度・学年を使ってクラス候補を絞る。SQLに年度・学年の候補抽出パラメータはない。Applied・LastResult・Revisionは更新しない。Useを通らないため受信ContextRevisionの比較、Gateによる直列化、期限延長も行わない。［根拠：Core/SqlServerStudentService.cs:197-216、Web/Controllers/StudentsController.cs:22-22、Views/Students/Search.cshtml:5-5］

結果・例外時の扱い

ModelState.IsValidによる実行抑止はない。受信した件数・並び順も許可項目だが、SearchModelは適用済み値または初期値を設定する。候補読取等のWebFaultを処理する個別catchはなく、失敗は外側へ進む。［根拠：Web/Controllers/StudentsController.cs:22-22・39-39］

ブラウザーの未所属チェック変更と初期処理は、チェック済みなら所属3コントロールのvalueを空にする。disabled切替、自動候補更新、自動検索は行わない。disabledはサーバー再描画時のビューが設定する。検索画面の年度・学年変更には、編集画面REG-016〜018向けの即時クラス解除処理を適用しない。［根拠：wwwroot/js/site.js:7-8、Views/Students/Search.cshtml:5-5］

3.4 ページ・並び順・表示件数の変更

処理概要

適用済み条件で再検索する。入力中で未適用の検索条件は使用しない。件数・キー・方向の変更は先頭ページに移動し、通常のページ移動は要求ページを使用する。［根拠：Infrastructure/WebStudentCoordinator.cs:11-12、Views/Students/Search.cshtml:8-8］

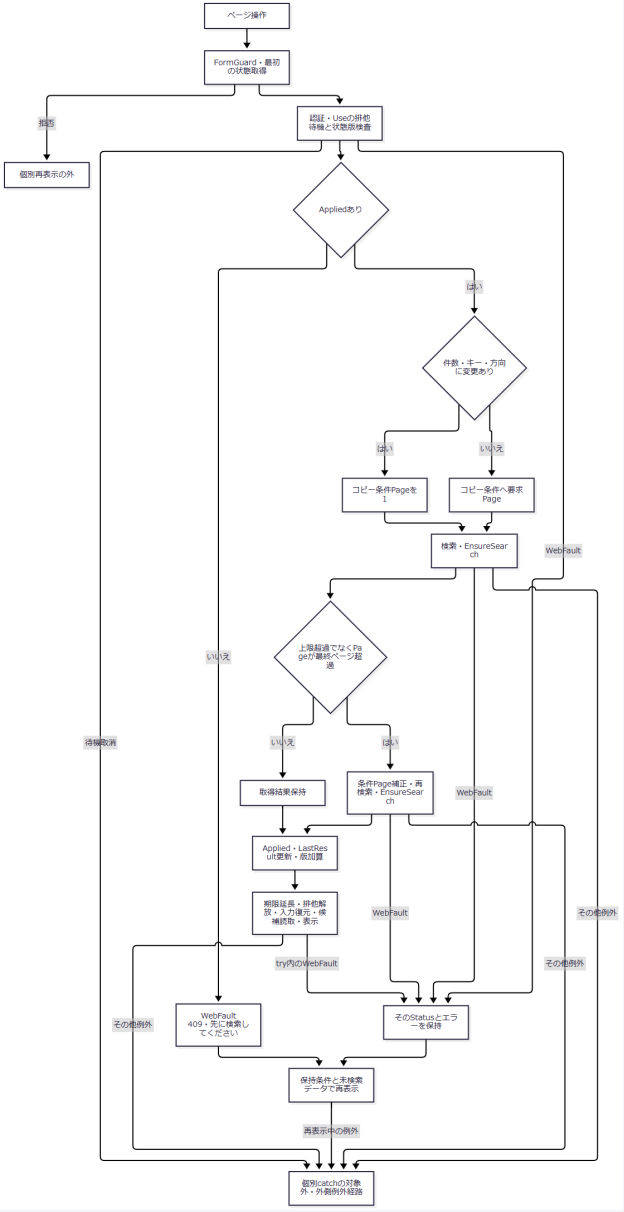
処理手順

1. FormGuard後、現在利用者とTabIdで状態を取得する。この取得は個別catchの外である。
2. Coordinatorで認証とUseの排他・待機後状態・版検査を行う。

3. AppliedがNULLなら409「先に検索してください。」とする。
4. Appliedをコピーし、要求件数・キー・方向を設定する。いずれかが変更された場合はPage=1、それ以外は要求Pageとする。
5. 検索してEnsureSearchで成否を確認する。
6. 上限超過でなく、条件のPageが最終ページを超えていれば、条件Pageを最終ページへ補正し再検索する。最終ページは最低1であり、0件結果もこの判定対象になる。
7. 最終検索の成功後、条件コピーをApplied、結果をLastResultへ設定し、版を1加算する。
8. 保持条件から入力を復元し、候補を読んで表示する。

[根拠：Web/Controllers/StudentsController.cs:18-19、Infrastructure/WebStudentCoordinator.cs:11-12]

処理フロー




```
flowchart TD
  A[ページ操作] --> B[FormGuard・最初の状態取得]
  B -->|拒否| X[個別再表示の外]
```

```
        B --> C[認証・Useの排他待機と状態版検査]
        C -->|待機取消| O[個別catchの対象外・外側例外経路]
        C --> D{Appliedあり}
        D -->|いいえ| W[WebFault 409・先に検索してください]
        D -->|はい| E{件数・キー・方向に変更あり}
        E -->|はい| F[コピー条件Pageを1]
        E -->|いいえ| G[コピー条件へ要求Page]
        F --> H[検索・EnsureSearch]
        G --> H
        H --> I{上限超過でなくPageが最終ページ超過}
        I -->|はい| J[条件Page補正・再検索・EnsureSearch]
        I -->|いいえ| K[取得結果保持]
        J --> L[Applied・LastResult更新・版加算]
        K --> L
        L --> M[期限延長・排他解放・入力復元・候補読取・表示]
        C -->|WebFault| W2[そのStatusとエラーを保持]
        H -->|WebFault| W2
        J -->|WebFault| W2
        M -->|try内のWebFault| W2
        W --> N[保持条件と未検索データで再表示]
        W2 --> N
        H -->|その他例外| O
        J -->|その他例外| O
        M -->|その他例外| O
        N -->|再表示中の例外| O
```

SQL・テーブル関連（ER図）

検索対象とJOINは3.2、関係図は4.1による。候補表示には4.2の参照を追加する。

DB操作の詳細

通常はSearchAsyncを1回呼び、最終ページ超過の補正時はさらに1回呼ぶ。それぞれ独立したSnapshot読取であり、2回目の検索結果を最終結果として保持する。DB更新は行わず、成功後の更新先はメモリ状態である。［根拠：Infrastructure/WebStudentCoordinator.cs:12-12、Core/SqlServerStudentService.cs:123-168］

結果・例外時の扱い

この入口はModelState.IsValidで実行を抑止しない。サービス引数の検査と、MVCの変換成否を同一視しない。個別catchはCoordinatorだけでなく、try内の入力復元・SearchModel作成も含み、WebFaultのStatusを引き継いで保持条件と未検索データで再表示する。catch内の再表示が失敗した場合は外側へ進む。成功後の表示失敗でも更新済み状態は取り消さない。［根拠：Web/Controllers/StudentsController.cs:19-19］

3.5 クリア

処理概要

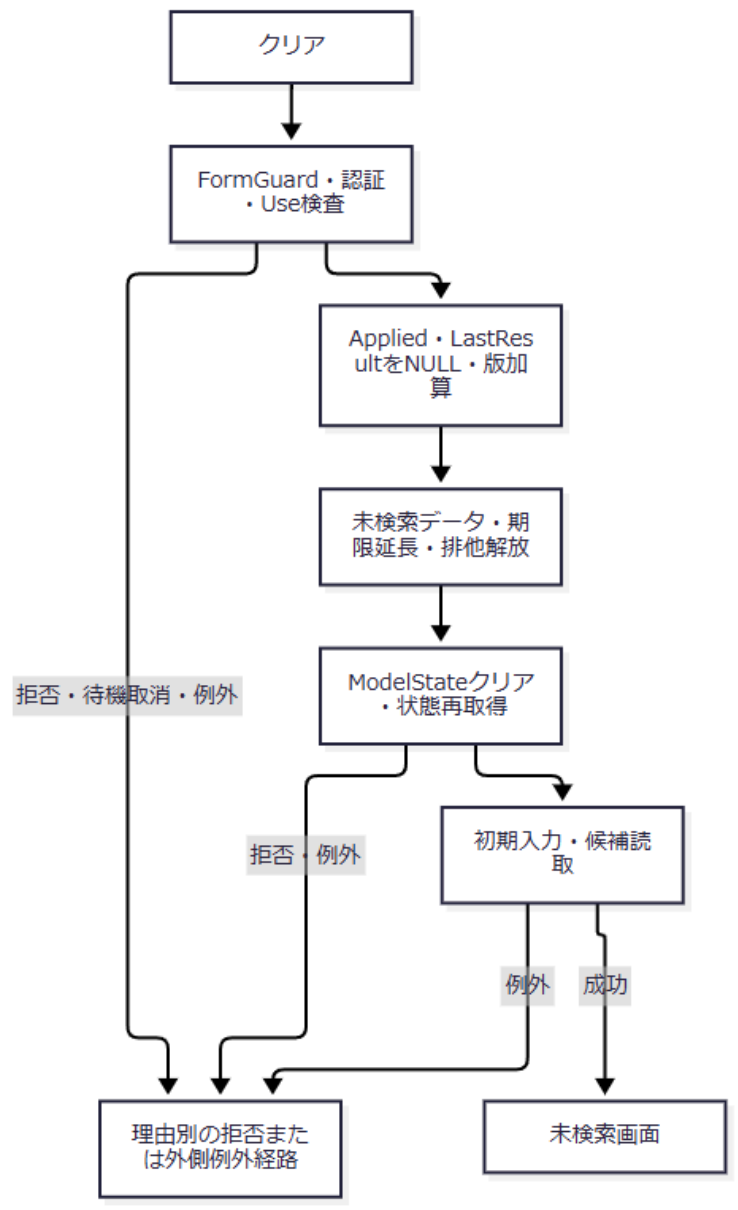
検索状態IDとTabを維持し、適用済み条件と直近結果を解除して初期入力へ戻す。［根拠：Infrastructure/WebStudentCoordinator.cs:15-15］

処理手順

1. FormGuardと認証確認を行う。
2. Useで状態取得、排他待機、待機後再確認、版一致・上限を検査する。
3. Applied・LastResultをNULLにし、版を1加算して未検索データを返す。
4. 正常完了時に期限を延長し、取得済み排他を解放する。
5. ModelStateをクリアし、検索状態を再取得する。
6. 初期入力と再読取候補で表示する。在籍01・02、20件、学籍番号昇順、ページ1へ戻す。

［根拠：Web/Controllers/StudentsController.cs:21-21・39-39、Infrastructure/WebStudentCoordinator.cs:15-15、Infrastructure/SearchContextStore.cs:14-14、Web/Inputs/StudentSearchInput.cs:7-22］

処理フロー




```
flowchart TD
  A[クリア] --> B[FormGuard・認証・Use検査]
  B -->|拒否・待機取消・例外| X[理由別の拒否または外側例外経路]
```

```
        B --> C[Applied・LastResultをNULL・版加算]
        C --> D[未検索データ・期限延長・排他解放]
        D --> E[ModelStateクリア・状態再取得]
        E -->|拒否・例外| X
        E --> F[初期入力・候補読取]
        F -->|成功| G[未検索画面]
        F -->|例外| X
```

SQL・テーブル関連（ER図）

学生検索は行わず、4.1・4.2の候補関連だけを使用する。解除対象はメモリのApplied・LastResultであり、学生表の削除ではない。

DB操作の詳細

候補のSnapshot読取のみを行う。メモリ条件・結果のNULL設定と版加算は候補読取より先に完了する。 [根拠：Web/Controllers/StudentsController.cs:21-21・39-39]

結果・例外時の扱い

状態・版の拒否、排他待機取消、候補失敗を処理する個別catchはない。クリア後の状態再取得や候補表示が失敗しても、解除した条件・結果を復元しない。 [根拠：Web/Controllers/StudentsController.cs:21-21、Infrastructure/WebHostSetup.cs:23-24]

3.6 検索復帰

処理概要

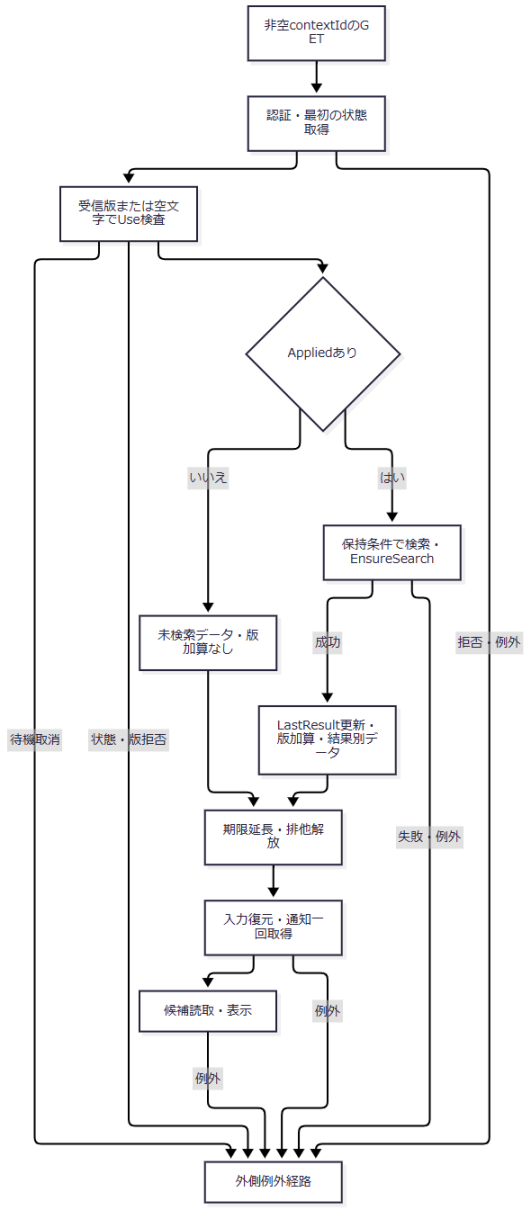
非空contextIdを伴うGETは保持条件で一覧を取り直す。編集フォームの未保存値から検索条件を作らない。 [根拠：Web/Controllers/StudentsController.cs:8-11、Infrastructure/WebStudentCoordinator.cs:14-14]

処理手順

1. 認証確認後、復帰先状態を取得する。このGETではTabを渡さない。
2. 受信contextRevisionまたは空文字を使ってResumeを呼ぶ。
3. Useで排他待機、待機後状態、版一致・上限を確認する。
4. AppliedがNULLなら未検索データを返し、版を加算しない。正常完了による期限延長は行う。
5. Appliedがあれば保持条件で検索し、EnsureSearch成功後にLastResultを更新して版を1加算する。
6. 保持条件から入力を復元し、通知を一回取得する。
7. 候補を読んで結果を表示する。

[根拠：Web/Controllers/StudentsController.cs:8-11・40-40、Infrastructure/WebStudentCoordinator.cs:14-14、Infrastructure/FlashMessageStore.cs:9-9]

処理フロー




```
flowchart TD
  A[非空contextIdのGET] --> B[認証・最初の状態取得]
  B --> C[受信版または空文字でUse検査]
```

```
        C -->|待機取消| X[外側例外経路]
        C --> D{Appliedあり}
        D -->|いいえ| E[未検索データ・版加算なし]
        D -->|はい| F[保持条件で検索・EnsureSearch]
        F -->|成功| G[LastResult更新・版加算・結果別データ]
        E --> H[期限延長・排他解放]
        G --> H
        H --> I[入力復元・通知一回取得]
        I --> J[候補読取・表示]
        B -->|拒否・例外| X
        C -->|状態・版拒否| X
        F -->|失敗・例外| X
        I -->|例外| X
        J -->|例外| X
```

SQL・テーブル関連（ER図）

Appliedなしの場合は4.2の候補読取だけ、Appliedありの場合は3.2の学生検索と候補読取を行う。関連図は4.1による。

DB操作の詳細

Appliedがあるときの検索は保持したPage・PageSize・並び順をそのままSearchAsyncへ渡す。サービスは取得ページを内部補正するが、ResumeはApplied.Pageを修正しない。SearchResultにも補正ページがなく、画面はApplied.Pageを表示する。そのため保存・削除で件数が減った復帰時には、最終ページの一覧を取得していても、ページ表示が古い値のままとなる可能性がある。通常ページ操作の保持ページ補正とは異なる。［根拠：Infrastructure/WebStudentCoordinator.cs:12-14・21-21、Core/SqlServerStudentService.cs:160-168、Core/Contracts.cs:57-58］

結果・例外時の扱い

GET内に個別catchはない。版不一致、失効、再検索失敗は通知取得前に外側へ進む。通知は10分期限でTakeがキーを削除する一回取得であり、取得後の候補・表示失敗では通知を消費したまま表示できない場合がある。［根拠：Web/Controllers/StudentsController.cs:8-11、Infrastructure/FlashMessageStore.cs:8-9］

3.7 登録・編集・参照への遷移と戻る操作

処理概要

新規登録と行リンクは検索状態IDをreturnContextIdとして渡す。検索状態版はリンクに含めず、遷移先の読込みで現在版を取得する。戻る操作はその読込時版を使って検索復帰する。［根拠：Views/Students/Search.cshtml:6-8、Web/Controllers/StudentsController.cs:23-25、Views/Students/Edit.cshtml:16-16］

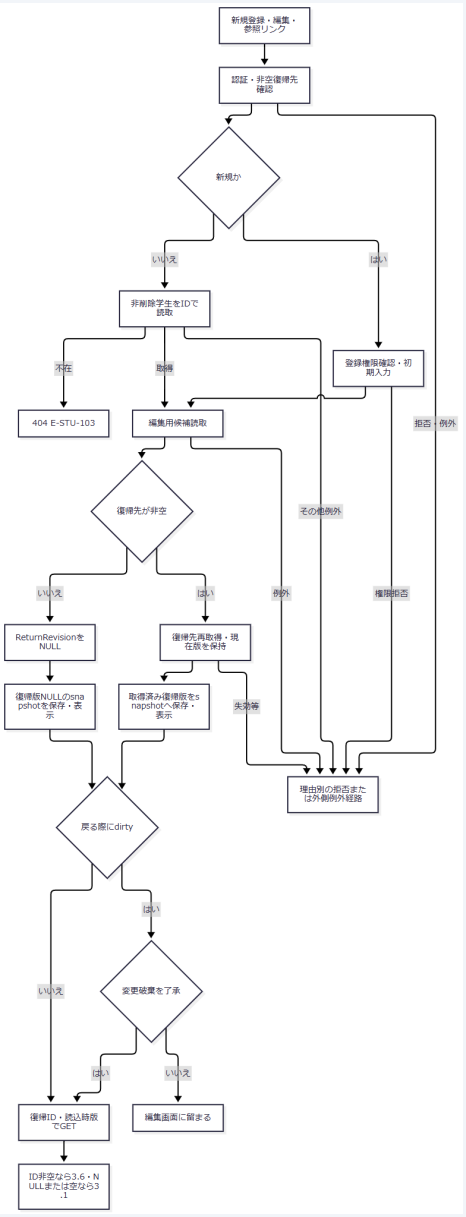
処理手順

1. 遷移先で認証と復帰先の所有者・セッション・期限を確認する。新規は登録権限も確認する。
2. 既存学生は学生IDと非削除条件で読む。不在なら404、E-STU-103とする。

3. 編集用候補を読み、明細から入力を作る。編集用候補は所属人数も集計する。
4. 候補読取後、非空の復帰先を再取得して現在版をReturnRevisionへ設定する。復帰先NULL・空ではReturnRevisionもNULLとする。
5. 所有者・セッション・対象ID・読込明細・入力コピー・ReturnRevisionを30分期限の編集スナップショットへ保存し、Edit画面を表示する。
6. 「検索へ戻る」はReturnContextIdと読込時ReturnRevisionをGETへ渡す。dirty状態で破棄確認を取り消した場合は画面に留まる。

[根拠：Infrastructure/WebStudentCoordinator.cs:6-6・17-17、Web/Controllers/StudentsController.cs:23-25、Infrastructure/EditorSnapshotStore.cs:7-10、Views/Students/Edit.cshtml:16-16、wwwroot/js/site.js:2-5]

処理フロー




```
flowchart TD
  A[新規登録・編集・参照リンク] --> B[認証・非空復帰先確認]
  B --> C{新規か}
```

```
        C -->|はい| D[登録権限確認・初期入力]
        C -->|いいえ| E[非削除学生をIDで読取]
        E -->|不在| X[404 E-STU-103]
        E -->|取得| F[編集用候補読取]
        D --> F
        F --> G{復帰先が非空}
        G -->|はい| H[復帰先再取得・現在版を保持]
        G -->|いいえ| HO[ReturnRevisionをNULL]
        H --> I[取得済み復帰版をsnapshotへ保存・表示]
        HO --> IO[復帰版NULLのsnapshotを保存・表示]
        I --> J{戻る際にdirty}
        IO --> J
        J -->|いいえ| R[復帰ID・読込時版でGET]
        J -->|はい| K{変更破棄を了承}
        K -->|はい| R
        K -->|いいえ| L[編集画面に留まる]
        R --> S[ID非空なら3.6・NULLまたは空なら3.1]
        B -->|拒否・例外| O[理由別の拒否または外側例外経路]
        D -->|権限拒否| O
        E -->|その他例外| O
        F -->|例外| O
        H -->|失効等| O
```

復帰版を取得できた枝へNULLを代入しない。非空復帰先の版と、復帰先なしのNULLを枝ごとに保持する。

SQL・テーブル関連（ER図）

学生単体読取と4.1・4.2の候補関連を使用する。検索状態・編集スナップショットとの関係はメモリ上の参照であり、DBのFKではない。

DB操作の詳細

既存学生は SELECT * FROM dbo.m_student WHERE student_id=@id AND is_deleted=0、@id:BigInt で最大1行の明細を読む。GetAsyncは正でないIDならNULLを返す。読込SQLには明示トランザクションを付けない。候補は別のSnapshotトランザクションで取得し、編集用では非削除かつ在籍01・02のクラス人数を集計する。 [根拠：Core/SqlServerStudentService.cs:172-181・196-216]

結果・例外時の扱い

Viewerも既存学生を読み込めるが、編集画面のfieldsetは無効化され、サーバー側でも保存を拒否する。初回の復帰先確認成功だけで後続表示を保証せず、候補await後の再取得で失効等を拒否する。 [根拠：Web/Controllers/StudentsController.cs:25-25、Infrastructure/RequestIdentity.cs:14-15、Web/Models/StudentEditPageModel.cs:14-14、Views/Students/Edit.cshtml:5-5]

編集フォームのinputイベントはdirtyを立て、確認情報を空にする。dirtyな戻る・再読込リンクは破棄確認を行い、取消なら遷移を止める。beforeunloadもdirtyかつ離脱確定前に警告する。これらは保存操作ではない。「入力クリア／編集取消」のPOSTは読込時入力へ戻す操作であり、検索画面のクリアとは異なる。［根拠：wwwroot/js/site.js:2-5、Views/Students/Edit.cshtml:16-16、Web/Controllers/StudentsController.cs:35-35］

3.8 保存結果から検索への復帰

処理概要

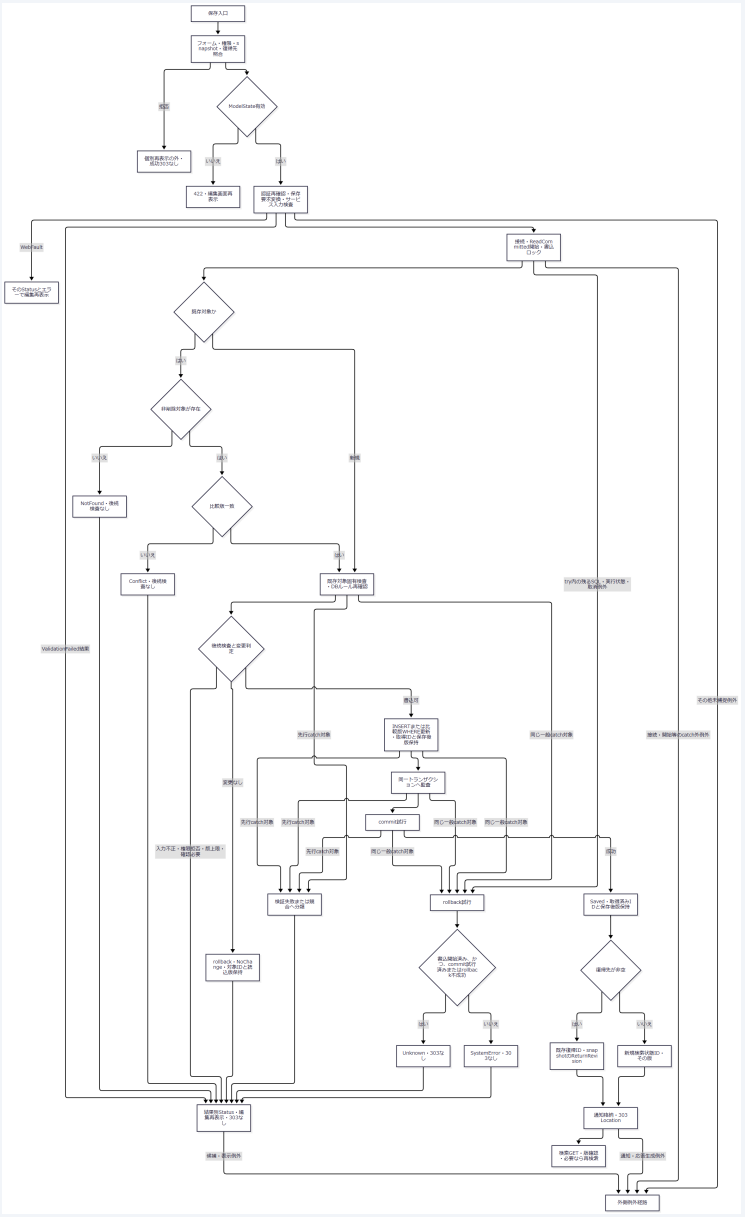
遷移先の保存がSavedとなった場合だけ、通知を格納して検索GETへ303を返す。本節は保存の確定境界と検索への影響を扱い、登録画面の全項目仕様は再掲しない。［根拠：Web/Controllers/StudentsController.cs:28-38］

処理手順

1. FormGuard、登録・更新権限、スナップショットの所有者・セッション・対象を確認する。
2. 復帰先を確認し、受信ReturnContextIdとsnapshot.Original.ReturnContextIdが一致することを確認する。ここまでの拒否は保存入口の個別catchの外である。
3. ModelState不正なら422で編集画面へ戻す。有効ならCoordinatorで権限・復帰先を再確認し、入力を保存要求へ変換する。
4. サービスで正規化、理由、IDと比較版の組、年度・学年を検査する。接続・ReadCommitted開始後、学生書込用の排他アプリケーションロックを取得する。
5. 既存対象を非削除条件で再読取りする。不在はNotFound、比較版不一致はConflictを直ちに返し、後続のルール確認・書込みへ進まない。学籍番号変更を拒否し、終了区分訂正には管理者権限と理由を要求する。
6. DBコード・クラスを読み直して保存ルールを確認する。新規・変更コードは有効状態と業務日内の期間を要求する。既存の同じ停止値は存在確認の上で保持できる。新たな在籍01・02の所属はクラスの稼働状態・期間・本人を除く所属人数による定員を確認し、終了区分の変更期間はクラス期間との重なりを確認する。
7. 変更なしは明示rollbackしてNoChangeとする。変更ありは版上限、削除行を含む学籍番号重複、非削除の同一人物候補を確認する。同一人物の確認が必要ならConfirmationRequiredとし、成功復帰へ進まない。
8. 新規は正規化列・固定分類・作成更新情報・初期版1をINSERTする。更新は学籍番号を変更列から除き、学生ID・比較版・非削除のWHEREで更新する。取得IDがなければ競合例外とする。
9. 同じトランザクションへ監査を書き、commit成功後にSavedと取得ID・保存後版を返す。
10. Savedだけを成功復帰とする。復帰先が非空ならsnapshotのReturnRevisionを使い、NULL・空なら新しい検索状態とその版を作る。「保存しました。」を格納し、303と検索GETのLocationを返す。
11. 後続GETは3.6または3.1を実行し、適用済み条件があれば再検索する。

［根拠：Web/Controllers/StudentsController.cs:29-38、Infrastructure/WebStudentCoordinator.cs:18-18、Core/SqlServerStudentService.cs:227-323・344-383］

処理フロー




```
flowchart TD
  A[保存入口] --> B[フォーム・権限・snapshot・復帰先照合]
  B -->|拒否| X[個別再表示の外・成功303なし]
```

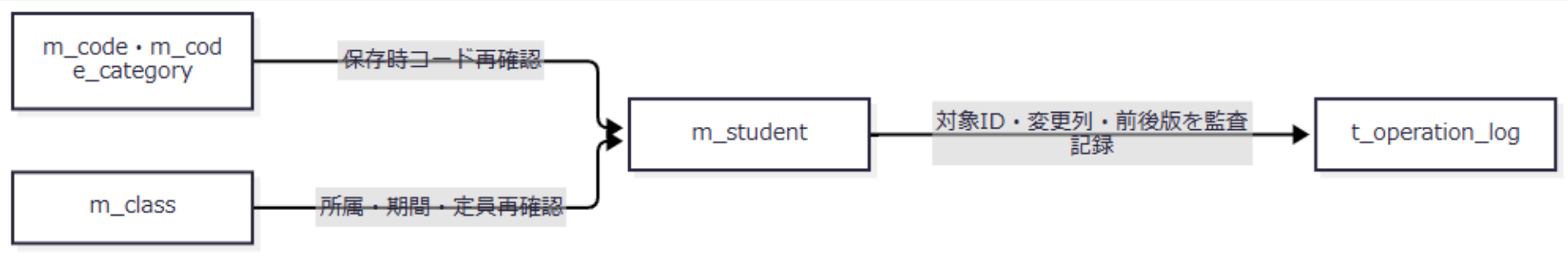
```
B --> C{ModelState有効}
C -->|いいえ| V[422・編集画面再表示]
C -->|はい| D[認証再確認・保存要求変換・サービス入力検査]
D -->|WebFault| V1[そのStatusとエラーで編集再表示]
D -->|ValidationFailed結果| N[結果別Status・編集再表示・303なし]
D --> E[接続・ReadCommitted開始・書込ロック]
E --> F{既存対象か}
F -->|はい| G{非削除対象が存在}
G -->|いいえ| NF[NotFound・後続検査なし]
G -->|はい| H{比較版一致}
H -->|いいえ| CF[Conflict・後続検査なし]
H -->|はい| I[既存対象固有検査・DBルール再確認]
F -->|新規| I
I --> J{後続検査と変更判定}
J -->|入力不正・権限拒否・版上限・確認必要| N
J -->|変更なし| NC[rollback・NoChange・対象IDと読込版保持]
J -->|書込可| K[INSERTまたは比較版WHERE更新・取得IDと保存後版保持]
K --> L[同一トランザクションへ監査]
L --> M[commit試行]
M -->|成功| S[Saved・取得済みIDと保存後版保持]
S --> R{復帰先が非空}
R -->|はい| R1[既存復帰ID・snapshotのReturnRevision]
R -->|いいえ| R2[新規検索状態ID・その版]
R1 --> T[通知格納・303 Location]
R2 --> T
T --> U[検索GET・版確認・必要なら再検索]
NF --> N
CF --> N
NC --> N
I -->|先行catch対象| P[検証失敗または競合へ分類]
K -->|先行catch対象| P
L -->|先行catch対象| P
M -->|先行catch対象| P
P --> N
E -->|try内の残るSQL・実行状態・取消例外| Q[rollback試行]
I -->|同じ一般catch対象| Q
K -->|同じ一般catch対象| Q
L -->|同じ一般catch対象| Q
M -->|同じ一般catch対象| Q
Q --> Q1{書込開始済み、かつ、commit試行済みまたはrollback不成功}
Q1 -->|はい| UN[Unknown・303なし]
Q1 -->|いいえ| SE[SystemError・303なし]
UN --> N
SE --> N
```

```
        E -->|接続・開始等のcatch外例外| O[外側例外経路]
        D -->|その他未捕捉例外| O
        T -->|通知・応答生成例外| O
        N -->|候補・表示例外| O
```

判定Q1は「書込開始済み、かつ（commit試行済み、またはrollback不成功）」の意味である。書込未開始なら、rollback不成功でもこの一般catchの結果はSystemErrorとなる。先行catchが処理する例外へこの判定を適用しない。［根拠：Core/SqlServerStudentService.cs:316-323］

SQL・テーブル関連（ER図）

学生保存の関連は4.1の学生・クラス・コードである。監査の t_operation_log は対象表名・対象キーを記録するが、ここで学生とのFKを追加しない。




```
flowchart LR
  C[m_code・m_code_category] -->|保存時コード再確認| S[m_student]
  K[m_class] -->|所属・期間・定員再確認| S
  S -->|対象ID・変更列・前後版を監査記録| L[t_operation_log]
```

この図は処理上の参照・書込み関係であり、FKの多重度を示す図ではない。［根拠：Core/SqlServerStudentService.cs:344-390］

DB操作の詳細

書込ロックはトランザクション所有のExclusiveなアプリケーションロックで、取得待機は5000ms、負の結果はSQL例外とする。既存読取は学生ID・非削除、更新WHEREはさらに比較版を含める。保存列の型・サイズは項目一覧の対応とサービスのColumns定義を使用し、更新者は @actor:NVarChar(128)、処理時刻は @now:DateTimeOffset、ID・比較版はBigIntである。［根拠：Core/SqlServerStudentService.cs:30-37・92-101・246-260・294-305］

監査はCREATEまたはUPDATE、対象ID、変更列JSON、変更前後版、実施者・時刻、理由、同一人物確認有無、受付識別情報を同じトランザクションへINSERTする。正常commit後のSavedは、新規で取得ID・版1、更新で取得ID・読込版+1を返す。NoChangeは要求対象ID・読込版を保持し、ConfirmationRequiredも要求対象ID・読込版と候補情報を返す。これらを成功節点でNULLへ置き換えない。［根拠：Core/SqlServerStudentService.cs:262-262・287-315・385-390］

結果・例外時の扱い

保存トランザクション内のcatchは次の順序である。

順序	捕捉対象と結果
1	InputValidationException。rollbackを試み、ErrorsをValidationFailedへ引き継ぐ。
2	番号2601・2627のSqlException。rollbackを試み、学籍番号使用済みのValidationFailedを返す。
3	DBConcurrencyException。rollbackを試み、Conflictを返す。
4	残るSqlException・InvalidOperationException・OperationCanceledException。rollback結果と書込開始・commit試行からUnknownまたはSystemErrorを返す。

一般catchだけが、書込開始済み、かつ（commit試行済み、またはrollback不成功）をUnknownとする。Unknownは再保存前の検索確認を促し、成功303を返さない。接続・トランザクション開始はこのtryの前にあり、その失敗が同じSaveResultになるとは限らない。早期NotFound・Conflict等の返却枝には明示rollbackがなく、資源破棄へ進む。［根拠：Core/SqlServerStudentService.cs:240-252・316-329］

Saved以外は成功復帰しない。NoChangeは200、ValidationFailedは422、Forbiddenは403、NotFoundは404、Conflict・ConfirmationRequiredは409、SystemError・Unknownは503を結果から設定し、編集画面を再表示する。Coordinator内WebFaultはそのStatusを保持する。［根拠：Web/Controllers/StudentsController.cs:30-33、Infrastructure/WebPresentation.cs:23-23］

DBのSaved、303発行、検索復帰、通知表示は別の到達段階である。保存前に復帰先を確認するが、DB完了後の期限再確認と版の差替えは行わない。別操作が検索版を進めた場合、Savedでも後続GETが409となり得る。303前の通知格納でも現在認証情報を取得するため、DB確定だけで303・通知表示まで保証しない。［根拠：Infrastructure/WebStudentCoordinator.cs:18-18、Web/Controllers/StudentsController.cs:31-38、Infrastructure/SearchContextStore.cs:14-14］

3.9 論理削除結果から検索への復帰

処理概要

Administratorが既存学生を論理削除し、Savedの場合だけ通知と303で検索へ戻る。編集中の業務値は削除フォームから保存しない。［根拠：Views/Students/Edit.cshtml:17-17、Web/Controllers/StudentsController.cs:37-38］

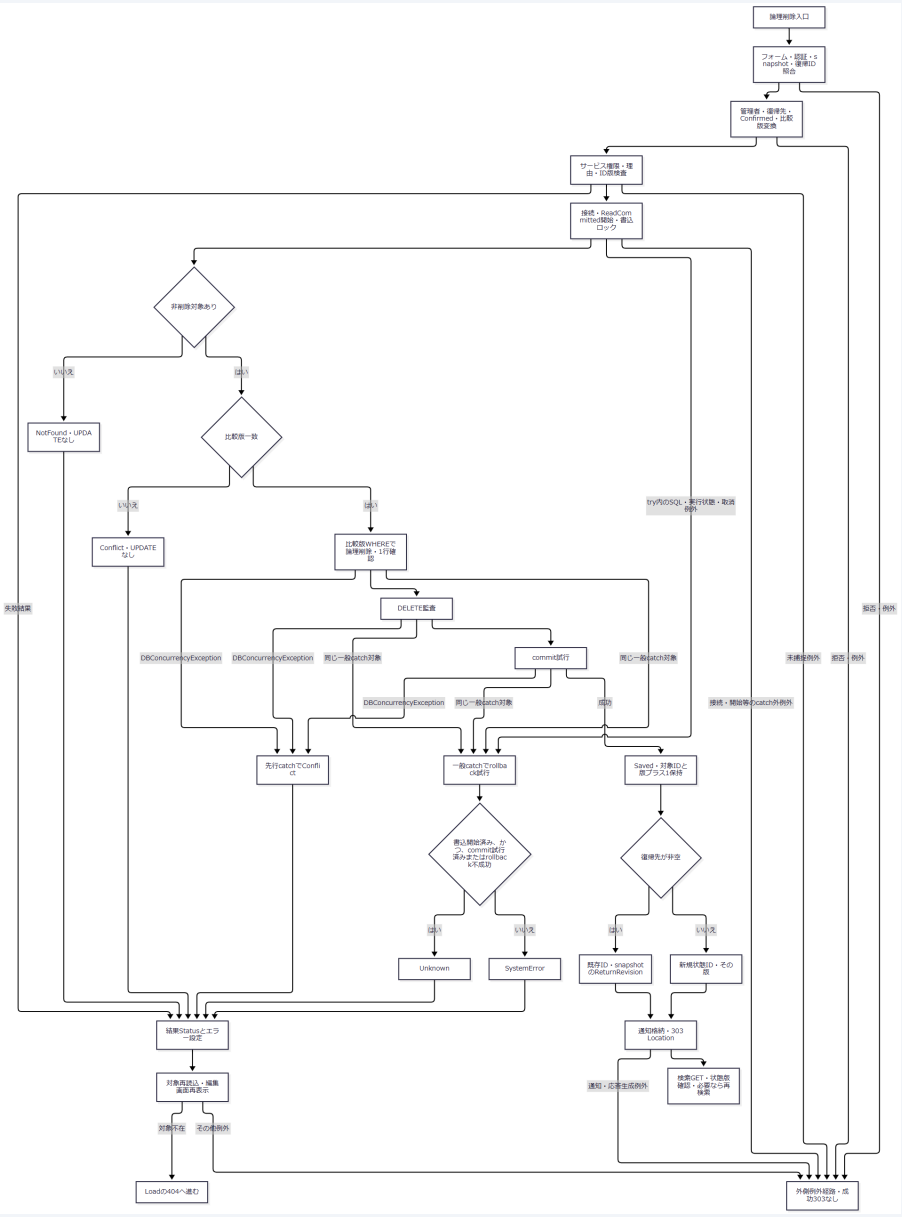
処理手順

1. FormGuard、認証、スナップショットの所有者・セッション・対象、復帰ID一致を確認する。
2. Coordinatorで管理者権限、非空復帰先、Confirmedを確認する。未確認は422。ExpectedVersionを正のlongへ変換する。
3. サービスで削除権限、必須理由の原入力・Trim後各60スカラー、正のID・版、版上限を確認する。
4. DB接続とReadCommitted開始後、学生書込ロックを取得する。
5. 非削除行の版を読み直す。不在はNotFound、版不一致はConflictを直ちに返し、UPDATEへ進まない。
6. 学生ID・比較版・非削除のWHEREで削除フラグ、削除日時・削除者・理由、更新日時・更新者、版を更新する。1行更新でなければ競合例外とする。
7. DELETE監査を同じトランザクションへ書き、commit成功後にSaved・対象ID・版+1を返す。

8. Savedなら既存復帰IDとsnapshot.ReturnRevision、または新規検索状態とその版を用い、「論理削除しました。」を格納して303を返す。
9. 非Savedなら結果のエラーとStatusを設定して対象を再読み込む。後続検索GETでは非削除条件により削除済み学生を一覧から除く。

[根拠：Web/Controllers/StudentsController.cs:37-38、Infrastructure/WebStudentCoordinator.cs:19-19、Core/SqlServerStudentService.cs:392-416]

処理フロー




```
flowchart TD
  A[論理削除入口] --> B[フォーム・認証・snapshot・復帰ID照合]
  B --> C[管理者・復帰先・Confirmed・比較版変換]
```

```
        C --> D[サービス権限・理由・ID版検査]
        D -->|失敗結果| N[結果Statusとエラー設定]
        D --> E[接続・ReadCommitted開始・書込ロック]
        E --> F{非削除対象あり}
        F -->|いいえ| NF[NotFound・UPDATEなし]
        F -->|はい| G{比較版一致}
        G -->|いいえ| CF[Conflict・UPDATEなし]
        G -->|はい| H[比較版WHEREで論理削除・1行確認]
        H --> I[DELETE監査]
        I --> J[commit試行]
        J -->|成功| S[Saved・対象IDと版プラス1保持]
        S --> R{復帰先が非空}
        R -->|はい| R1[既存ID・snapshotのReturnRevision]
        R -->|いいえ| R2[新規状態ID・その版]
        R1 --> T[通知格納・303 Location]
        R2 --> T
        T --> U[検索GET・状態版確認・必要なら再検索]
        NF --> N
        CF --> N
        H -->|DBConcurrencyException| P[先行catchでConflict]
        I -->|DBConcurrencyException| P
        J -->|DBConcurrencyException| P
        P --> N
        E -->|try内のSQL・実行状態・取消例外| Q[一般catchでrollback試行]
        H -->|同じ一般catch対象| Q
        I -->|同じ一般catch対象| Q
        J -->|同じ一般catch対象| Q
        Q --> Q1{書込開始済み、かつ、commit試行済みまたはrollback不成功}
        Q1 -->|はい| UN[Unknown]
        Q1 -->|いいえ| SE[SystemError]
        UN --> N
        SE --> N
        N --> L[対象再読込・編集画面再表示]
        L -->|対象不在| L1[Loadの404へ進む]
        B -->|拒否・例外| O[外側例外経路・成功303なし]
        C -->|拒否・例外| O
        D -->|未捕捉例外| O
        E -->|接続・開始等のcatch外例外| O
        T -->|通知・応答生成例外| O
        L -->|その他例外| O
```

Q1の括りは保存と同じく「書込開始済み、かつ（commit試行済み、またはrollback不成功）」である。DBConcurrencyExceptionを後段の一般catch条件に含めない。［根拠：Core/SqlServerStudentService.cs:415-416］

SQL・テーブル関連（ER図）

削除UPDATEの対象は m_student、監査INSERTの対象は t_operation_log である。3.8の処理関連図の学生から監査への関係を使う。クラス・コードの削除や更新は行わない。

DB操作の詳細

再確認は SELECT row_version FROM dbo.m_student WHERE student_id=@id AND is_deleted=0。UPDATEのWHEREは student_id=@id AND row_version=@version AND is_deleted=0 とする。設定列はis_deleted=1、deleted_at、deleted_by、delete_reason、updated_at、updated_by、row_version+1である。ID・版はBigInt、時刻はDateTimeOffset、実施者はNVarChar(128)、理由はNVarChar(120)。1行更新と監査後のcommit成功がSavedの条件である。［根拠：Core/SqlServerStudentService.cs:403-414］

監査にはDELETEと対象キー、前版・後版、削除・更新の変更列、実施者・時刻、理由、受付識別情報を設定する。理由の業務上限60スカラーと保存パラメータ・列容量120は別の制約である。［根拠：Core/SqlServerStudentService.cs:385-390・396-413、database/001_schema.sql:35-37・54-54］

結果・例外時の扱い

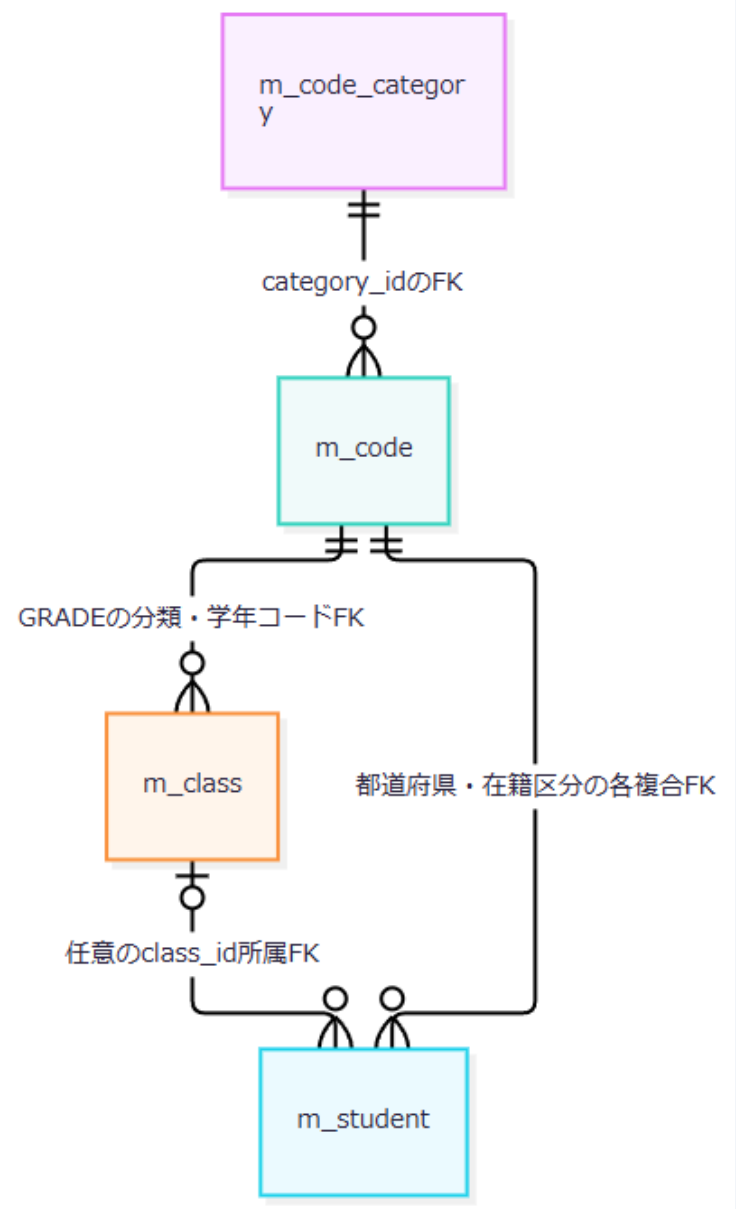
DBConcurrencyExceptionを先に捕捉してConflictを返す。後続のSqlException・InvalidOperationException・OperationCanceledExceptionだけが、rollback結果と書込・commitフラグによるUnknown／SystemError判定へ進む。接続・開始の例外はそのcatch外である。理由検査はInputValidationExceptionをValidationFailedへ変換するが、不正Unicode等のArgumentExceptionを同じ結果へ変換するcatchはない。［根拠：Core/SqlServerStudentService.cs:396-402・415-416、Core/StudentNormalizer.cs:37-57］

削除コントローラーに保存と同じWebFaultの個別catchはない。非Savedでは成功303を返さず、結果Statusを設定してLoadする。その読込みで対象が不在なら404へ進むため、最終応答が最初の結果Statusを必ず保つわけではない。復帰版・期限の扱いと通知の一回取得は3.8・3.6と同じである。［根拠：Web/Controllers/StudentsController.cs:25-25・37-38］

補足説明

4.1 学生・所属・コードの関係

次図はDDLに宣言された検索関連FKを示す。学生のclass_idはNULL可能で、所属クラスは任意である。学生側class_idに一意制約はないため、同じクラスを複数学生が参照できる。学生の都道府県・在籍区分とクラスの学年は、分類・コードの組でコード表を参照する。検索のLEFT JOINとは別の宣言である。［根拠：database/001_schema.sql:28-43・62-80・187-197］




```
erDiagram
  m_code_category ||--o{ m_code : "category_idのFK"
  m_code ||--o{ m_class : "GRADEの分類・学年コードFK"
  m_class o|--o{ m_student : "任意のclass_id所属FK"
  m_code ||--o{ m_student : "都道府県・在籍区分の各複合FK"
```

対象	この画面と関連処理での意味・属性
m_student	student_idはbigint IDENTITYのPK。学籍番号は削除行を含むUNIQUE。表示氏名・カナと照合キーを別列で保持する。生年月日は和暦表示、都道府県・住所1は住所表示、本人・緊急電話は検索、入学日は期間・順序、所属・在籍区分は抽出・名称参照に使う。row_versionは正のbigintで学生更新版、is_deletedはbit NOT NULL DEFAULT 0。作成・更新情報は保存処理、削除情報は論理削除で設定する。
m_class	class_idはbigint IDENTITYのPK。school_yearはsmallint、学年分類・コードはvarchar(24)・varchar(2)、クラスコード・名称はvarchar(9)・nvarchar(36)で、いずれもNOT NULL。年度・学年は検索と候補連動、コード・名称は候補と一覧表示に使う。valid_from・valid_toはdate NOT NULL、is_activeはbit NOT NULL DEFAULT 1、display_orderはint NOT NULL DEFAULT 0。capacityはint NULLで保存時定員確認に使い、検索画面に表示しない。
m_code	category_id・code_valueはvarchar(24)・varchar(10) NOT NULLの複合PK。code_nameはnvarchar(24) NOT NULLで候補・結合名称に使う。display_orderはint NOT NULL DEFAULT 0。is_activeはbit NOT NULL DEFAULT 1、valid_fromはdate NOT NULL、valid_toはdate NULL、is_protectedはbit NOT NULL。停止・期間は検索候補から除外する条件ではなく、保守・新規変更値の保存判定に使う。
m_code_category	category_idはvarchar(24) NOT NULLのPK。category_nameはnvarchar(24) NOT NULL、code_lengthはsmallint NOT NULL、allow_addはbit NOT NULL、is_activeはbit NOT NULL DEFAULT 1。候補取得時の結合と有効状態、保守時のコード幅・追加許可に使用する。分類名は検索画面に表示しない。分類名称の入力側業務上限は保守処理から確定せず、DDLの12スカラーCHECKと区別する。

[根拠：database/001_schema.sql:7-134、Core/SqlServerStudentService.cs:77-90・137-153・197-214、Core/MasterMaintenanceService.cs:91-109]

クラスには年度・クラスコードの複合UNIQUE、年度1900～9998、年度内の利用期間、定員NULLまたは1～9999、非負表示順を宣言する。クラス学年分類はGRADE、学生都道府県分類はPREFECTURE、在籍分類はENROLLMENT_STATUSに限定する。学生には生年月日・入学日・終了日・所属と在籍区分の整合、連絡先・緊急連絡先の組、削除情報の組を確認するCHECKもある。これらはDDLの制約であり、画面の入力検査や検索述語とは区別する。 [根拠：database/001_schema.sql:188-203]

非削除学生向けの索引は、生年月日、所属・在籍区分、入学日、更新日時、本人電話、緊急電話に宣言される。照合キー等の文字列列はBIN2、表示文字列はSC対応の照合順序を指定する。氏名検索・順序が表示氏名の見た目ではなくname_keyを使うことは、この列区分にも対応する。 [根拠：database/001_schema.sql:9-13・206-211、Core/SqlServerStudentService.cs:108-108・141-141]

4.2 候補取得と表示名称の保存側制約

検索は LookupAsync(false) で全候補を取得する。第1結果はコードと分類の内部結合、第2結果はクラスと学年コードのLEFT JOINである。停止・期間外を除外するWHEREはない。コードIsActiveはコード・分類の双方が有効な場合だけtrueになるが、検索ビューはその値で候補を除外しない。 [根拠：Core/SqlServerStudentService.cs:197-216、Views/Students/Search.cshtml:5-5]

候補	キー・表示	取得順序・初期選択・連動
年度	全クラスのschool_year	重複除去・降順。初期は「すべて」で、現在年度を自動選択しない。
学年	GRADEのcode_value・code_name	コード全体の順序はcategory_id、display_order、code_value。初期は「すべて」。
在籍区分	ENROLLMENT_STATUSのcode_value・code_name	初期01・02。seedは01在学、02休学、03卒業、04退学。名称はDB候補に従う。全解除は条件なし。
都道府県	PREFECTUREのcode_value・code_name	コード取得順を保持し、初期は「すべて」。
クラス	class_id・年度／クラスコード／名称	年度降順、学年コードdisplay_order、クラスdisplay_order、class_code、class_id。画面で指定年度・学年に一致する候補を抽出。候補更新時に不一致選択を解除。
停止・期間外	コード・クラスの有効状態と期間	検索では除外しない。コードには注記なし。クラスは停止だけ注記し、期間外は注記しない。
所属人数	SchoolClass.Occupied	検索用falseは集計せず0を設定する。実人数の確認結果ではなく、画面にも表示しない。編集用trueは非削除かつ01・02の人数を集計する。

[根拠：Views/Students/Search.cshtml:5-5、Core/SqlServerStudentService.cs:201-214、database/002_seed.sql:19-22]

コード結果は分類、値、名称、順序、有効状態、期間、保護値を返し、クラス結果はID、年度、学年、コード・名称、期間、定員、人数、有効状態を返す。検索用候補パラメータはなく、正常読取後にcommitする。 [根拠：Core/Contracts.cs:61-65、Core/SqlServerStudentService.cs:202-216]

表示名称の保存側には、次の独立した制約がある。結合結果の名称NULL可能性から保存列のNULL可否を決めない。

値	保守実装の検査	保存パラメータ・DDL
コード名称	原入力・Trim後各12 Unicodeスカラー以内の必須値	@name:NVarChar(24)、m_code.code_name:nvarchar(24) NOT NULL、12スカラーCHECK
クラス名称	原入力・Trim後各18スカラー以内の必須値	保存 NVarChar(36)、m_class.class_name:nvarchar(36) NOT NULL、18スカラーCHECK
クラスコード	Identifierで原入力・Trim後・正規化後9スカラー以内、英数字・ハイフン形式	保存 VarChar(9)、m_class.class_code:varchar(9) NOT NULL

[根拠：Core/MasterMaintenanceService.cs:30-37・73-76・79-85・113-120、Core/StudentNormalizer.cs:47-64、database/001_schema.sql:67-81・112-128]

これはMasterMaintenanceService内に存在する保存制約である。検索WebのDI・ルートから保守入口へ接続されたことは示さない。停止候補を検索できることと、その値を新しく学生へ保存できることも別であり、保存時は3.8のDB再確認を行う。［根拠：Infrastructure/WebHostSetup.cs:13-19、Core/SqlServerStudentService.cs:344-383]

4.3 フォーム・認証・識別値の共通境界

MVCには認証、CSRF検査、モデルバインド・検証の設定があり、非nullable参照型による暗黙Requiredは抑止されている。HTTP文字列、MVC変換後の値、サービス引数を区別する。検索だけはModelState有効性を実行条件とし、page・options・clearには同じ条件がない。［根拠：Infrastructure/WebHostSetup.cs:14-18、Web/Controllers/StudentsController.cs:13-22]

各POSTアクション内のFormGuardは、入力型の公開プロパティ、明示制御項目、検証トークン、書式メタ情報を許可する。許可外キー、不正な__Invariant内容、例外項目以外の同名複数値は400で拒否する。StatusCodes・UnassignedOnlyは複数値を許可する。許可項目の存在をすべて必須化する検査ではない。検索・候補更新は在籍キー欠落を空配列へ補正するが、NULL配列がサービスへ届けばLength参照の例外となり、全解除と同じ契約ではない。［根拠：Web/Controllers/StudentsController.cs:14-14・22-22・41-41、Core/SqlServerStudentService.cs:111-112]

RequestIdentityは認証済み、非空白の主体64Unicodeスカラー以内、非空白の認証セッション64コード単位以内、Role変換成功を要求し、不備を401にする。Role変換後のEnum.IsDefined検査はなく、操作条件はViewer等との一致・不一致である。検索POSTは受信TabIdを渡し、RequestIdentityのヘッダー・query用TabIdプロパティを自動使用する構造ではない。［根拠：Infrastructure/RequestIdentity.cs:7-16、Web/Controllers/StudentsController.cs:13-22]

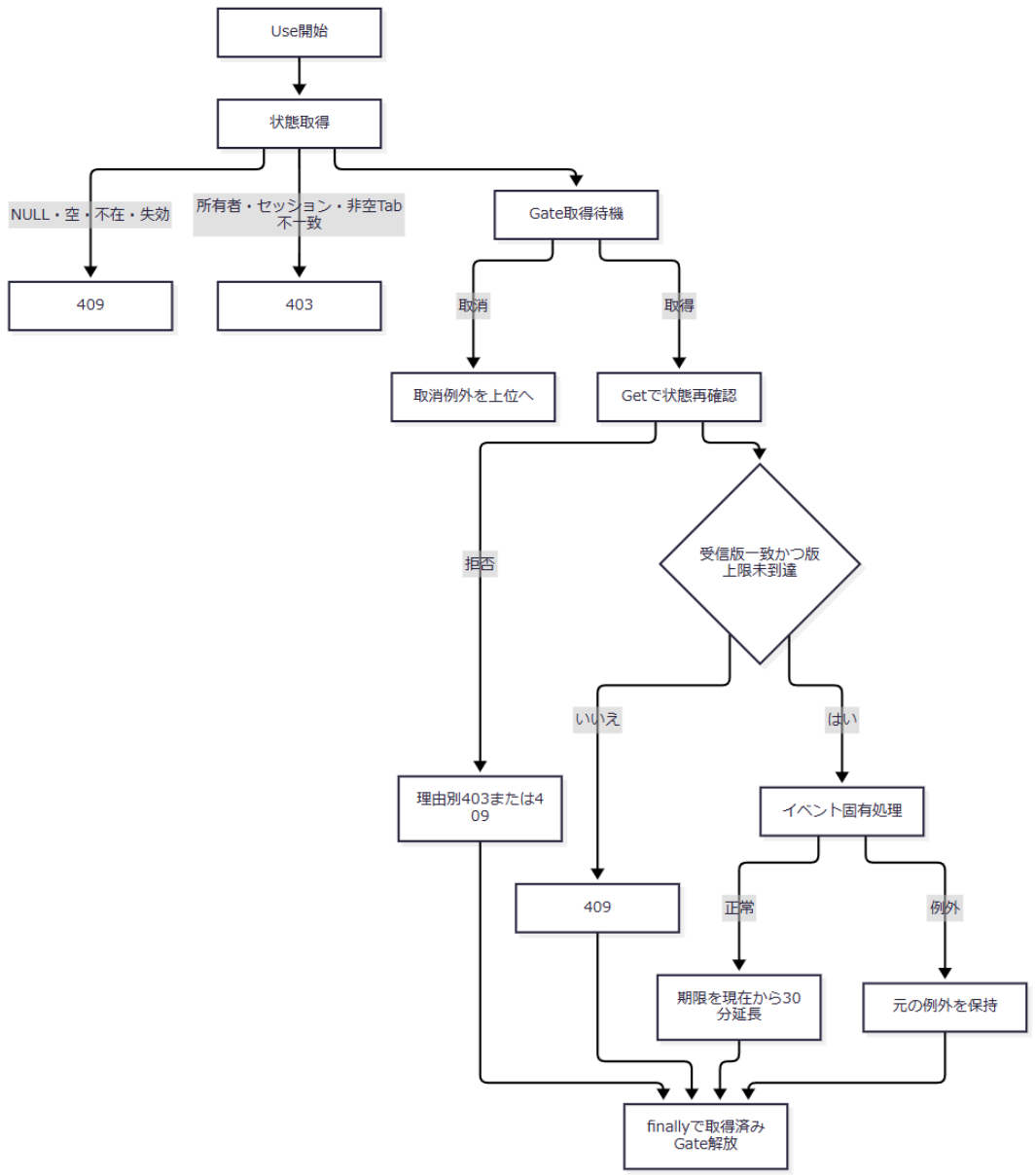
文字数はScalarCountによるUnicodeスカラー数で、サロゲートペアを1と数え、不正サロゲートはArgumentExceptionとなる。TextはNULLを空文字に置き、原入力上限、制御文字禁止、Trim、必須・空値、加工後上限の順で確認する。ブラウザーmaxlength、サービス原入力上限、正規化後上限、照合キー上限、nvarcharのUTF-16容量、DDLのスカラーCHECKはそれぞれ別である。たとえば氏名maxlength=16はサービスの8スカラー上限を拡張しない。［根拠：Core/StudentNormalizer.cs:37-57、Views/Students/Search.cshtml:5-5、database/001_schema.sql:10-13・44-47]

4.4 メモリ保持・排他・期限・一時情報

保持領域	内容と用途	利用条件・更新
SearchContextStore	SingletonのConcurrentDictionary。ID、Owner、Session、Tab、Expires、Revision、Applied、LastResult、SemaphoreSlimを保持。	初期版1・30分。正常Useで30分延長。検索・ページ・Appliedあり復帰・クリアで版加算。候補更新は版比較・期限延長なし。
EditorSnapshotStore	所有者・セッション・対象ID・読込明細・入力コピー・ReturnRevisionを保持する辞書。	作成から30分。空・不在・失効は独自409、所有者・セッション・対象相違は403。NULLキーの明示検査はなく、辞書呼出しのライブラリ例外が独自検査より前に生じ得る。
FlashMessageStore	主体・認証セッション・ContextIdの組をキーに通知を保持する辞書。	10分。Takeで削除し、期限内なら一回返す。取得後の表示失敗で再格納しない。
t_runtime_state	DDLはstate_id、owner_id、login_session_id、state_kind、encrypted_payload、expires_at、revision、consumedをNOT NULLで宣言。	上記保持実装からこの表への読書きは接続されていない。列宣言から暗号化保存・永続化・消費更新を追加しない。

[根拠：Infrastructure/WebHostSetup.cs:13-13、Infrastructure/SearchContextStore.cs:9-15、Infrastructure/EditorSnapshotStore.cs:7-11、Infrastructure/FlashMessageStore.cs:6-9、database/001_schema.sql:171-184]

検索状態Getは、NULLを明示的に拒否し、空・不在・期限が現在より過去の場合もW-SRC-002の409とする。所有者・セッションが異なる場合は403、非空Tabだけを比較し相違も403とする。NULL・空Tabは照合を省略する。Useは最初のGet、排他待機、待機後Get、版文字列の直接比較、long.MaxValue確認、個別処理の順に進む。正常に戻った場合だけ期限を延長する。[根拠：Infrastructure/SearchContextStore.cs:13-14]




```
flowchart TD
  A[Use開始] --> B[状態取得]
  B -->|NULL・空・不在・失効| X[409]
  B -->|所有者・セッション・非空Tab不一致| Y[403]
  B --> C[Gate取得待機]
```

```
        C -->|取消| Z[取消例外を上位へ]
        C -->|取得| D[Getで状態再確認]
        D -->|拒否| E[理由別403または409]
        D --> F{受信版一致かつ版上限未到達}
        F -->|いいえ| G[409]
        F -->|はい| H[イベント固有処理]
        H -->|正常| I[期限を現在から30分延長]
        H -->|例外| J[元の例外を保持]
        E --> R[finallyで取得済みGate解放]
        G --> R
        I --> R
        J --> R
```

待機取消はtryへ入る前に発生する。取得後の解放は結果・例外を引き継ぐものであり、固定の応答へ変換しない。DB await後にはGetを再実行しないため、待機後の確認を処理完了時の期限確認へ拡張しない。［根拠：Infrastructure/SearchContextStore.cs:14-14］

Appliedは入力由来SearchCriteriaのコピーであり、正規化済みSQLキーそのものではない。コメントは単一ノードで条件をサーバー保持する意図を述べ、実装はメモリ辞書である。検索状態版、学生行版、スナップショット、通知を別の値として扱う。［根拠：Infrastructure/SearchContextStore.cs:6-15、Infrastructure/WebStudentCoordinator.cs:8-9、Core/SqlServerStudentService.cs:113-153］

4.5 接続・取消・最終応答と未接続処理

サービスは接続文字列にEncrypt=Mandatory、TrustServerCertificate=falseを設定する。接続後、対象DBのsnapshot_isolation_state=1を確認し、不成立ならInvalidOperationExceptionを投げる。Command helperによる検索・候補コマンドのタイムアウトは30秒で、直接生成する診断コマンドは同helperの明示設定対象ではない。取消トークンを接続、トランザクション開始、SQL実行・読取、commitへ渡す。検索に明示rollbackとDB例外の一般catchはなく、未commit時はawait usingによる資源破棄へ進む。［根拠：Core/SqlServerStudentService.cs:39-73・123-170・199-216］

検索・ページの個別catchはWebFaultを扱い、そのFieldまたは空キーへModelStateエラーを追加してStatusを保持する。FieldへCriteria接頭辞を付加しない。候補更新・クリア・GET復帰には同じ個別catchがない。外側middlewareは応答未開始の場合だけ例外を捕捉し、WebFaultならそのStatus・Message、それ以外なら503と「処理結果を確認できません。検索で確認してください。」のHTMLを返す。応答開始後の例外をそのHTMLへ置換する契約ではない。［根拠：Web/Controllers/StudentsController.cs:15-22、Infrastructure/WebHostSetup.cs:23-24］

認証の401・403、CSRFのフレームワーク拒否、画面内の検証再表示、外側例外HTMLは別経路である。AntiforgeryResultFilterのJSON変換は /api に限られ、学生フォームへ適用しない。［根拠：Infrastructure/WebHostSetup.cs:14-16、Infrastructure/AntiforgeryResultFilter.cs:7-8］

StrictJson.ReadとWebStudentCoordinator.Optionsには宣言・実装があるが、学生フォームの接続入口はこれらを呼ばない。POST /students/options はSearchModelを直接作る。JSON本文検査、未接続の候補helperにあるselectable・occupancy等を公開JSON APIや実際の候補制御として扱わない。［根拠：Infrastructure/StrictJson.cs:6-14、Infrastructure/WebStudentCoordinator.cs:16-16、Infrastructure/WebPresentation.cs:8-18、Web/Controllers/StudentsController.cs:13-22］

全フォーム共通JavaScriptは、確認取消なら送信を止め、送信時は押したボタン以外の同じフォーム内ボタンを無効化し、aria-busy=trueを設定する。別フォーム間の排他ではない。検索状態の直列化はUseが担い、候補更新はその排他を通らない。［根拠：wwwroot/js/site.js:4-4、Views/Students/Search.cshtml:4-8、Infrastructure/SearchContextStore.cs:14-14、Web/Controllers/StudentsController.cs:22-22］

diagram-002 1/10

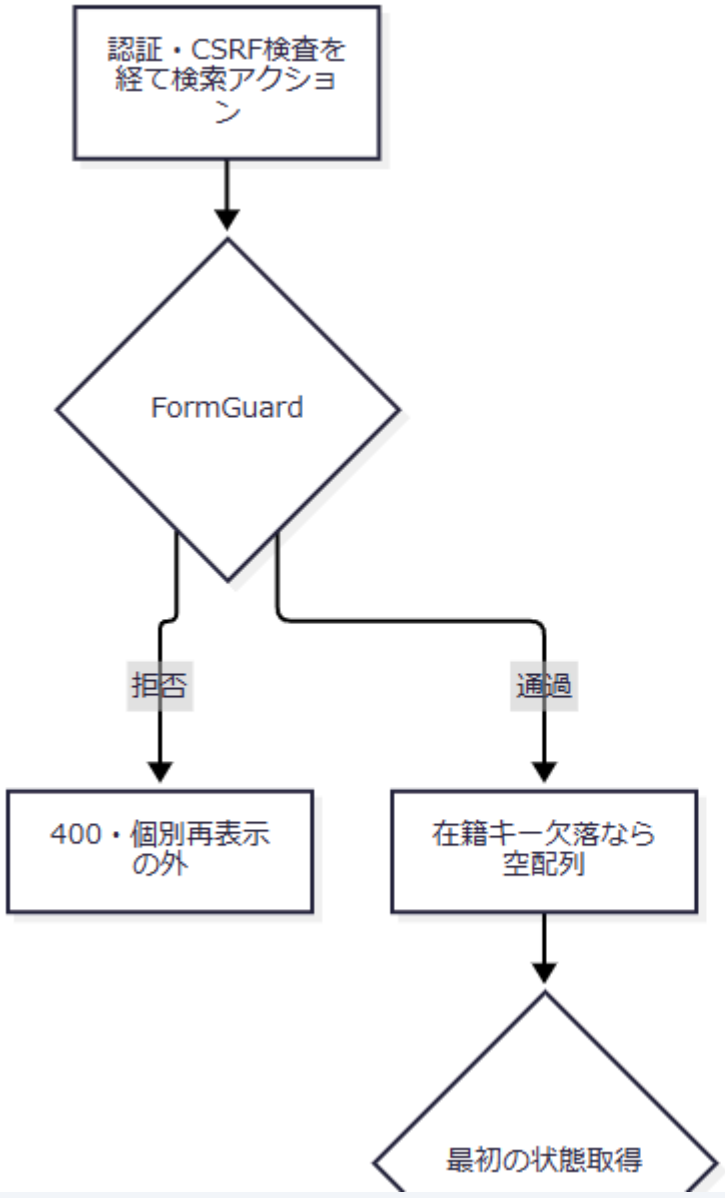


diagram-002 2/10

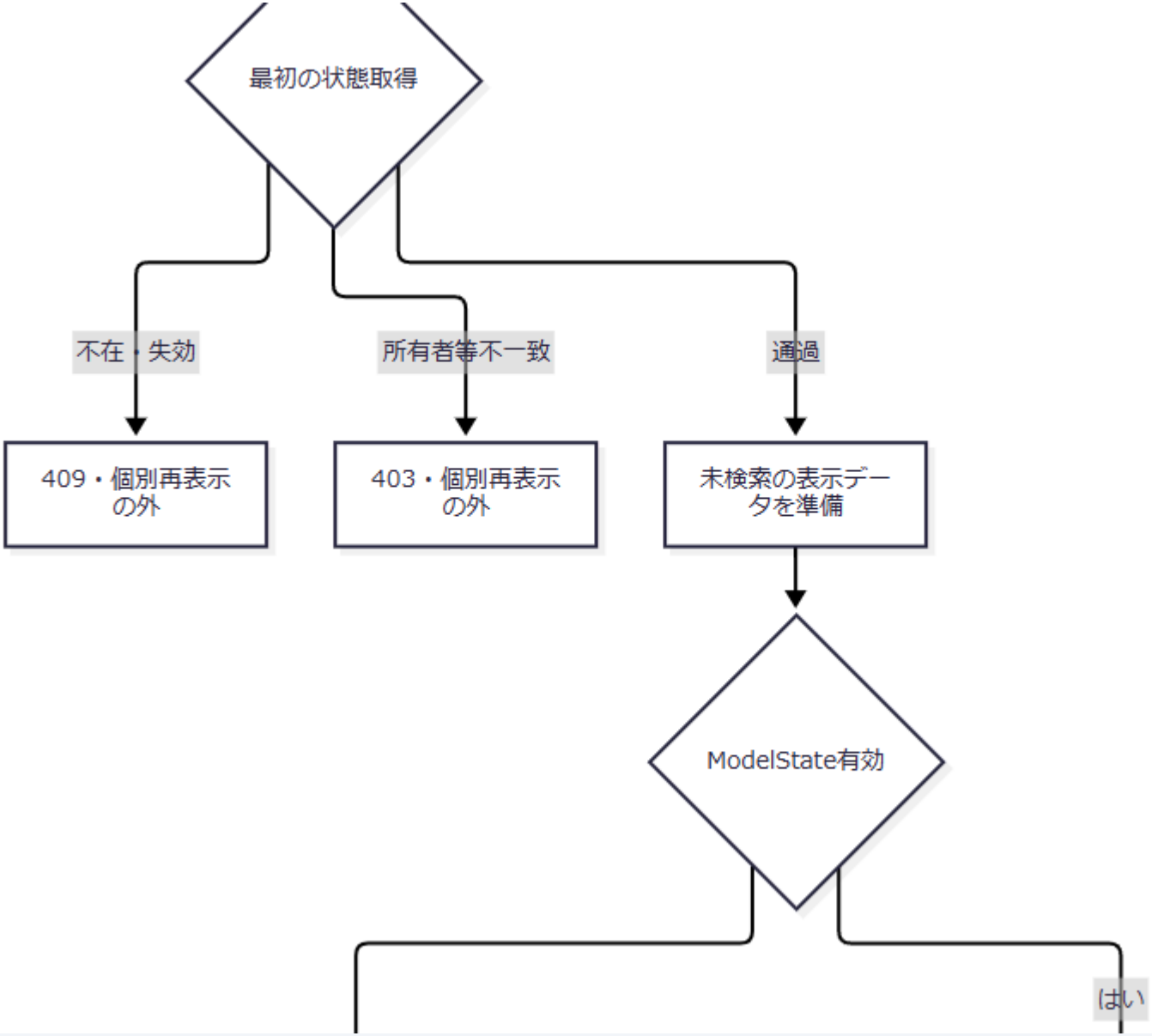
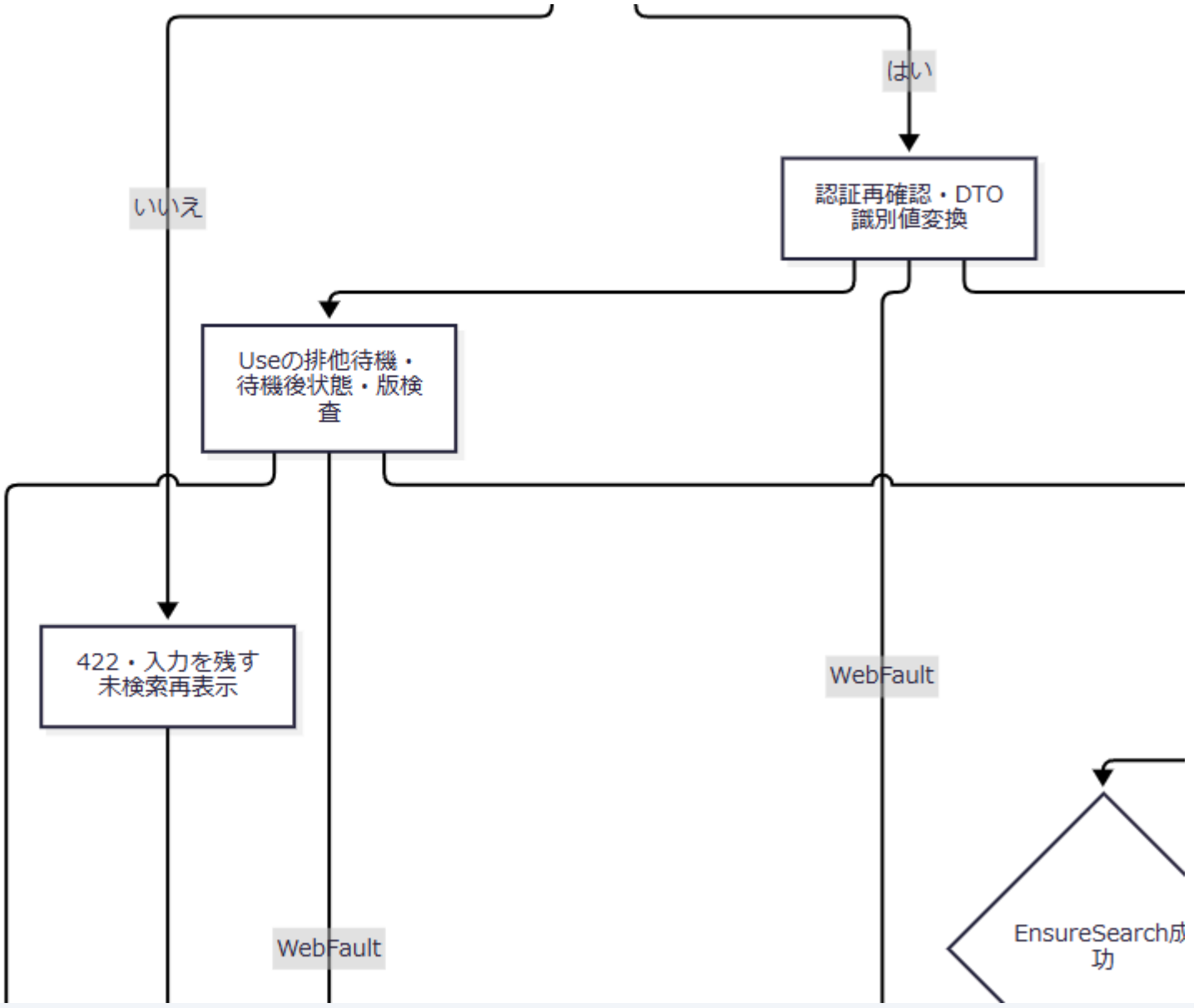
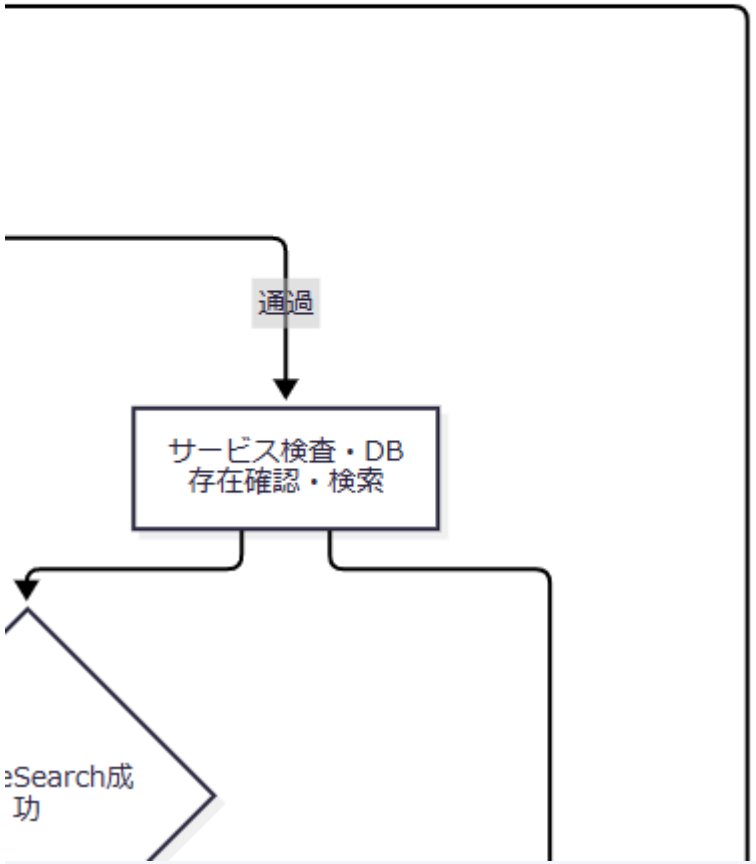
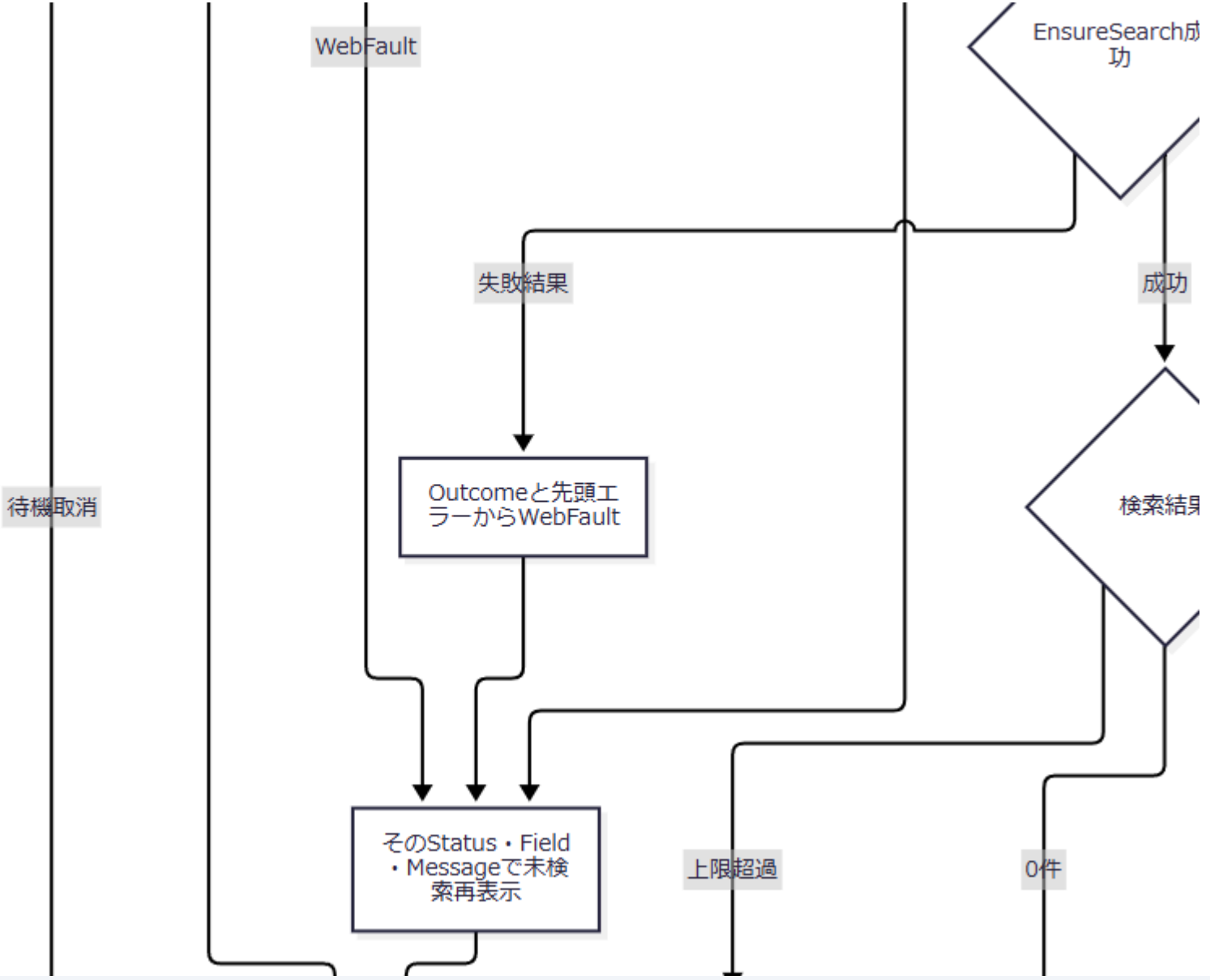
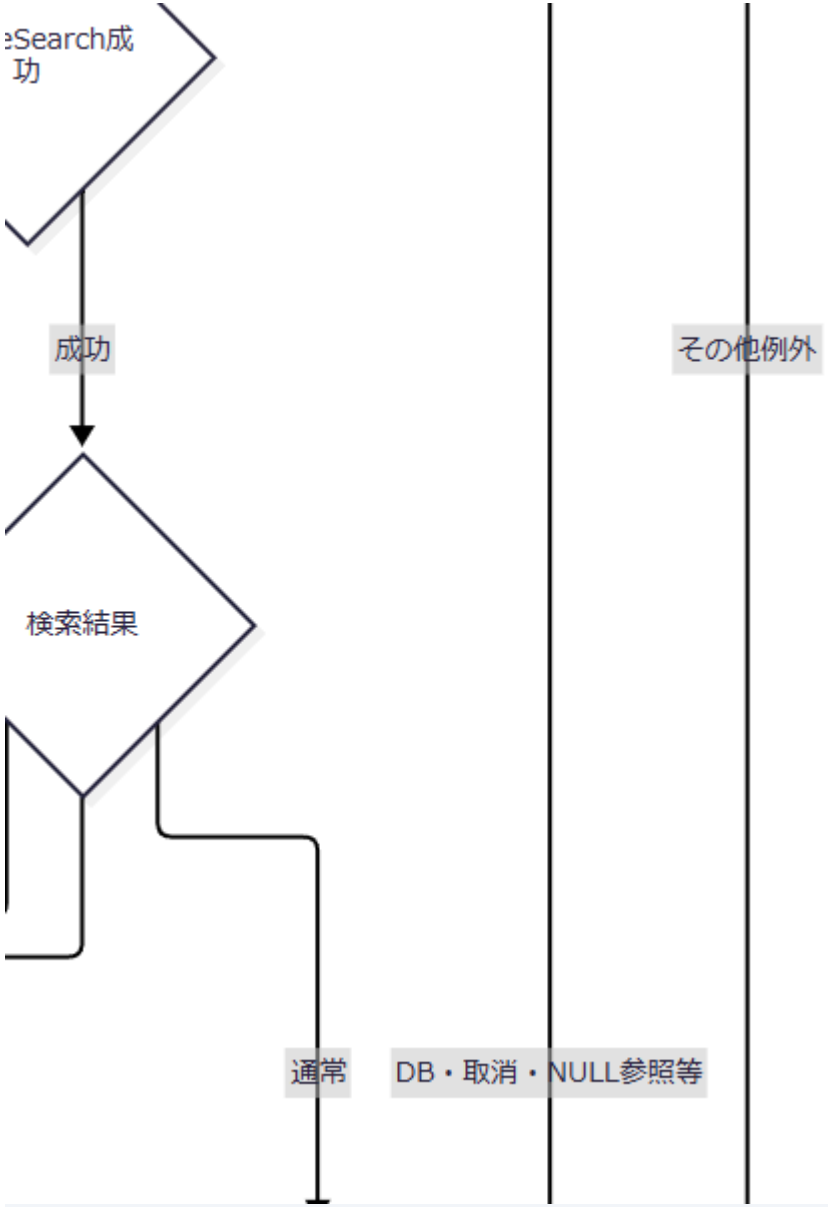

最初の状態取得
不在・失効
所有者等不一致
通過
409・個別再表示の外
403・個別再表示の外
未検索の表示データを準備
ModelState有効
はい

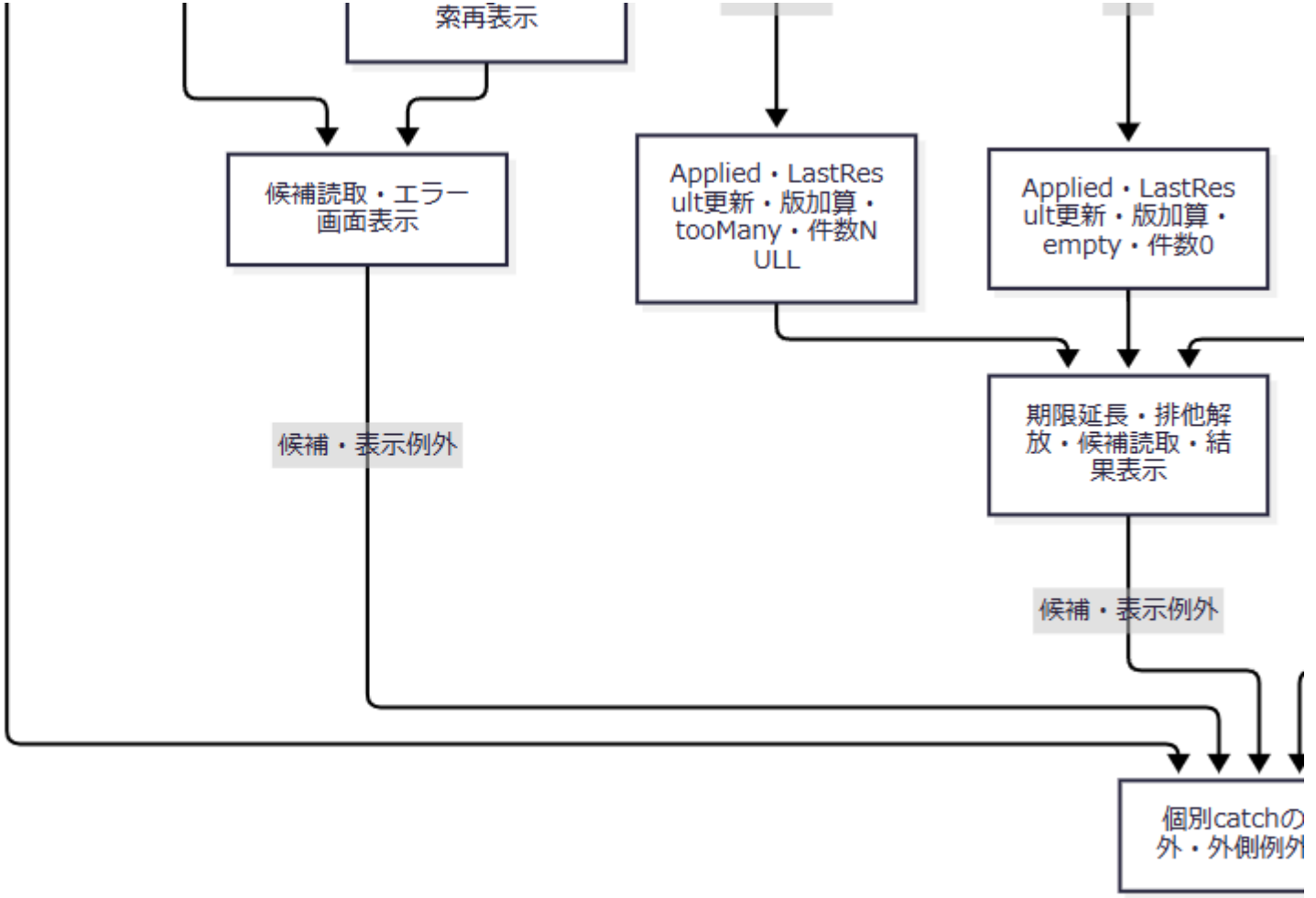
diagram-002 4/10

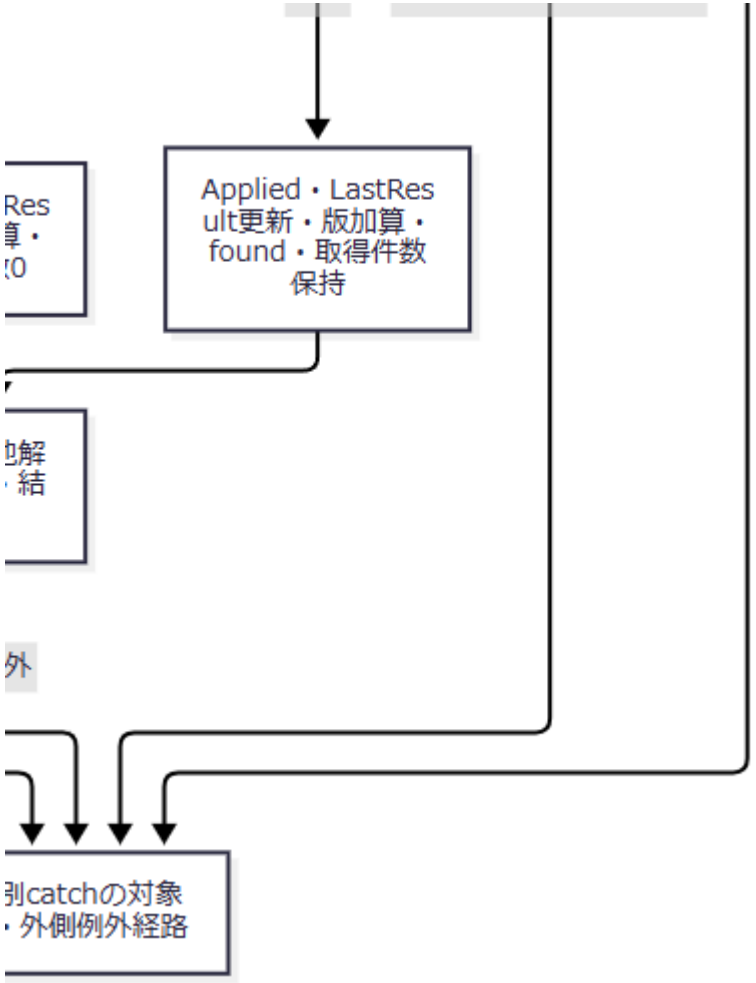

はい
いいえ
認証再確認・DTO
識別値変換
Useの排他待機・
待機後状態・版検
査
422・入力を残す
未検索再表示
WebFault
WebFault
EnsureSearch成
功

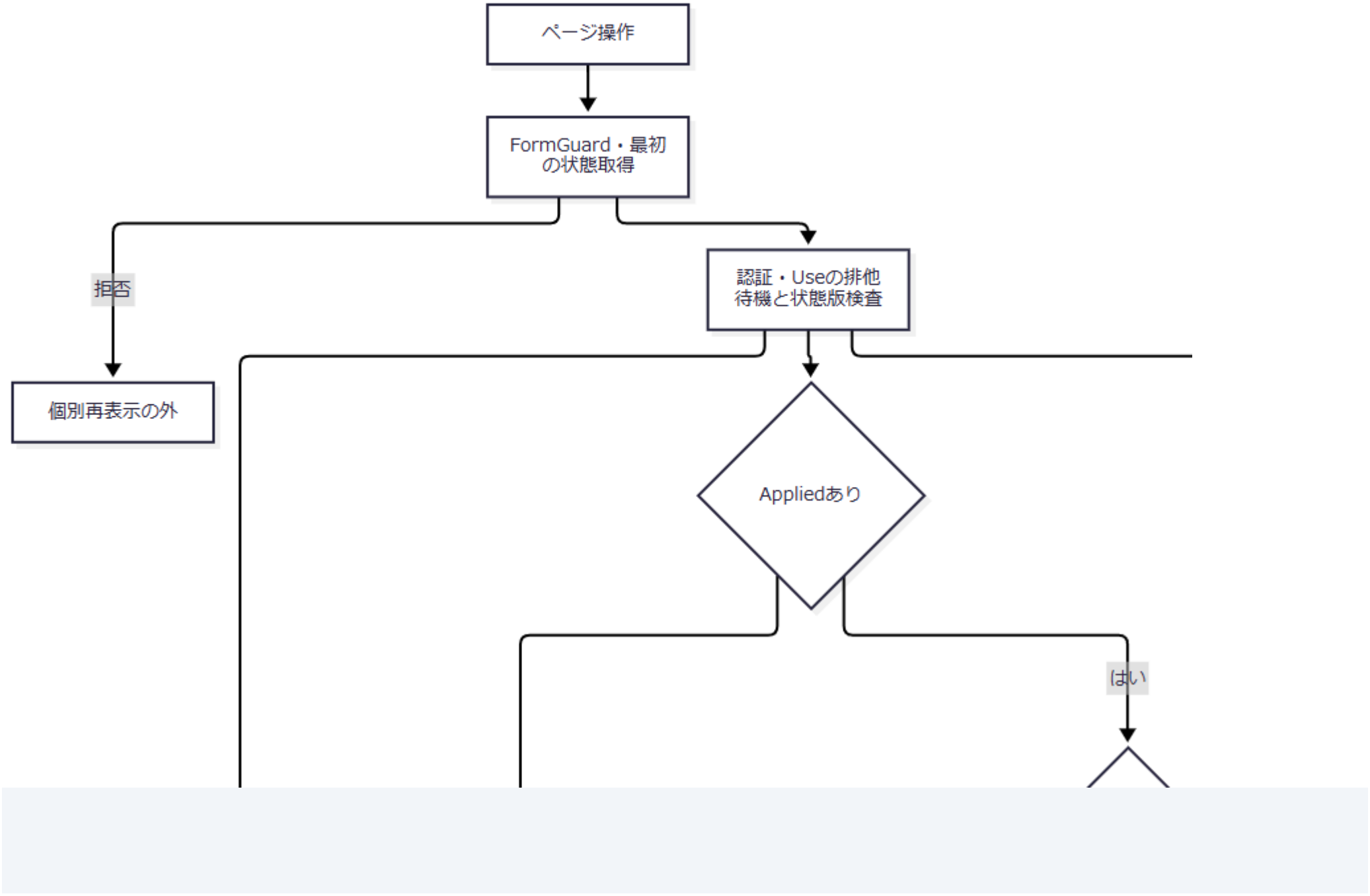

通過
サービス検査・DB
存在確認・検索
eSearch成
功


WebFault
EnsureSearch成功
失敗結果
成功
待機取消
Outcomeと先頭エラーからWebFault
検索結果
そのStatus・Field・Messageで未検索再表示
上限超過
0件


eSearch成
功
成功
その他例外
検索結果
通常
DB・取消・NULL参照等


索再表示
候補読取・エラー
画面表示
Applied・LastRes
ult更新・版加算・
tooMany・件数N
ULL
Applied・LastRes
ult更新・版加算・
empty・件数0
候補・表示例外
期限延長・排他解
放・候補読取・結
果表示
候補・表示例外
個別catchの
外・外側例外


Applied・LastRes
ult更新・版加算・
found・取得件数
保持
外
catchの対象
外側例外経路


ページ操作
FormGuard・最初の状態取得
拒否
認証・Useの排他待機と状態版検査
個別再表示の外
Appliedあり
はい


おい

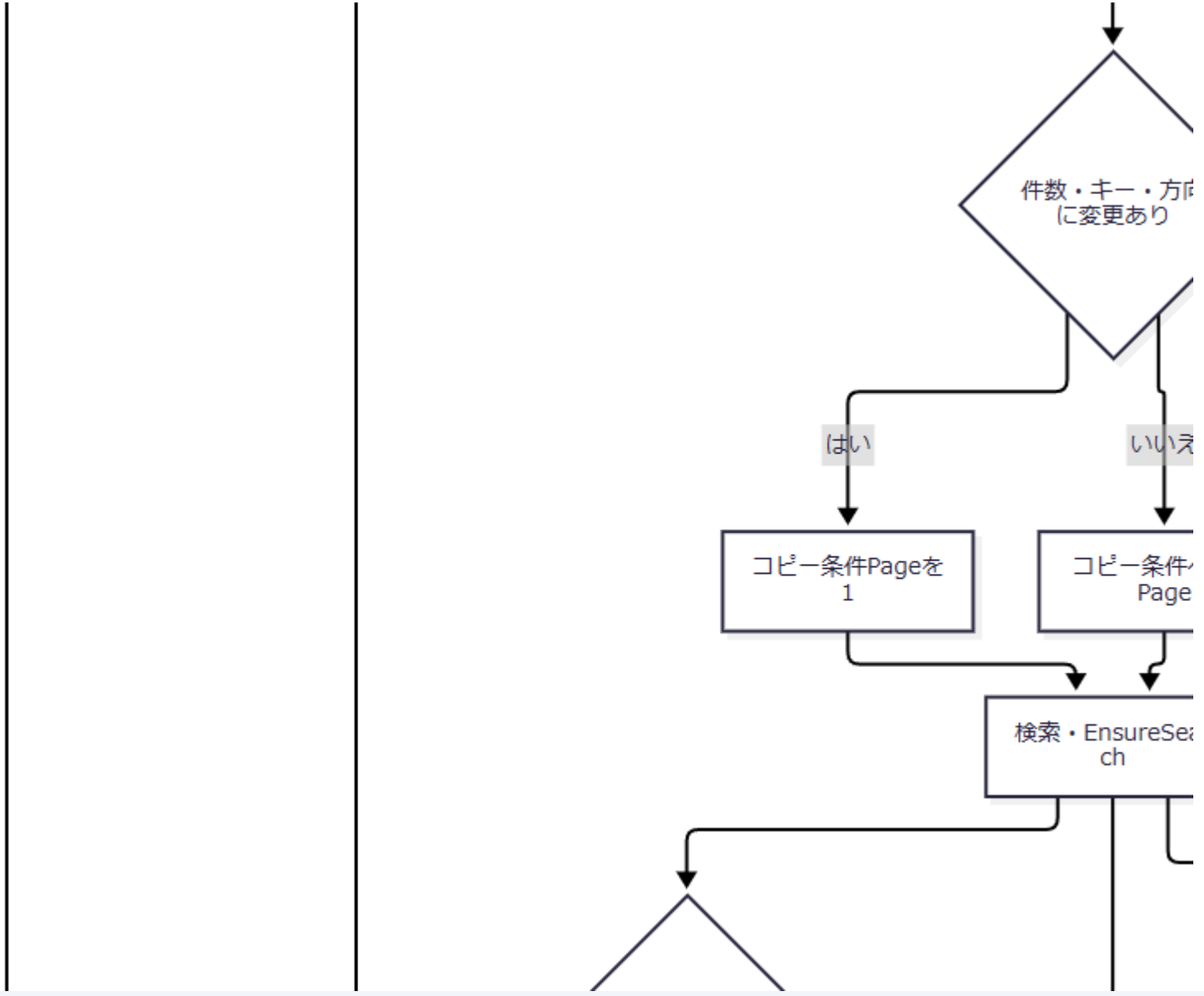
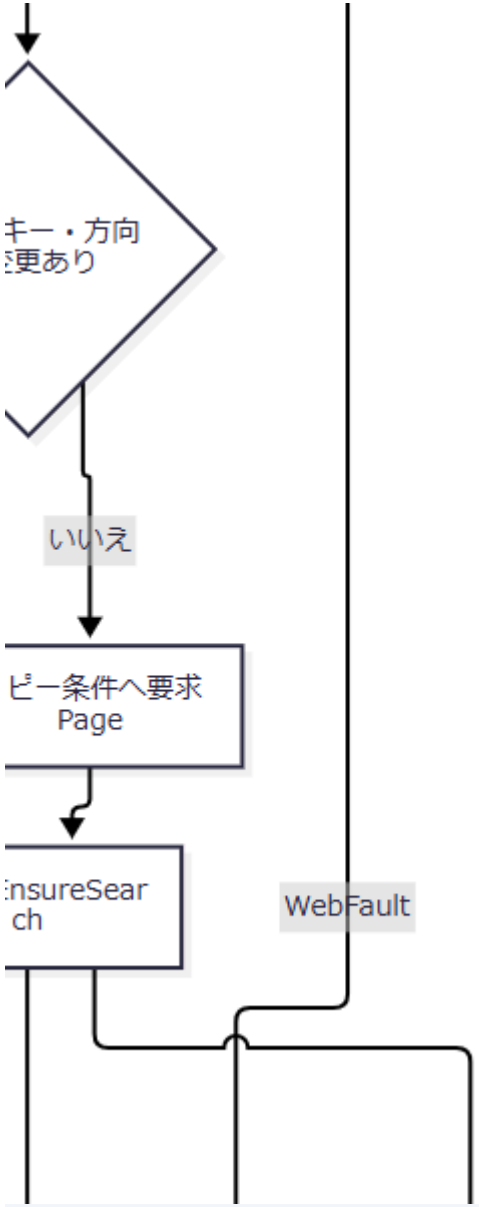

件数・キー・方
に変更あり
はい
いいえ
コピー条件Pageを
1
コピー条件
Page
検索・EnsureSea
ch

diagram-004 4/8



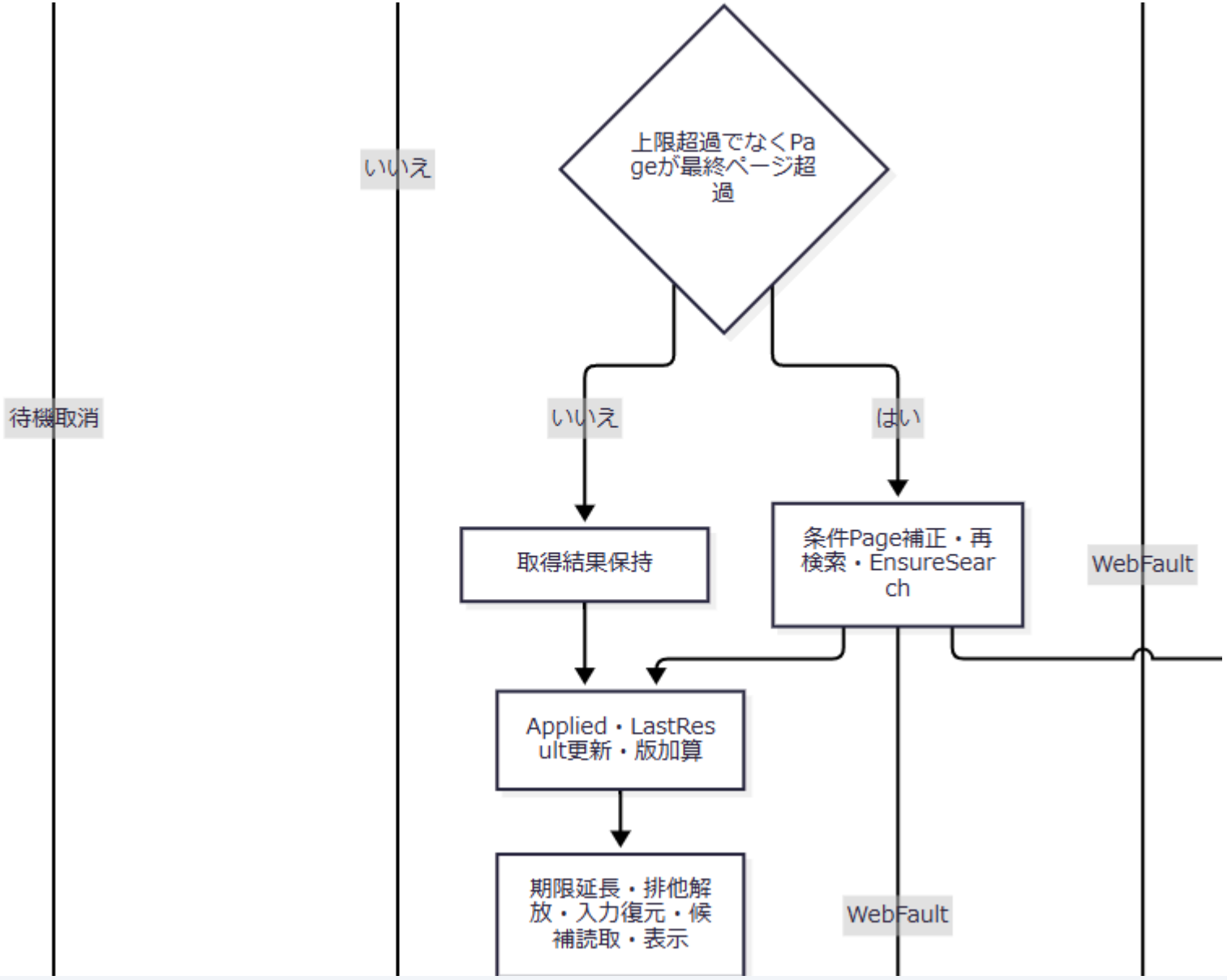
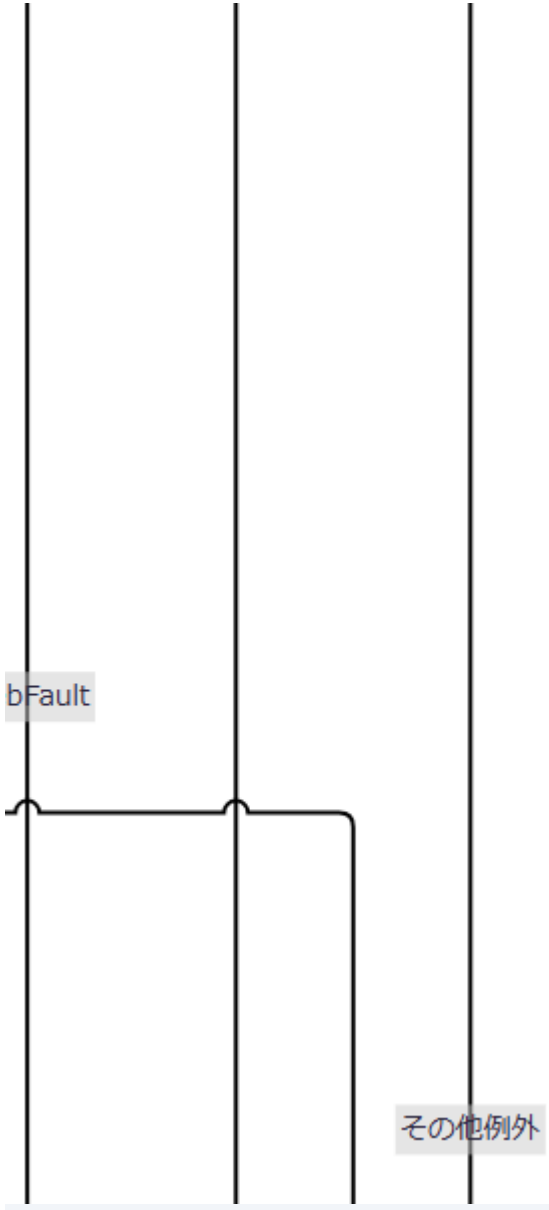
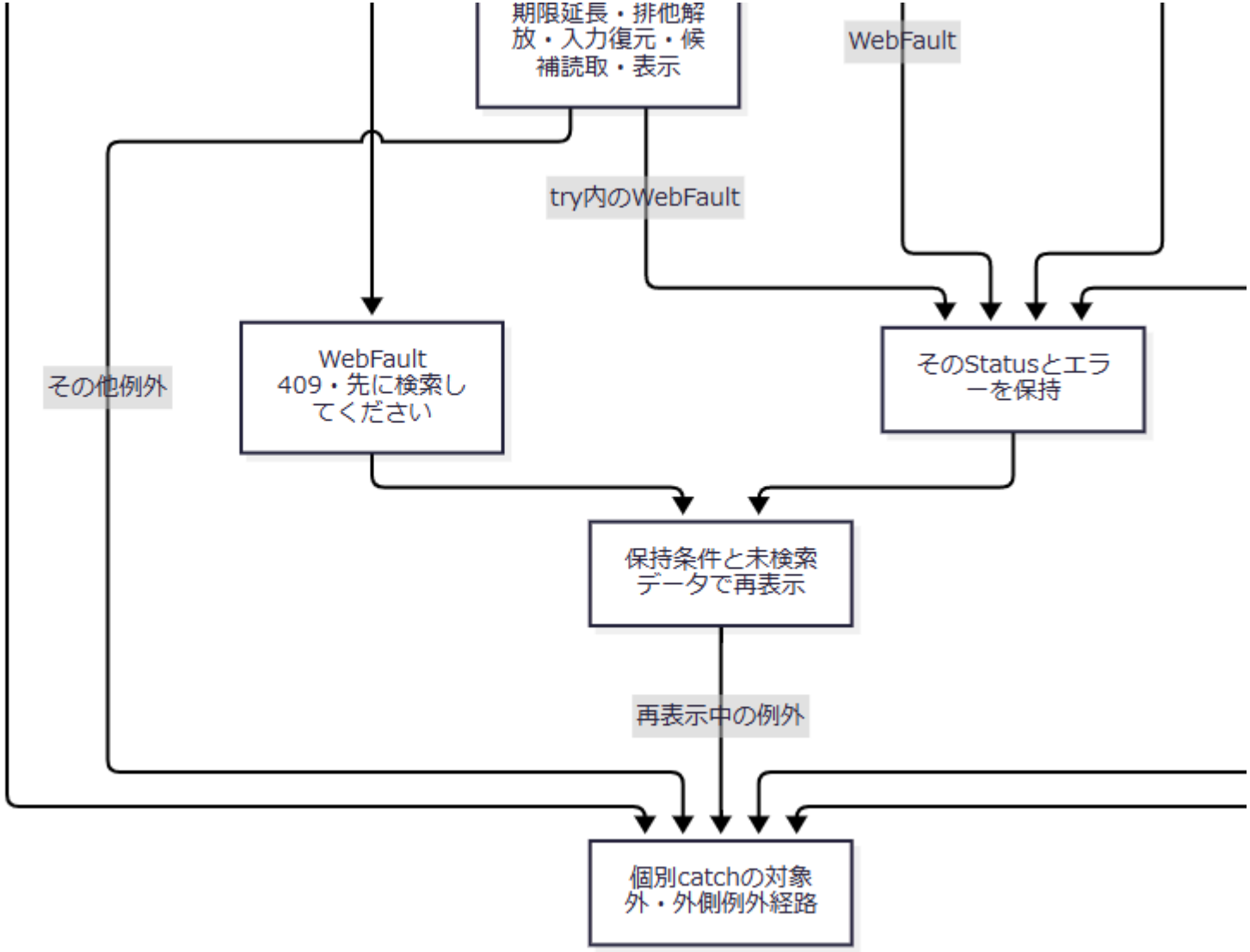
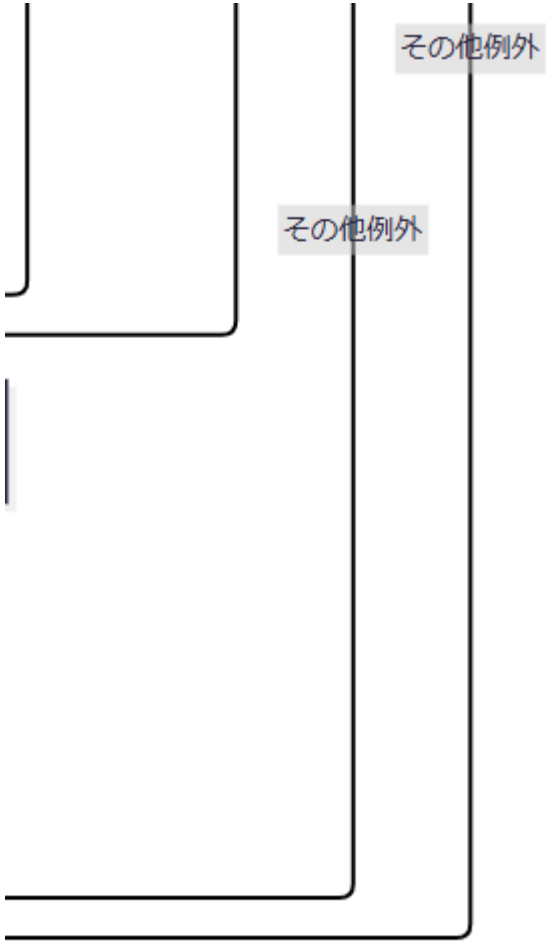
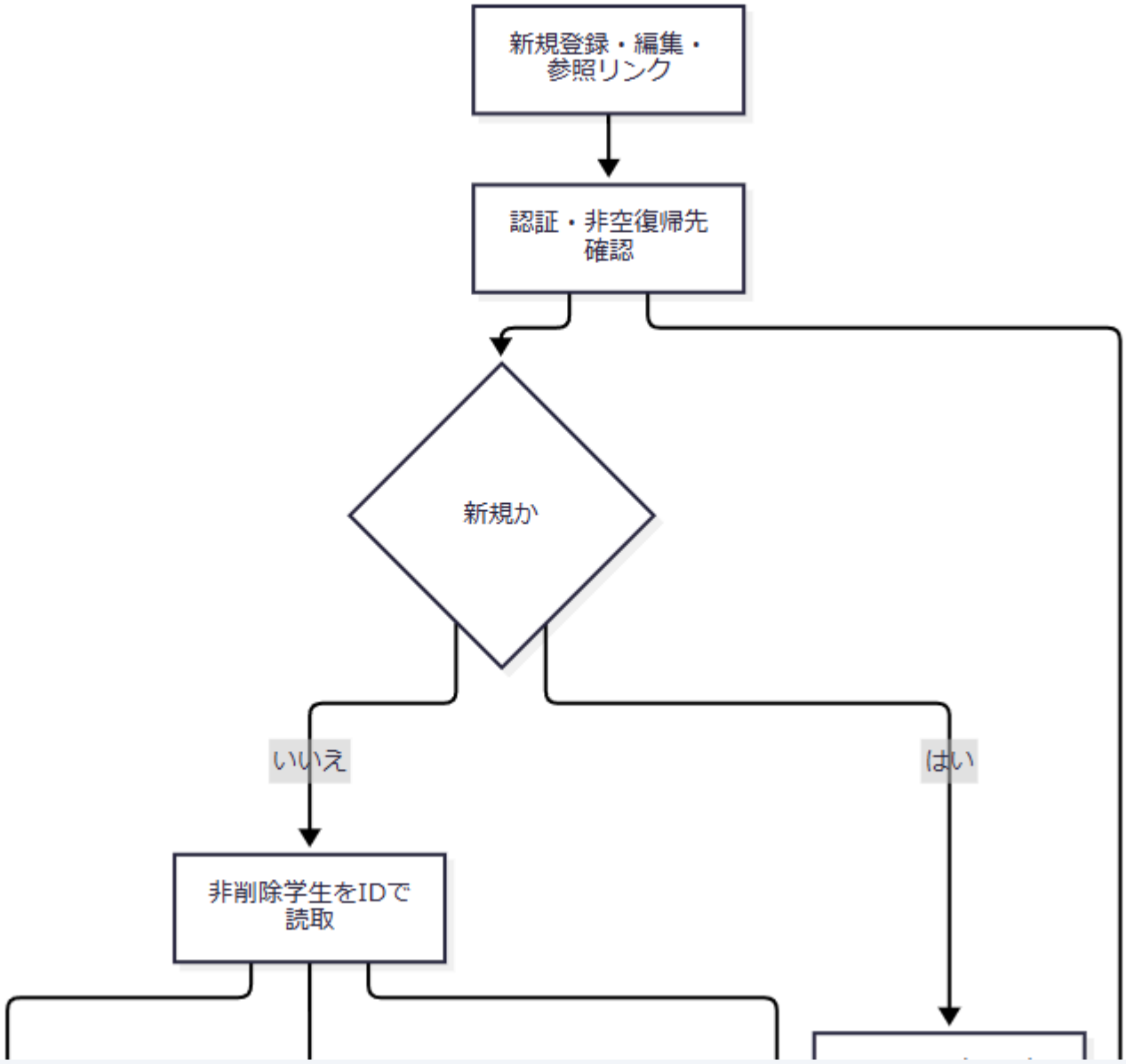

上限超過でなくPageが最終ページ超過
いいえ
待機取消
いいえ
はい
取得結果保持
条件Page補正・再検索・EnsureSearch
WebFault
Applied・LastResult更新・版加算
期限延長・排他解放・入力復元・候補読取・表示
WebFault

diagram-004 6/8




期限延長・排他解放・入力復元・候補読取・表示
WebFault
try内のWebFault
その他例外
WebFault
409・先に検索してください
そのStatusとエラーを保持
保持条件と未検索データで再表示
再表示中の例外
個別catchの対象外・外側例外経路


その他例外
その他例外


新規登録・編集・
参照リンク
認証・非空復帰先
確認
新規か
いいえ
はい
非削除学生をIDで
読取

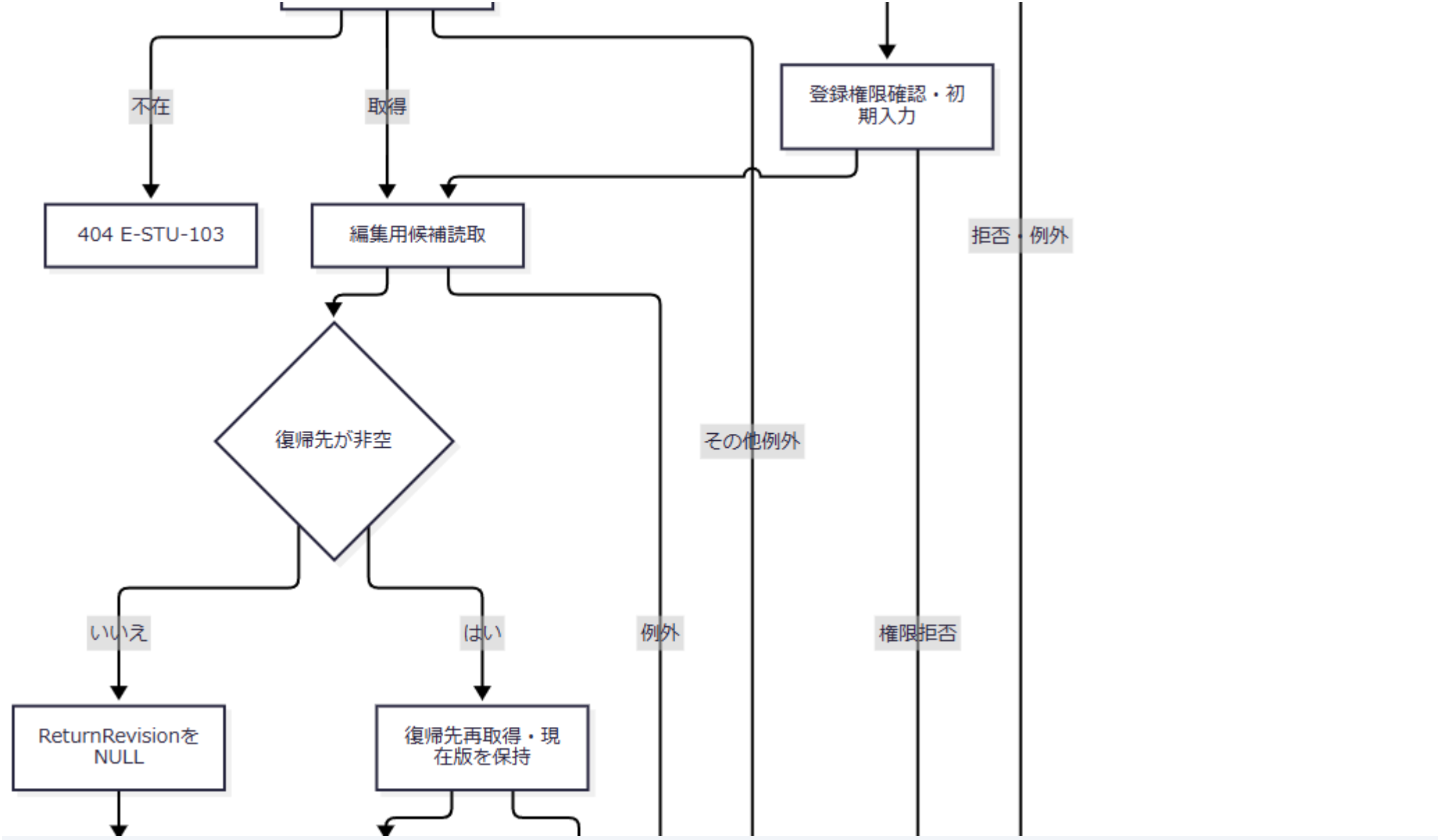

不在
取得
登録権限確認・初期入力
404 E-STU-103
編集用候補読取
拒否・例外
復帰先が非空
その他例外
いいえ
はい
例外
権限拒否
ReturnRevisionをNULL
復帰先再取得・現在版を保持

diagram-007 3/4

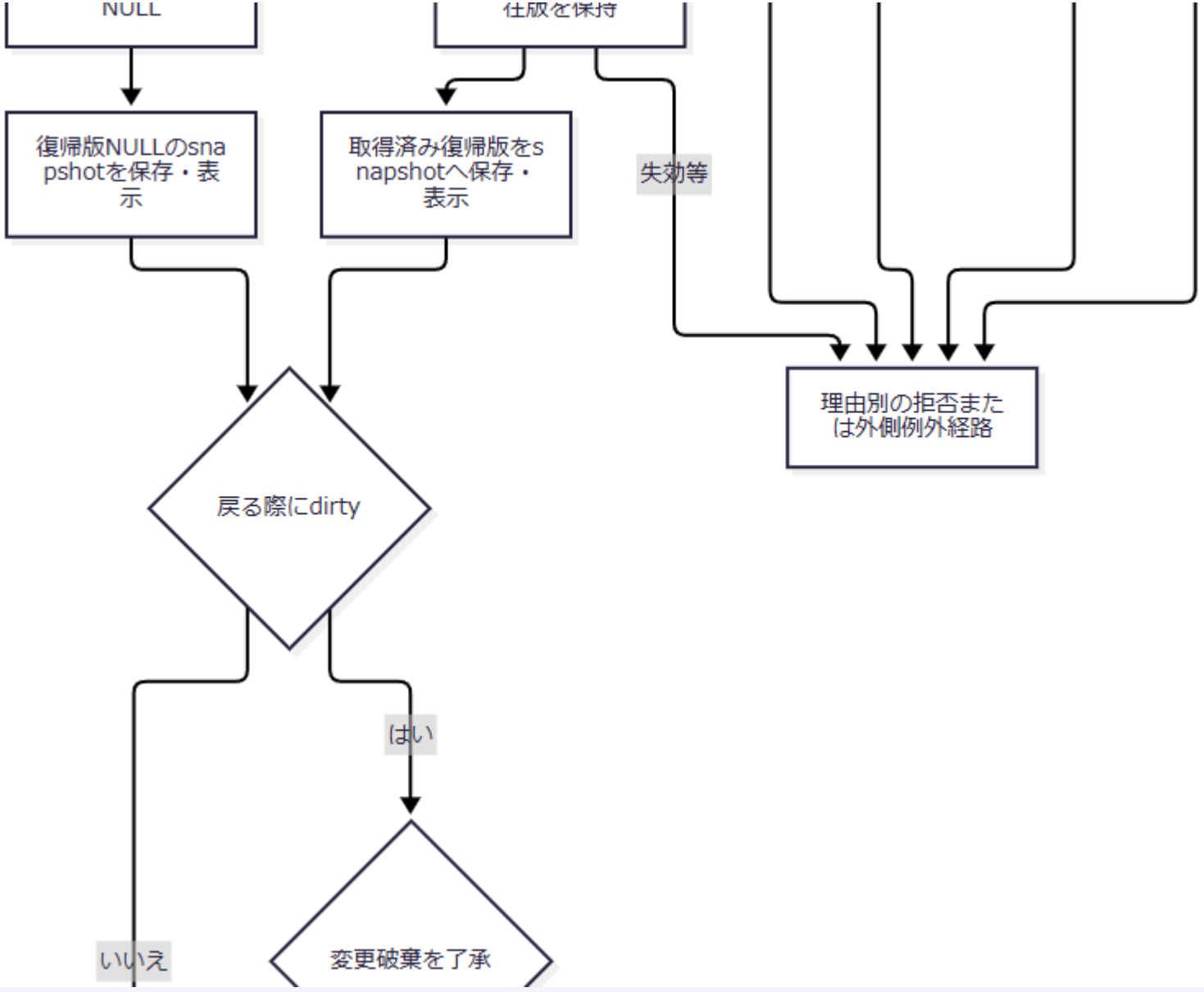
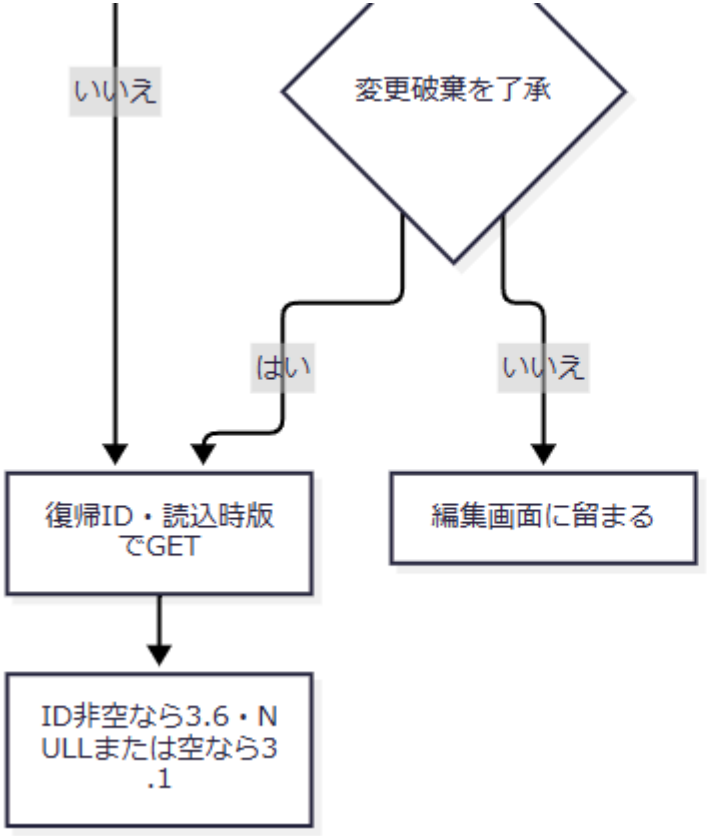


diagram-007 4/4



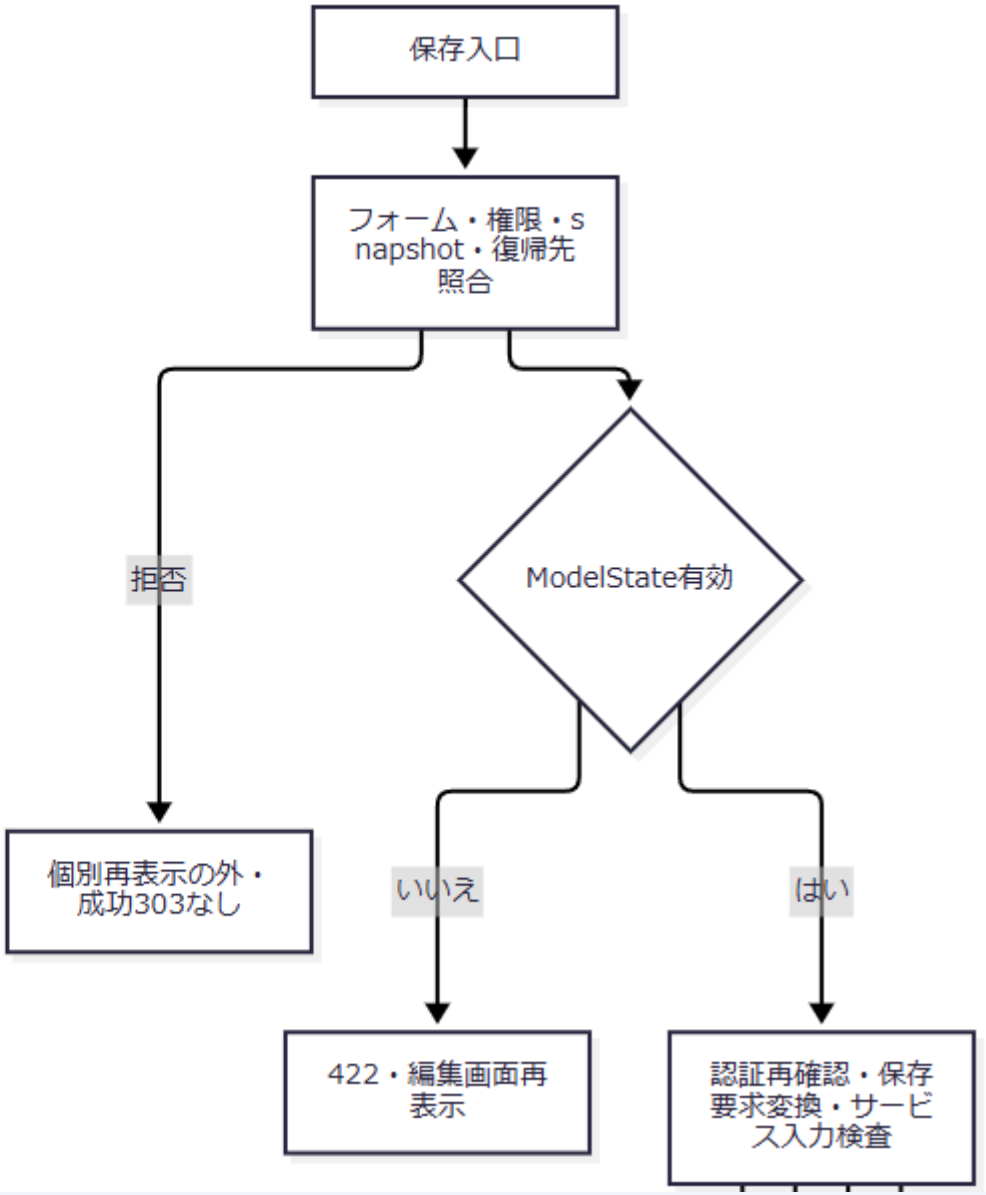

保存入口
フォーム・権限・snapshot・復帰先照合
拒否
ModelState有効
個別再表示の外・成功303なし
いいえ
はい
422・編集画面再表示
認証再確認・保存要求変換・サービス入力検査

diagram-008 2/21

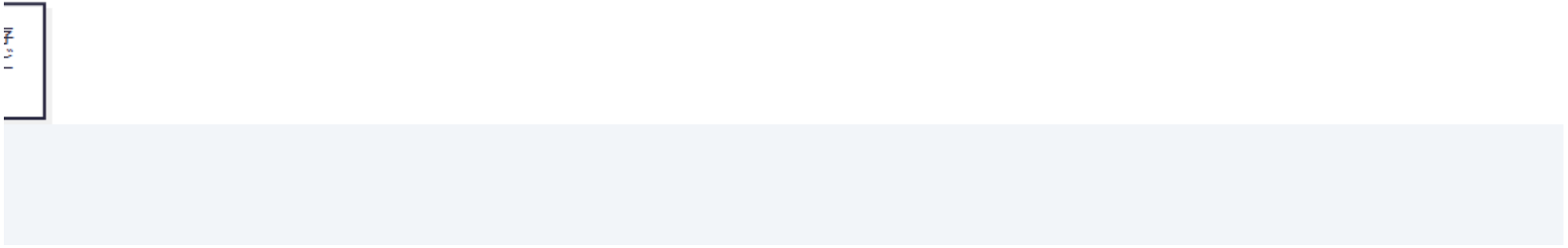


diagram-008 3/21

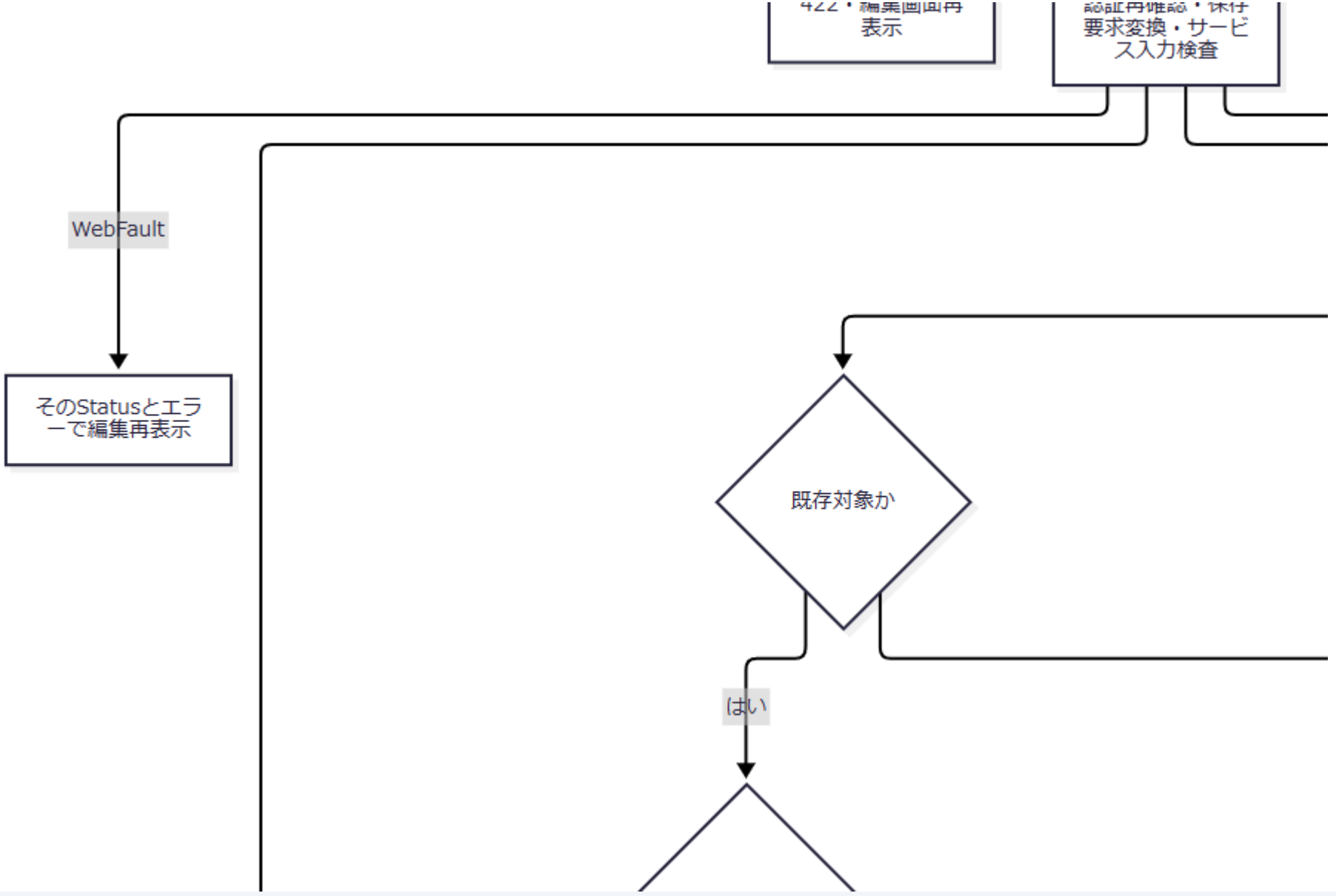

表示
要求変換・サービス入力検査
WebFault
そのStatusとエラーで編集再表示
既存対象か
はい

diagram-008 5/21

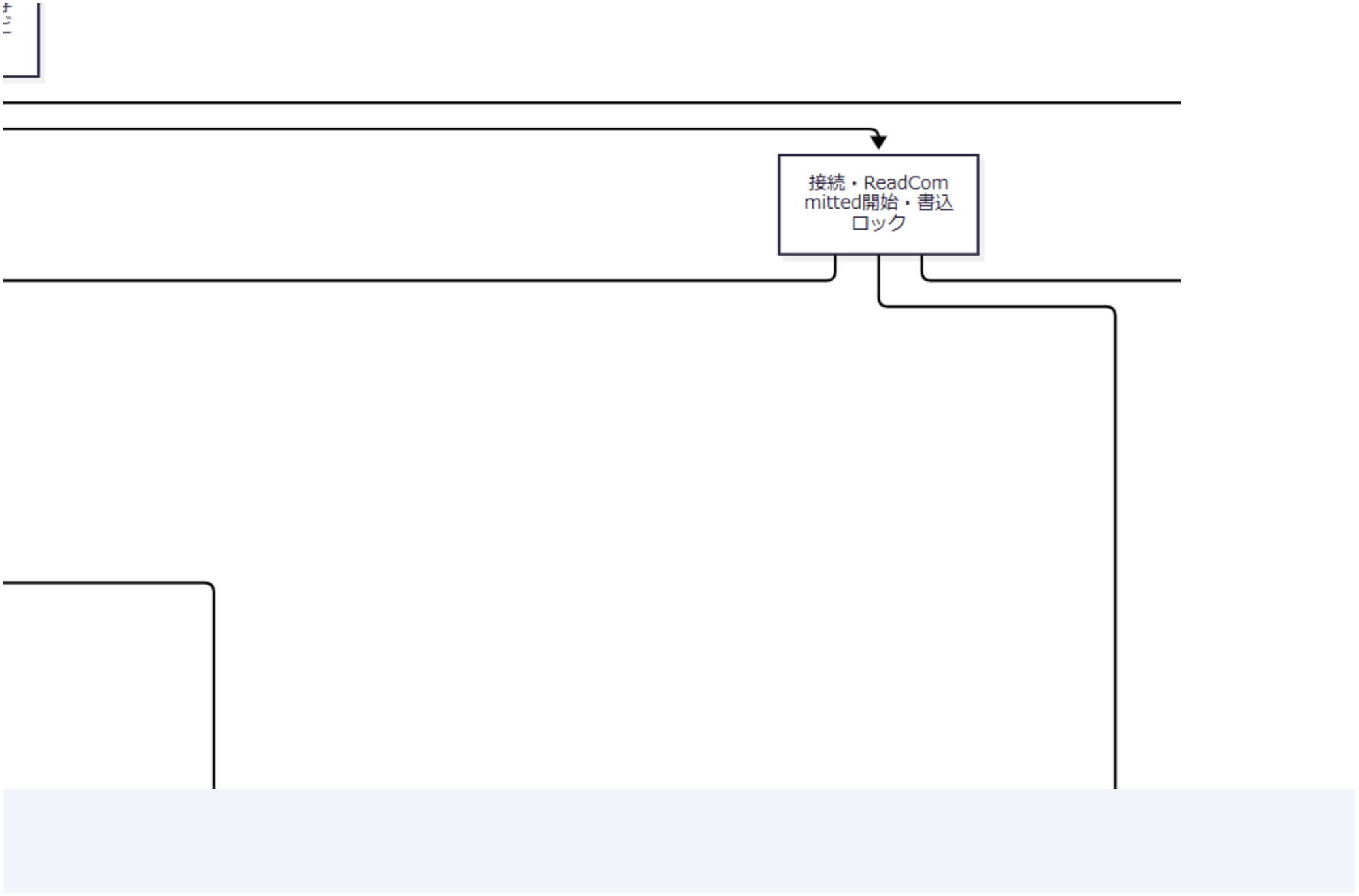
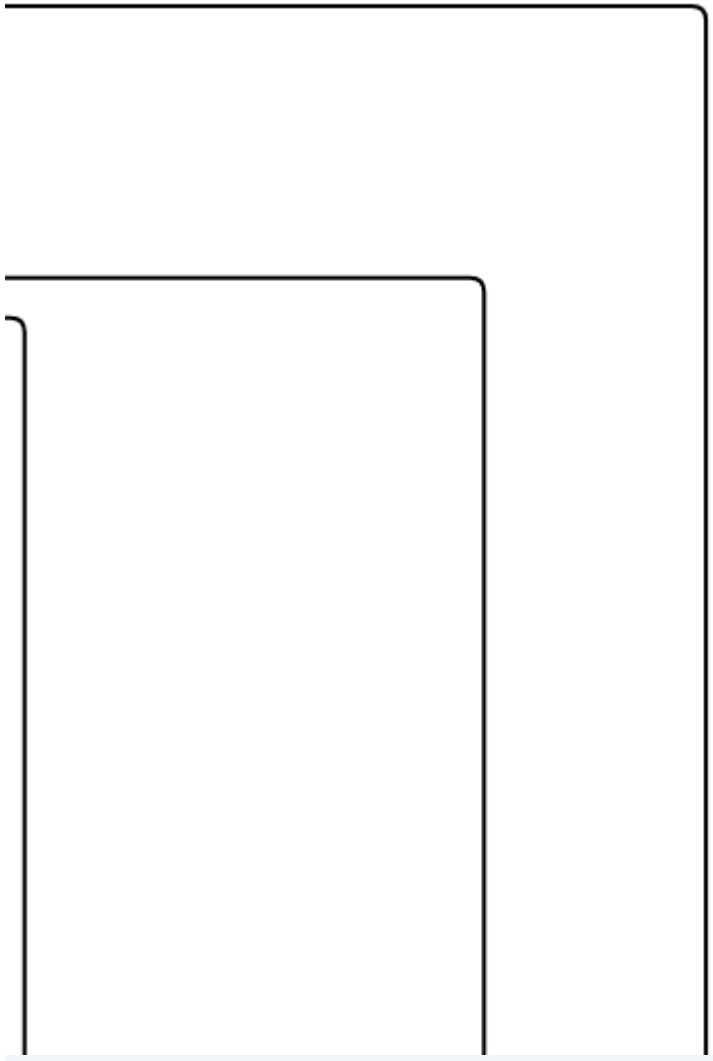
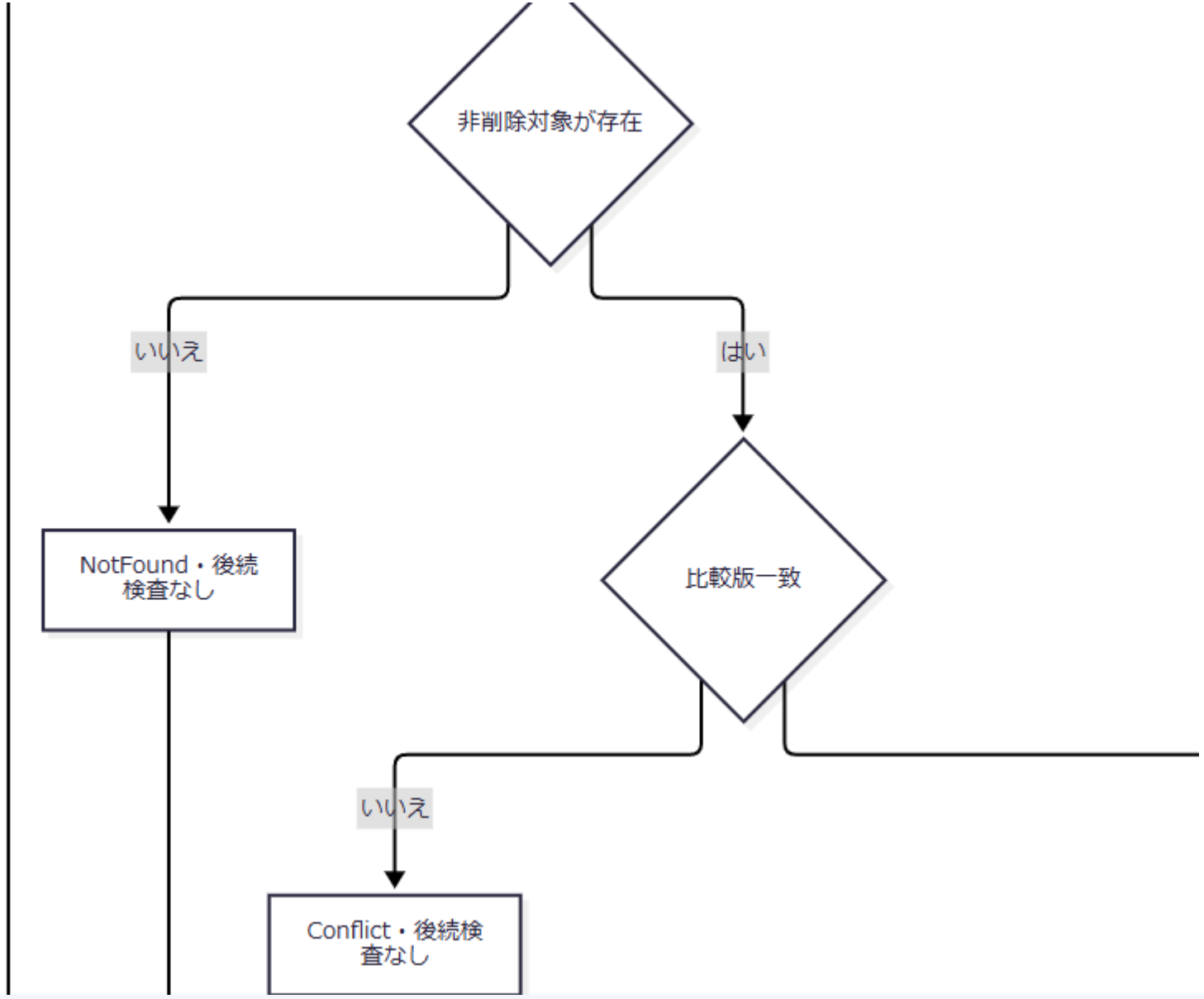
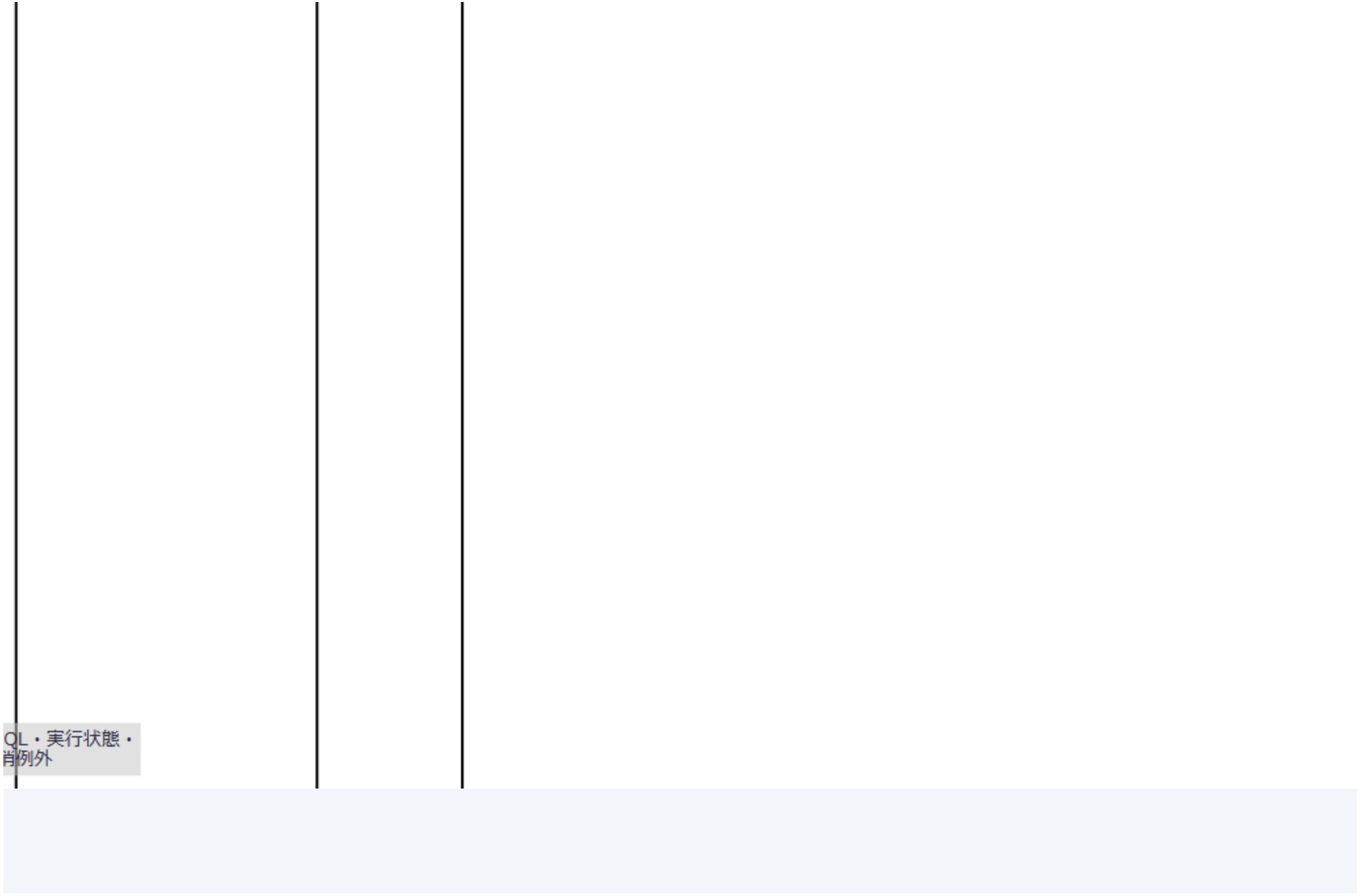


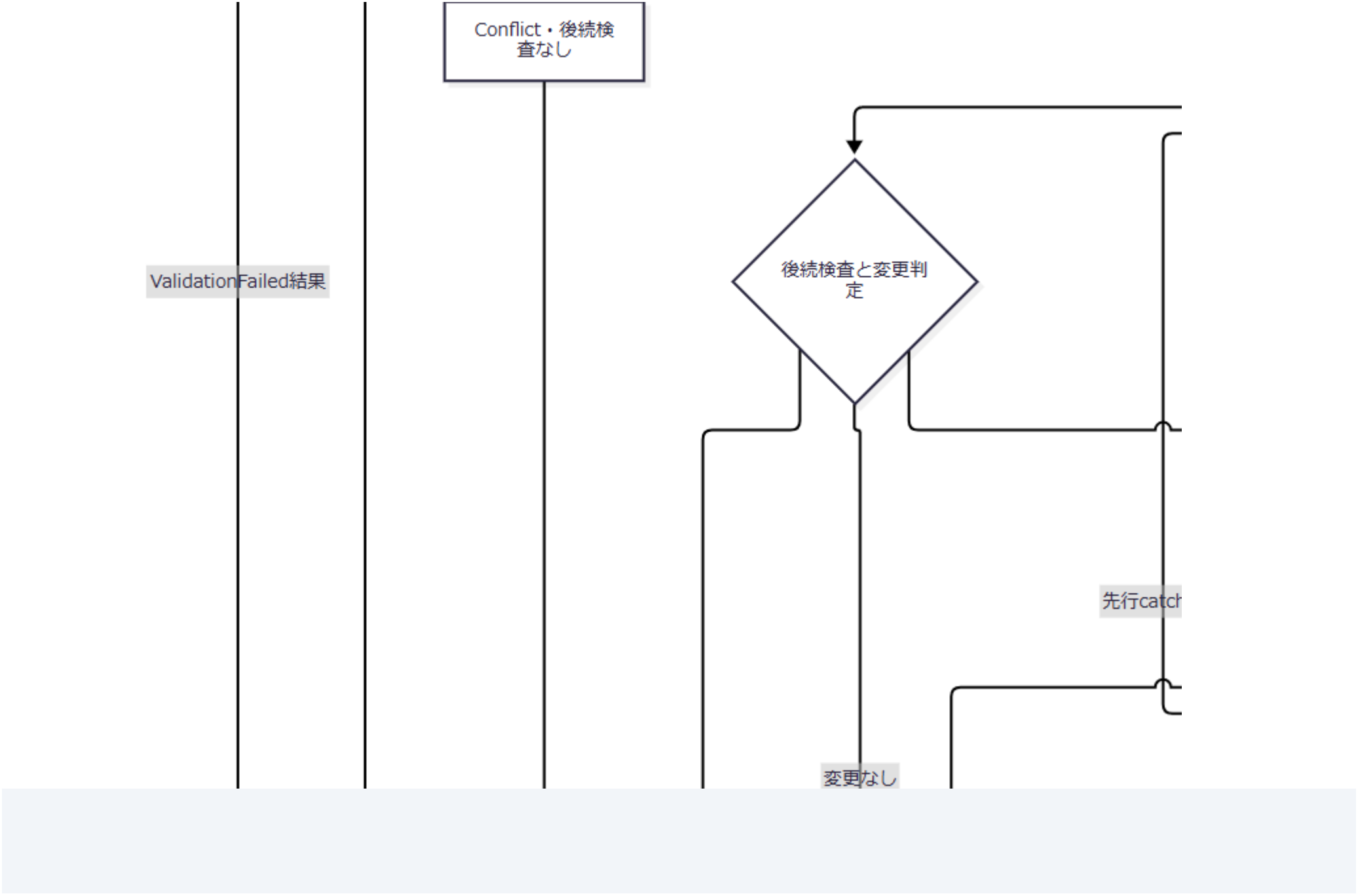
diagram-008 6/21

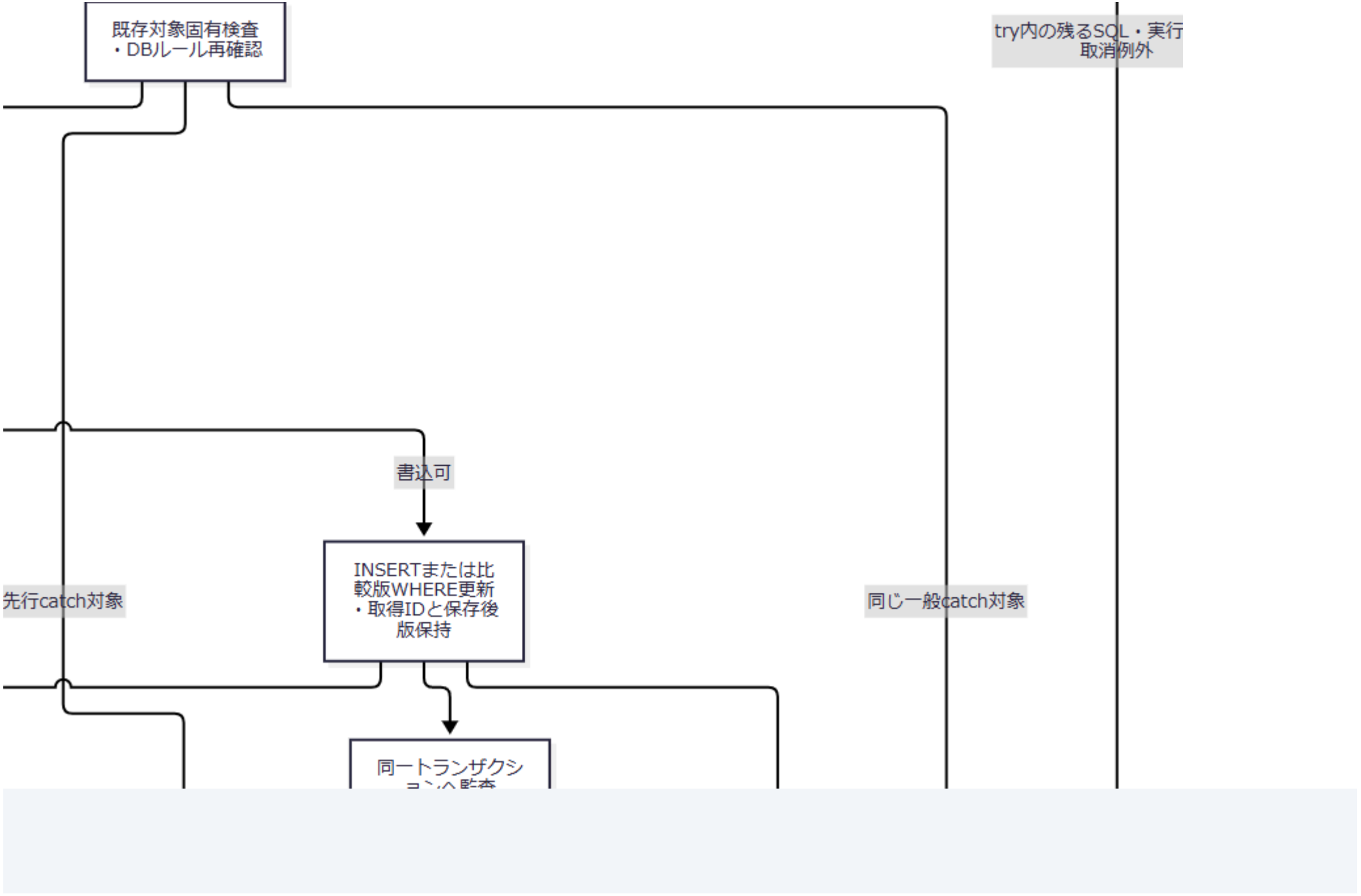



非削除対象が存在
いいえ
はい
NotFound・後続
検査なし
比較版一致
いいえ
Conflict・後続検
査なし

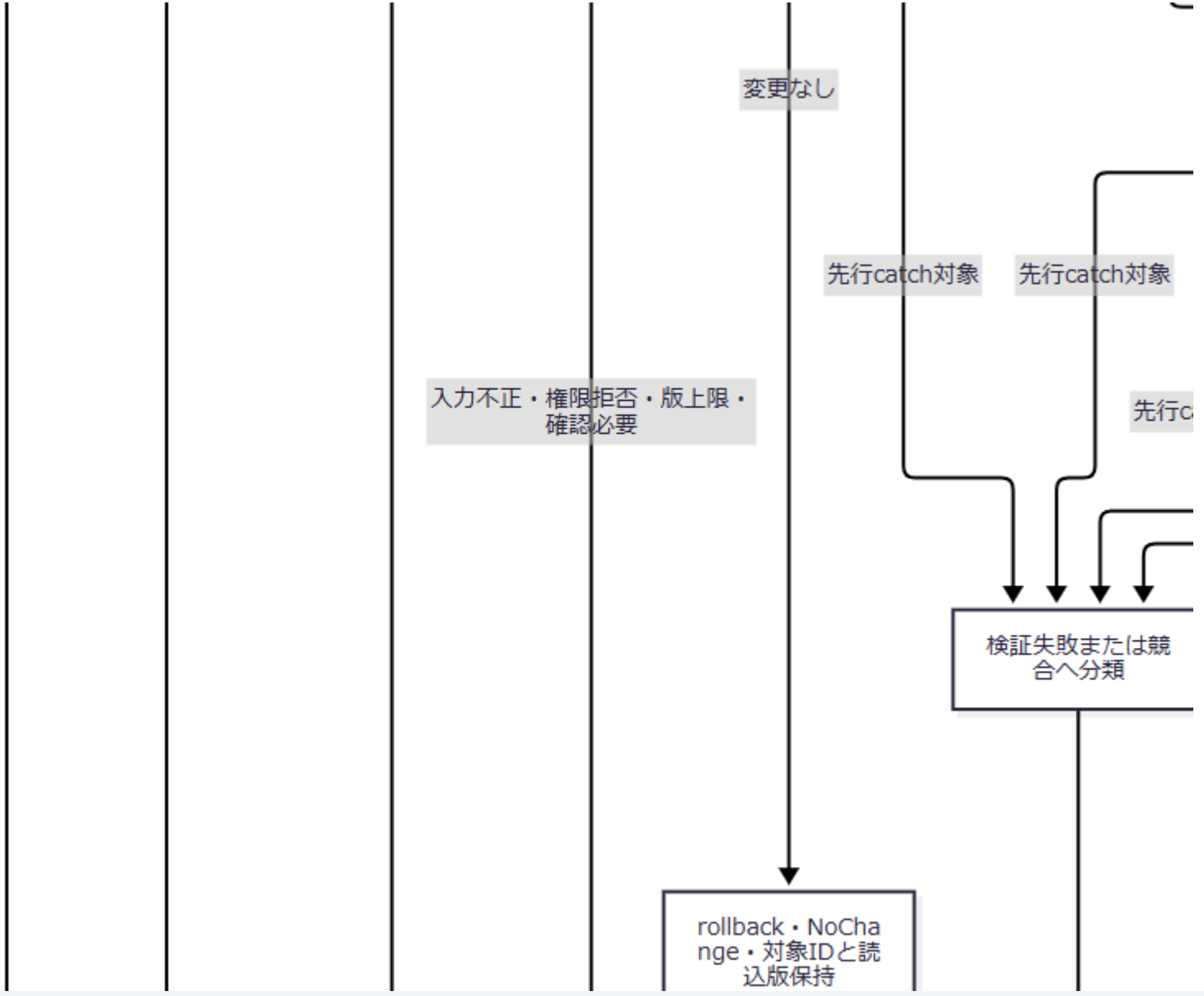

新規
はい
既存対象固有検査
・DBルール再確認
try内の残るSQL・実行
取消例外


QL・実行状態・
例外


Conflict・後続検査なし
ValidationFailed結果
後続検査と変更判定
先行catch
変更なし


既存対象固有検査
・DBルール再確認
try内の残るSQL・実行
取消例外
書込可
INSERTまたは比
較版WHERE更新
・取得IDと保存後
版保持
先行catch対象
同じ一般catch対象
同一トランザクシ


QL・実行状態・
肖例外
その他未捕捉例外
接続・開始等のcatch外例外


変更なし
先行catch対象
先行catch対象
入力不正・権限拒否・版上限・
確認必要
検証失敗または競
合へ分類
rollback・NoCha
nge・対象IDと読
込版保持

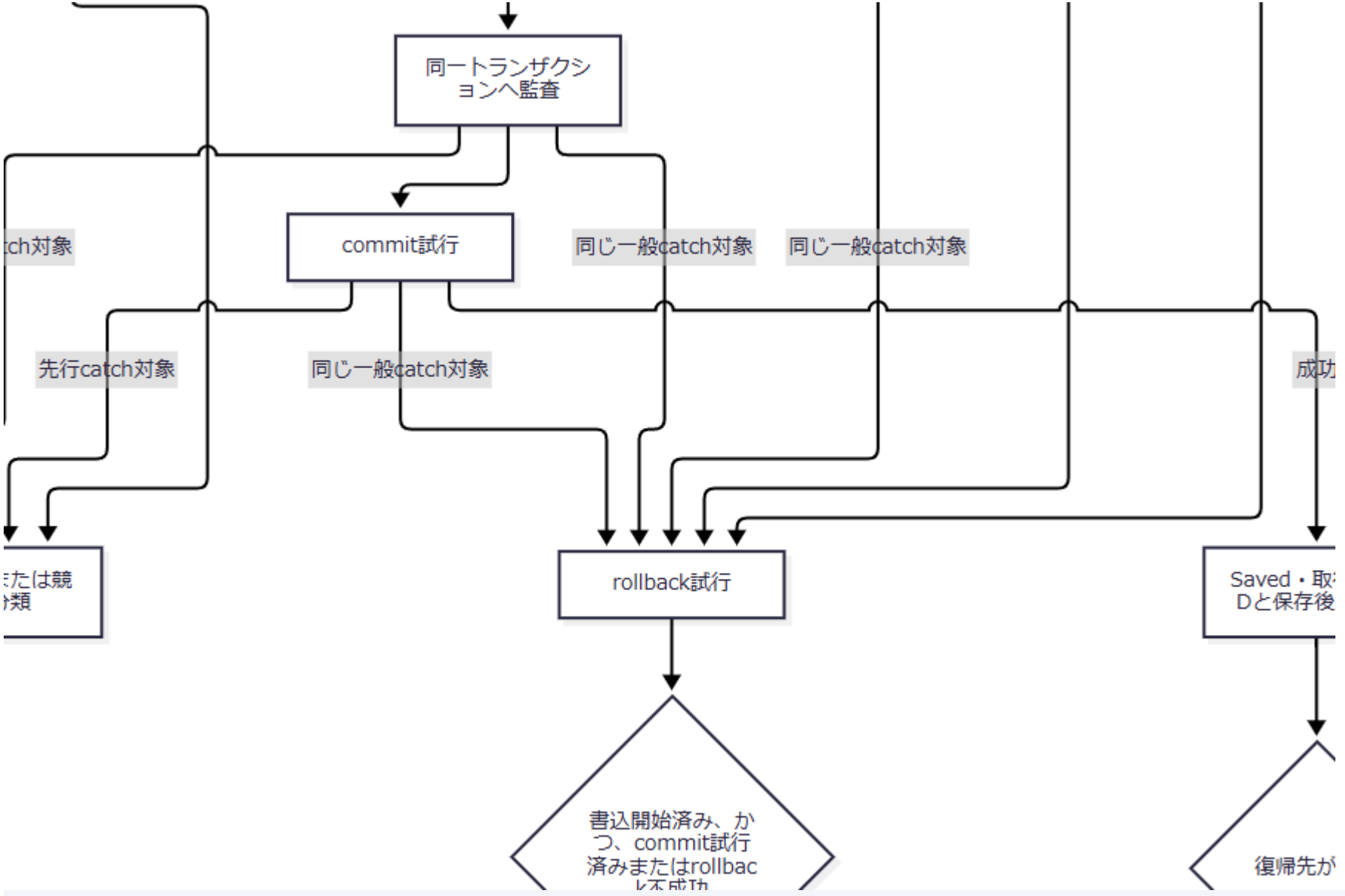
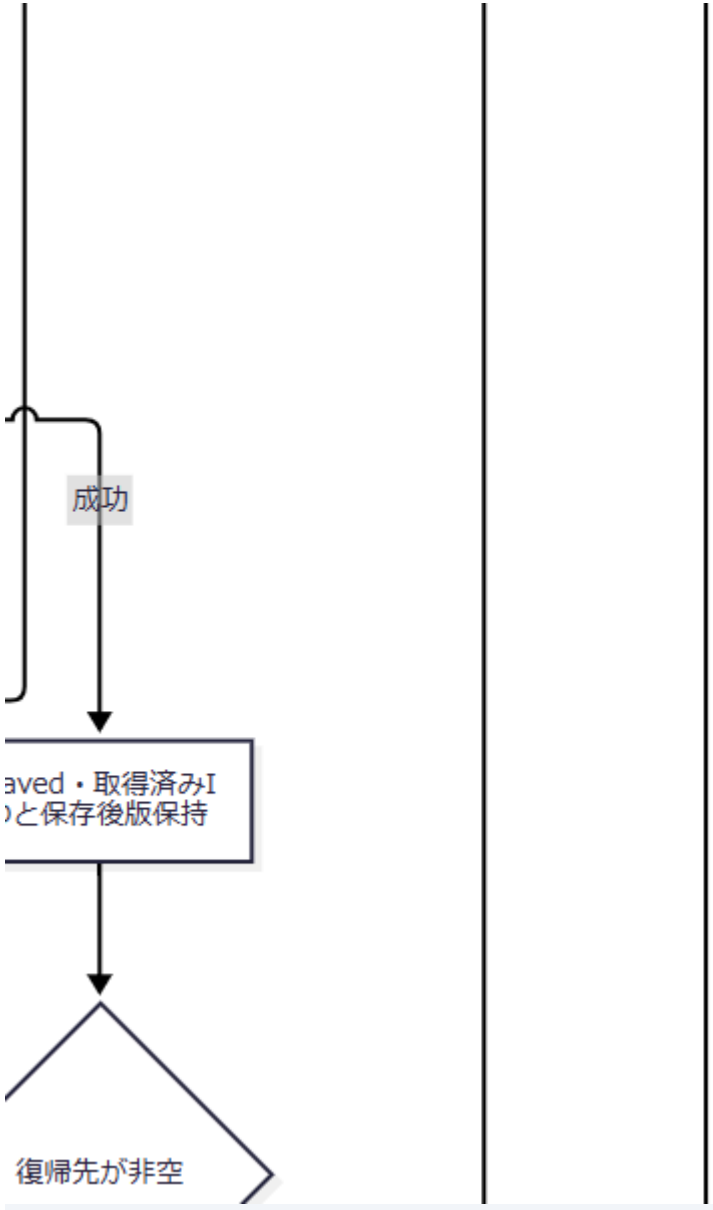
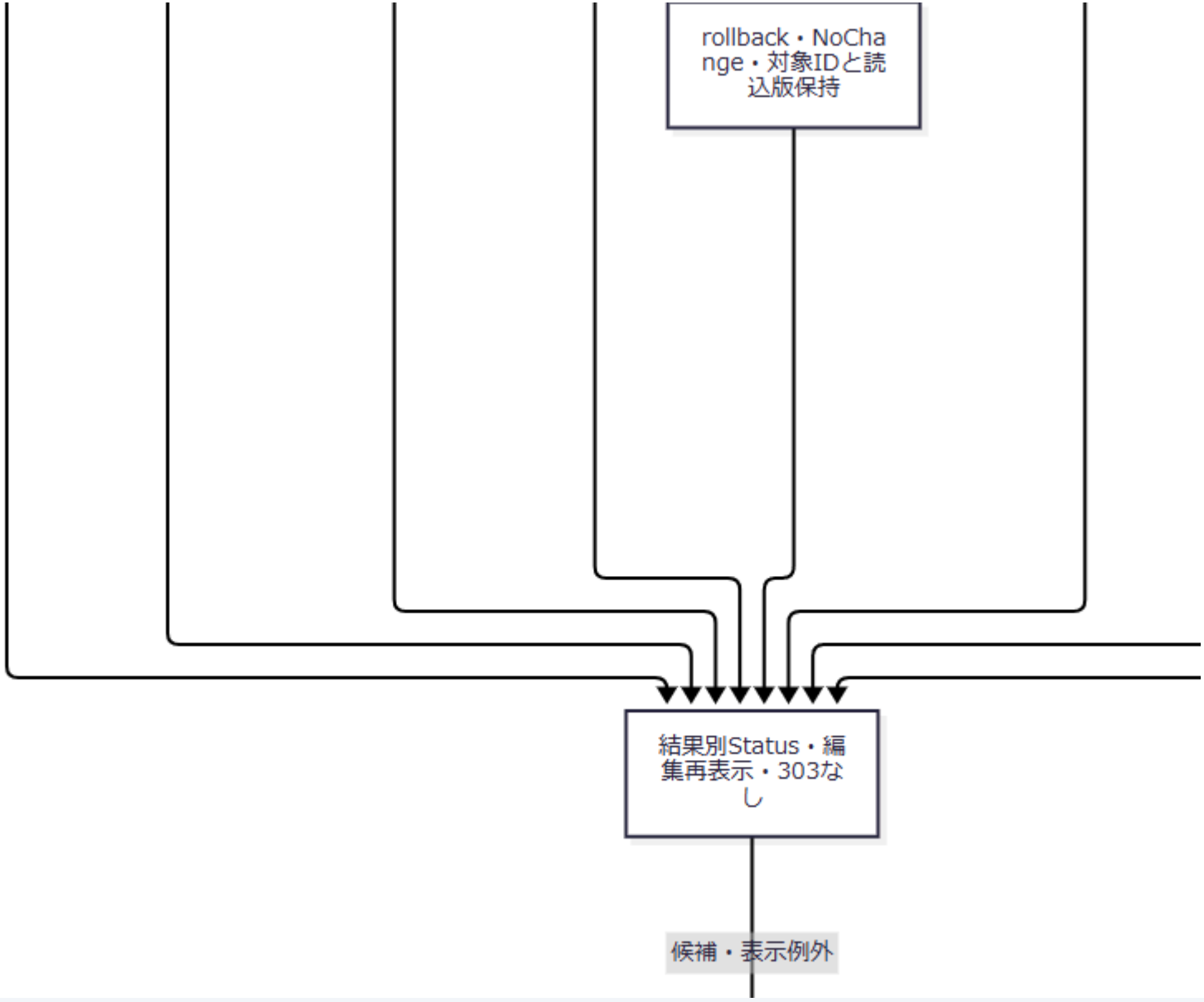
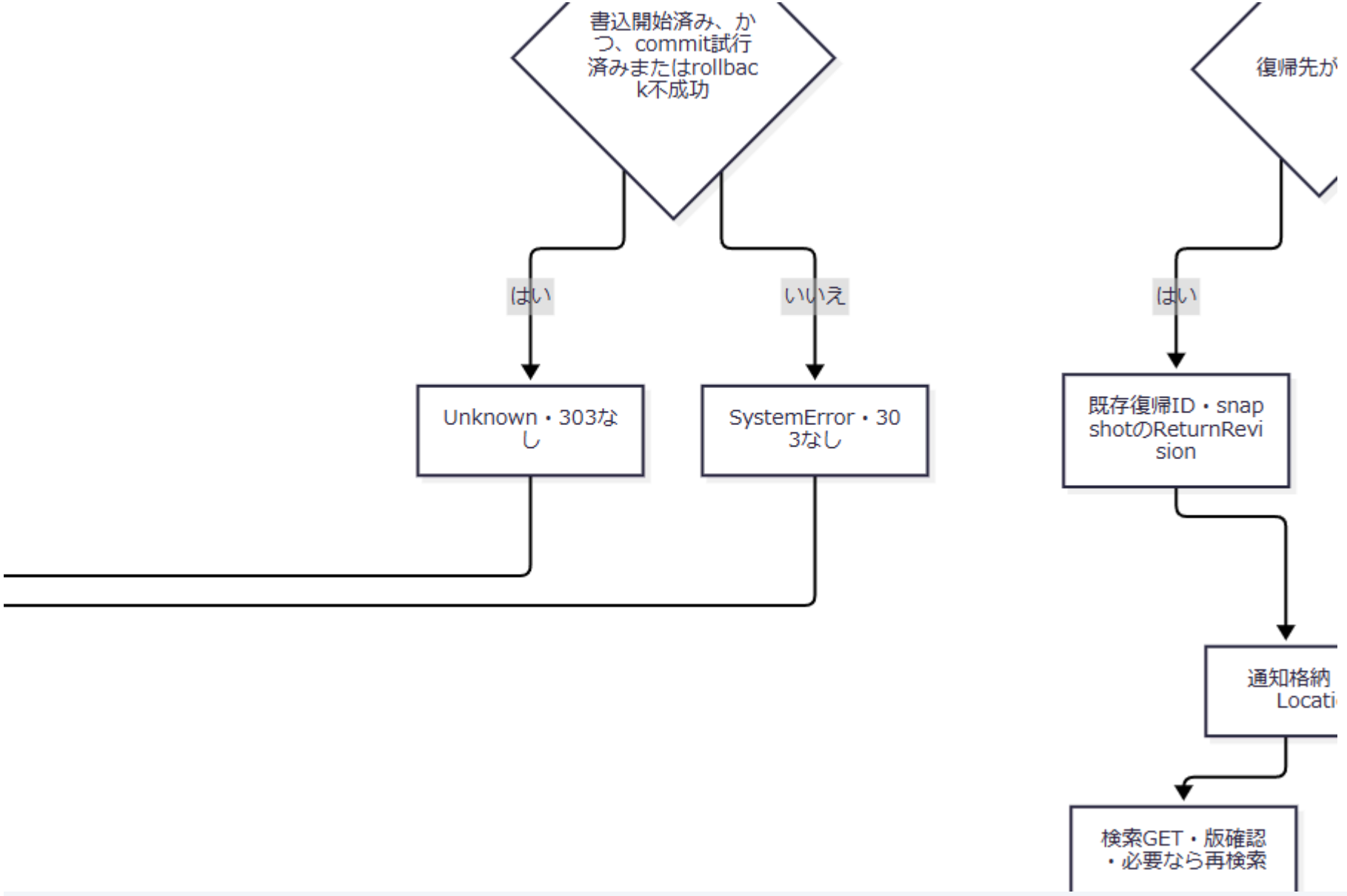
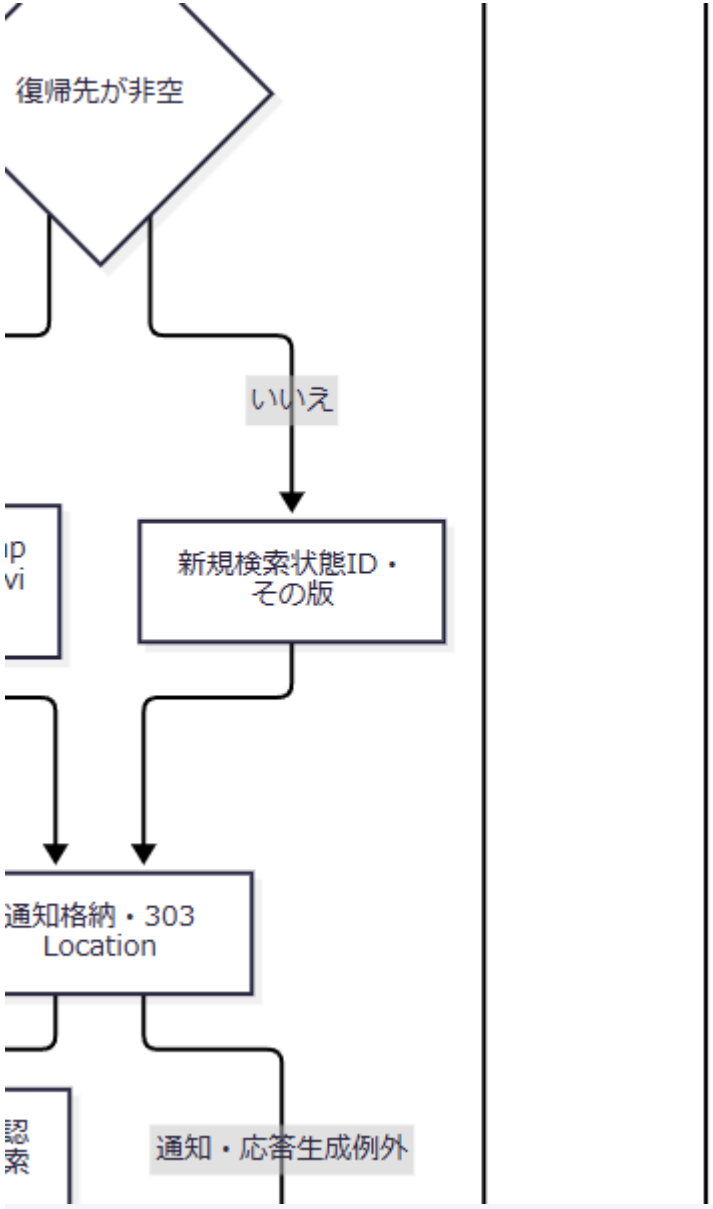

同一トランザクションへ監査
commit試行
同じ一般catch対象
同じ一般catch対象
先行catch対象
同じ一般catch対象
rollback試行
書込開始済み、かつ、commit試行済みまたはrollbac
Saved・取
Dと保存後
復帰先が

diagram-008 15/21




rollback・NoCha
nge・対象IDと読
込版保持
結果別Status・編
集再表示・303な
し
候補・表示例外

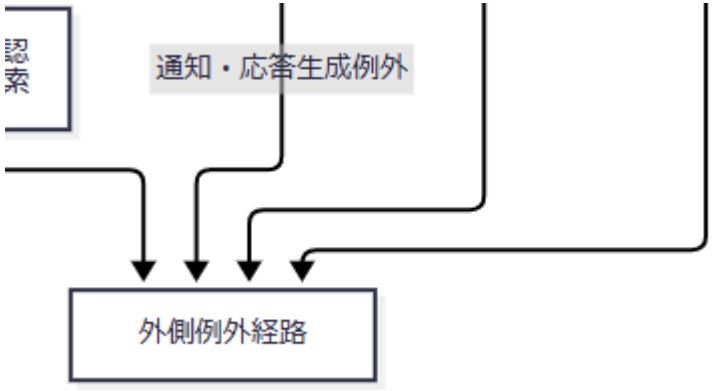

書込開始済み、かつ、commit試行済みまたはrollback不成功
はい
いいえ
Unknown・303なし
SystemError・303なし
復帰先が
はい
既存復帰ID・snapshotのReturnRevision
通知格納
Locati
検索GET・版確認
・必要なら再検索

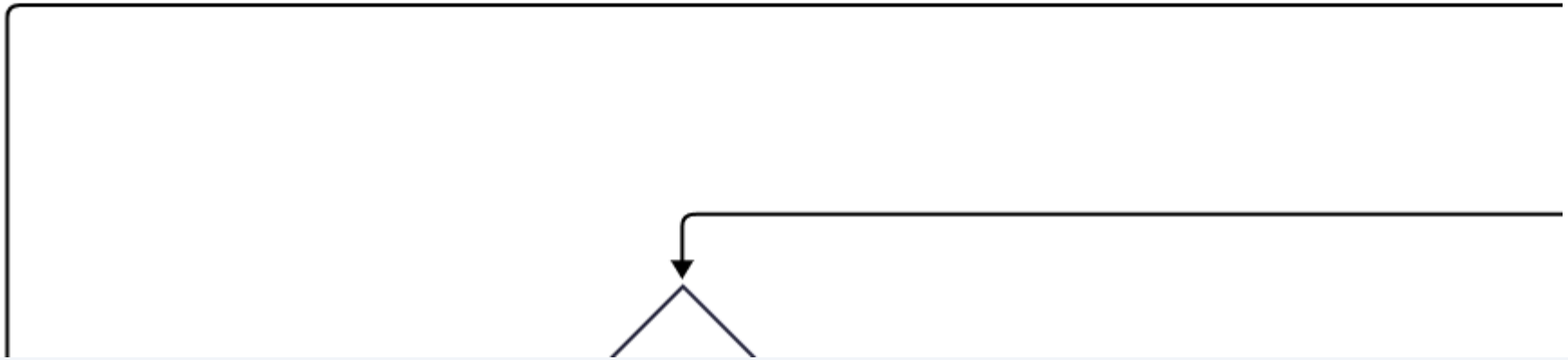

復帰先が非空
いいえ
新規検索状態ID・
その版
通知格納・303
Location
通知・応答生成例外


候補・表示例外


検索GET・版確認
・必要なら再検索

diagram-008 21/21





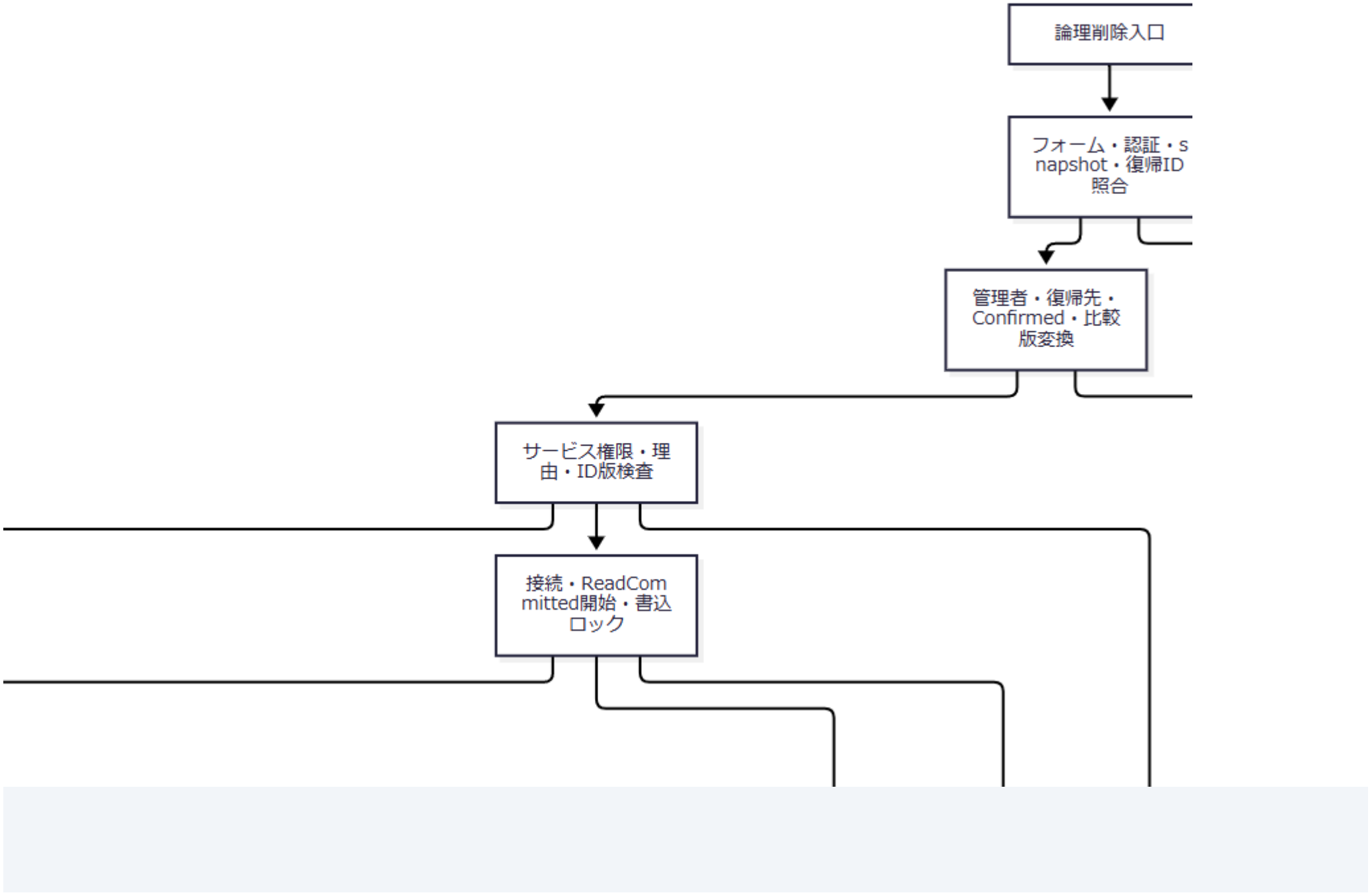

論理削除入口
フォーム・認証・snapshot・復帰ID照合
管理者・復帰先・Confirmed・比較版変換
サービス権限・理由・ID版検査
接続・ReadCommitted開始・書込ロック

diagram-010 3/15

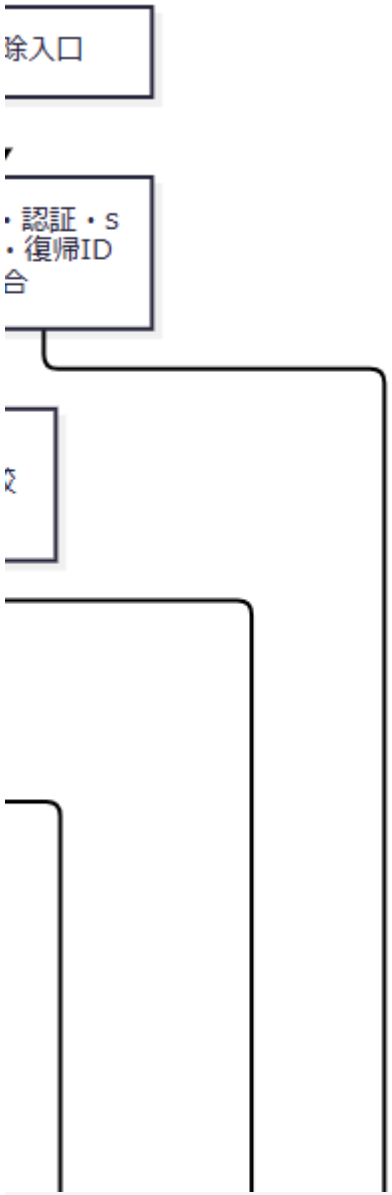
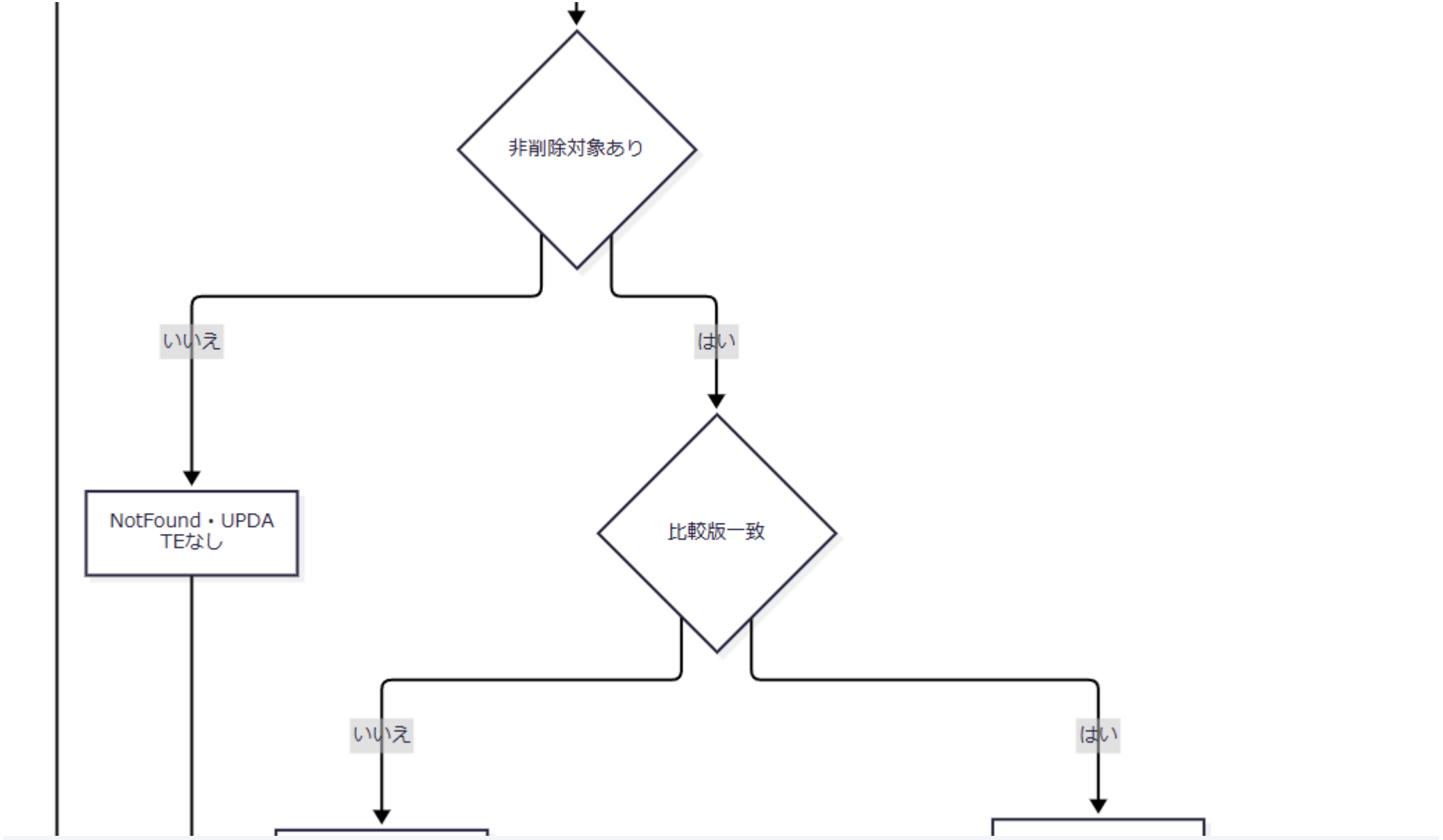


diagram-010 4/15



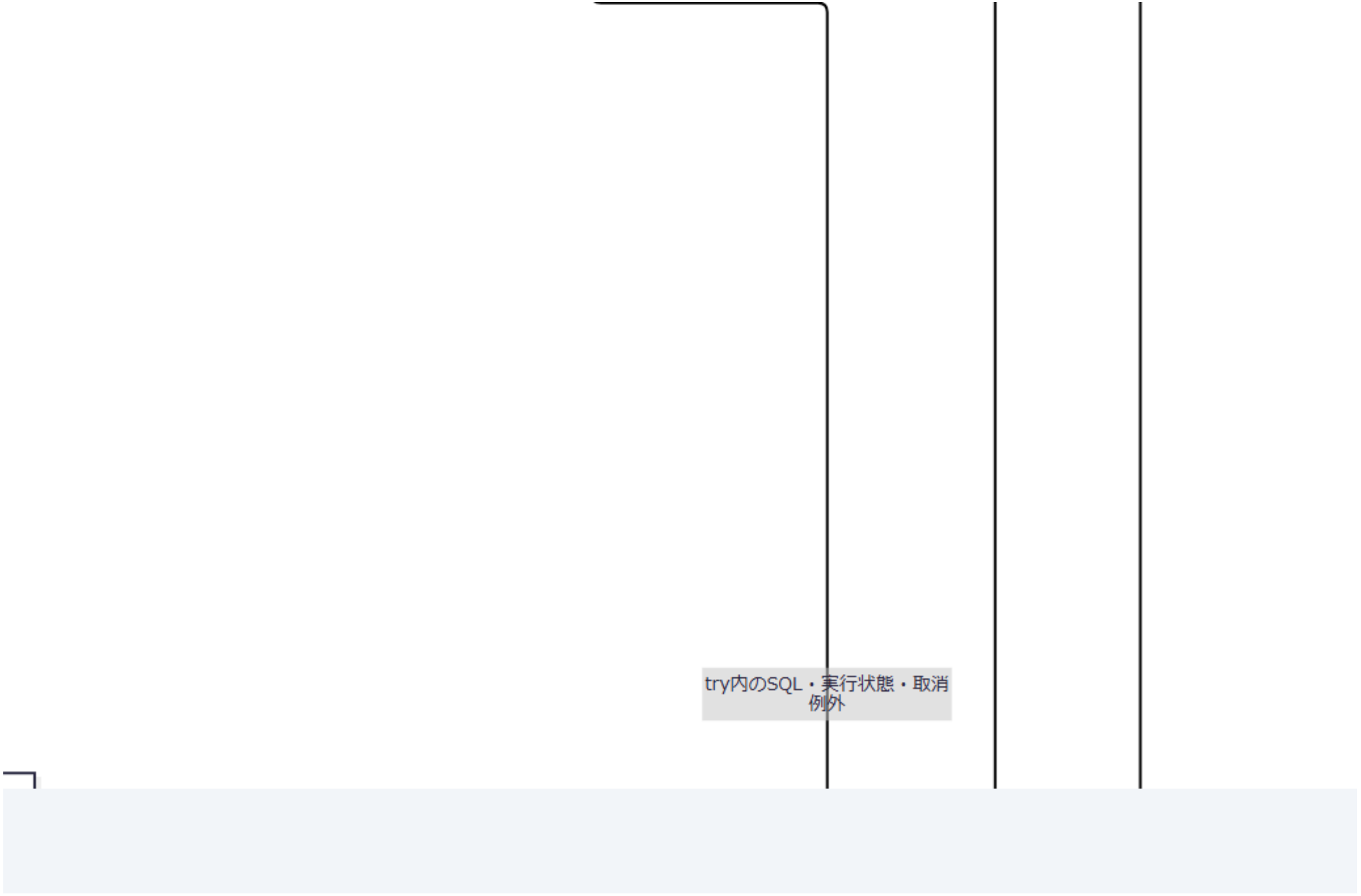
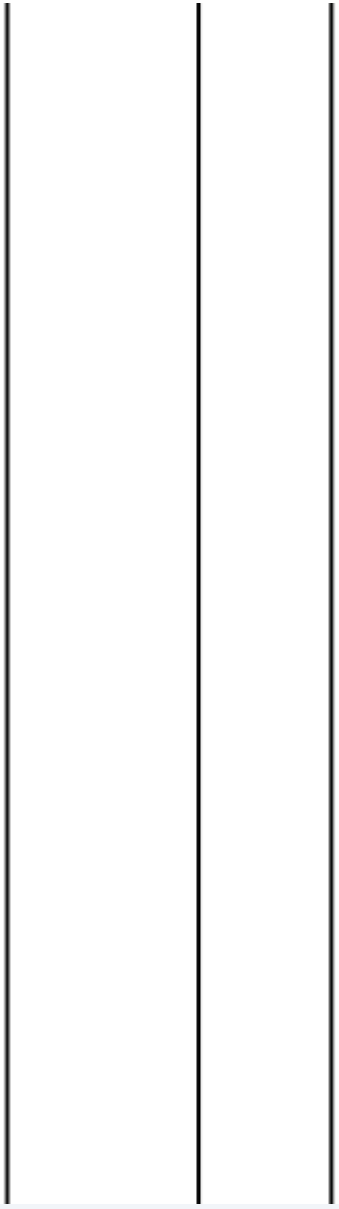
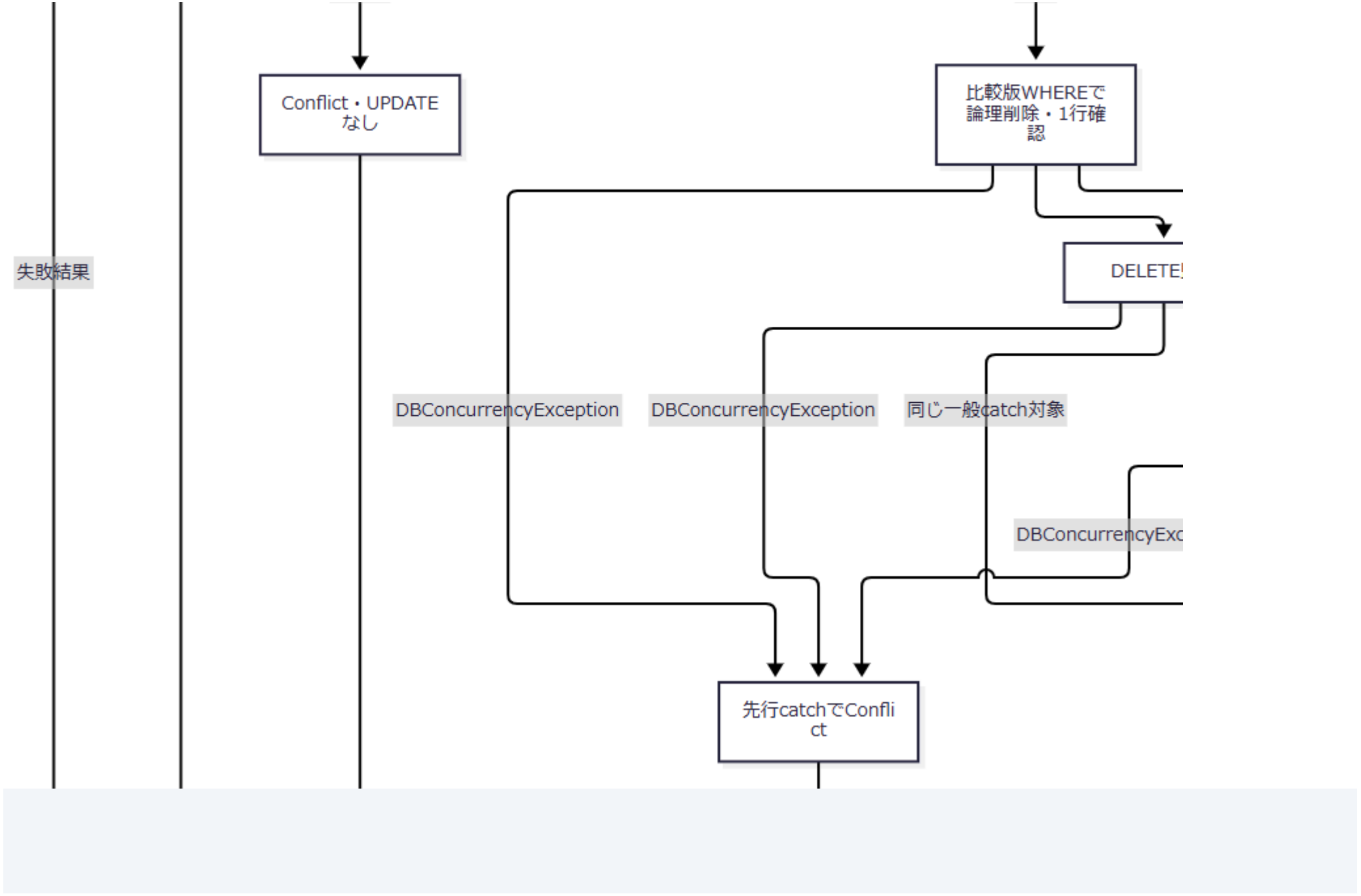

try内のSQL・実行状態・取消
例外

diagram-010 6/15




Conflict・UPDATE
なし
比較版WHEREで
論理削除・1行確
認
失敗結果
DELETE
DBConcurrencyException
DBConcurrencyException
同じ一般catch対象
DBConcurrencyExc
先行catchでConfli
ct

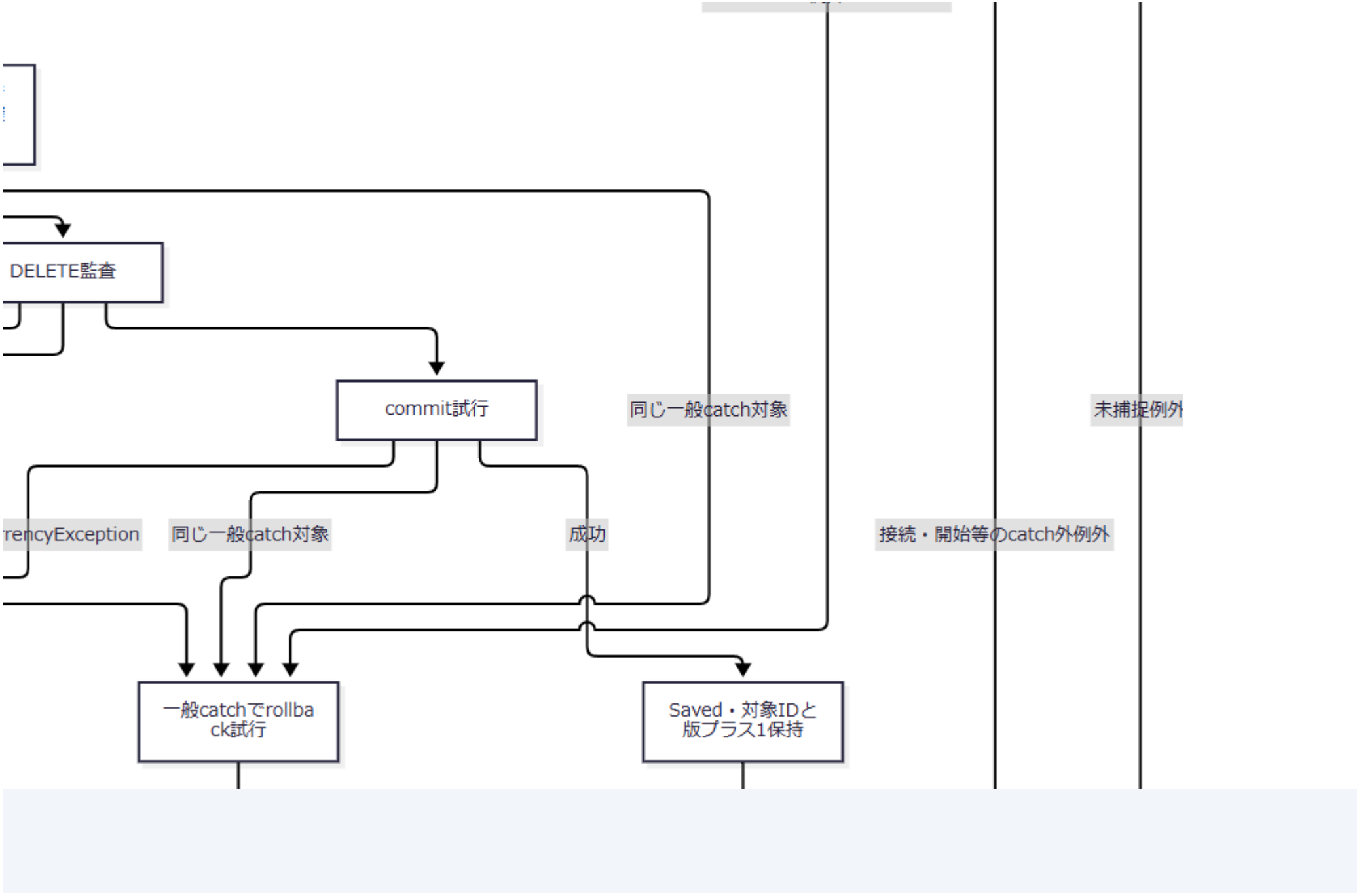
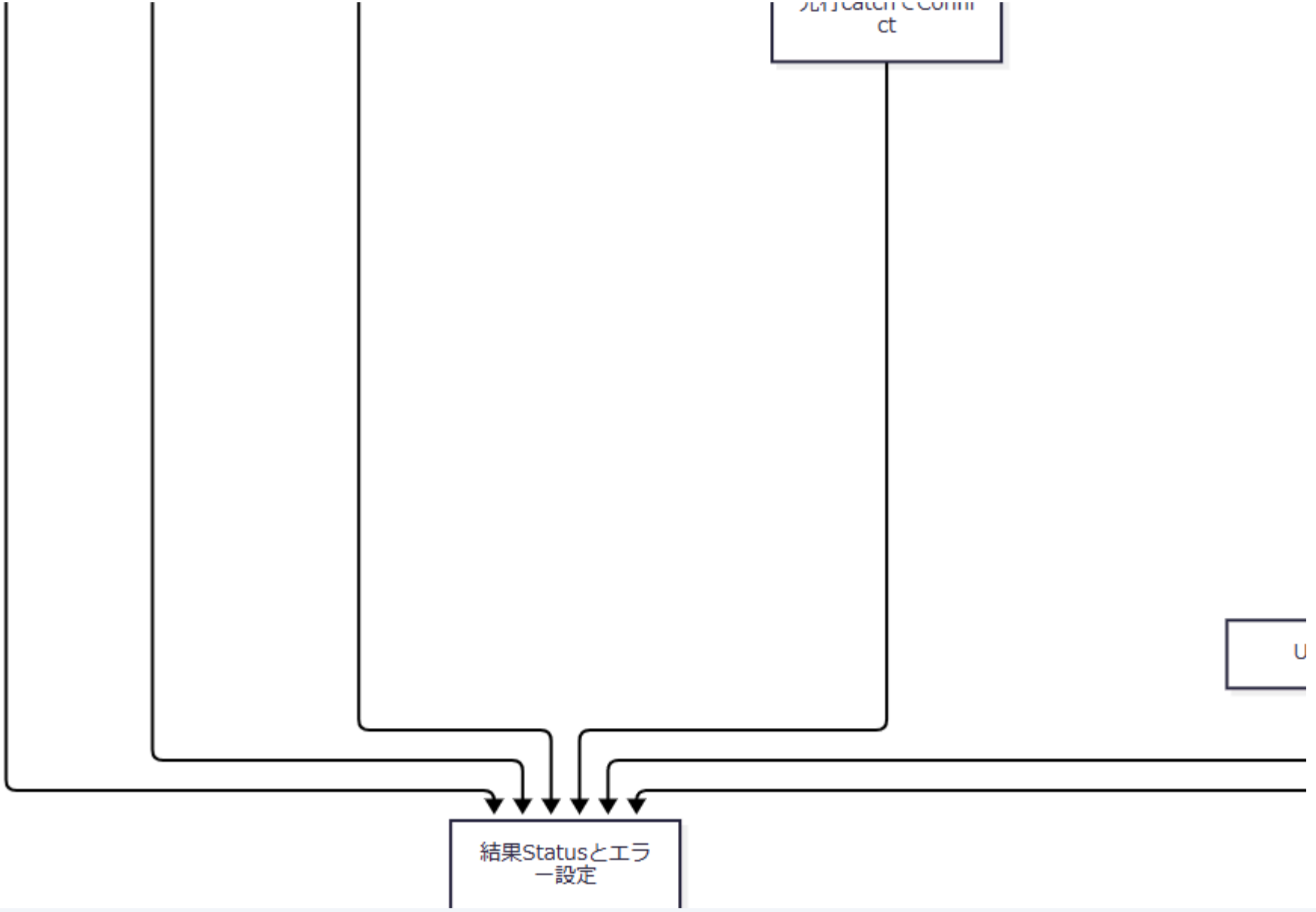

DELETE監査
commit試行
同じ一般catch対象
未捕捉例外
rencyException
同じ一般catch対象
成功
接続・開始等のcatch外例外
一般catchでrollba
ck試行
Saved・対象IDと
版プラス1保持

diagram-010 9/15




ct
結果Statusとエラー設定

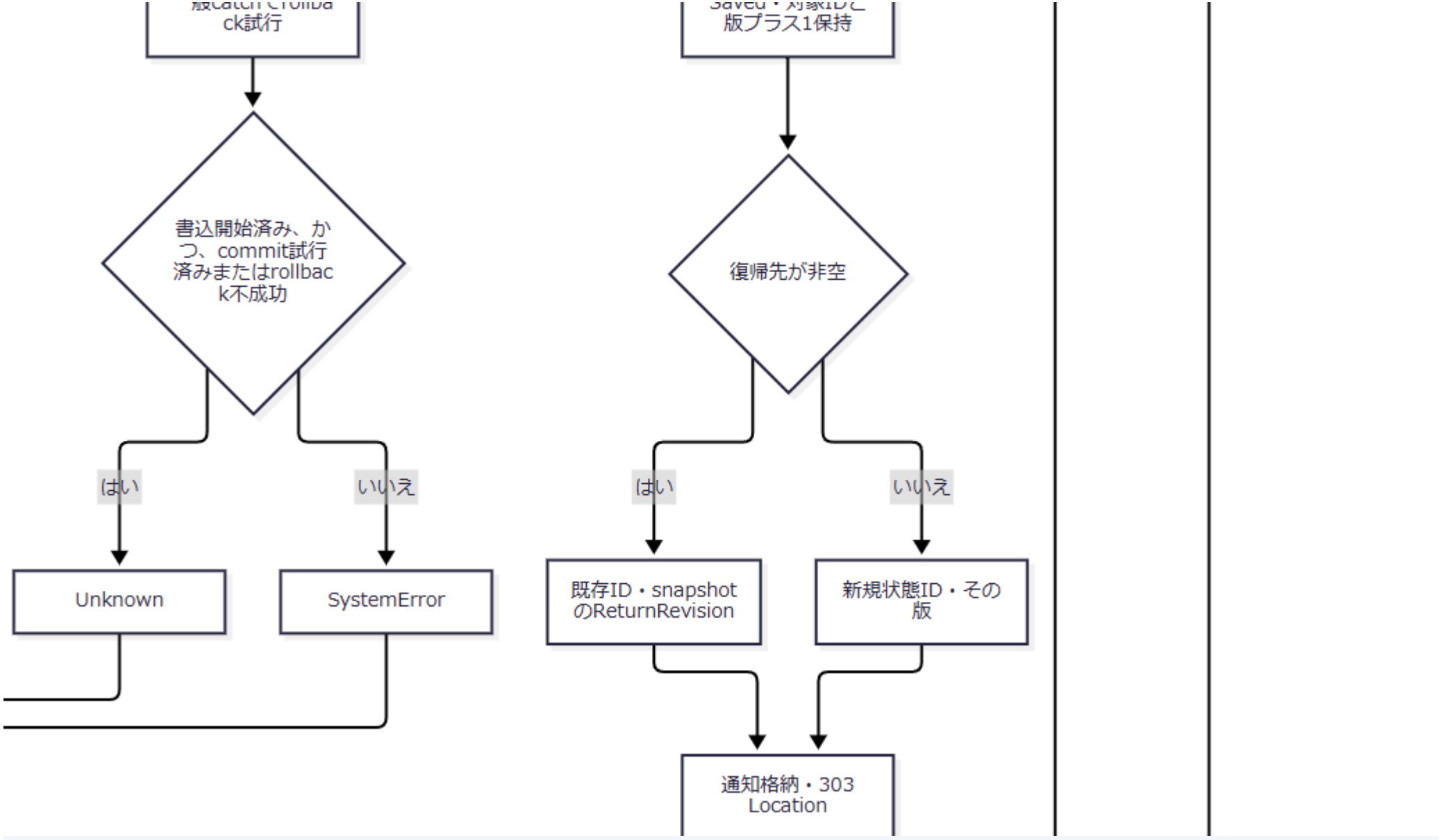
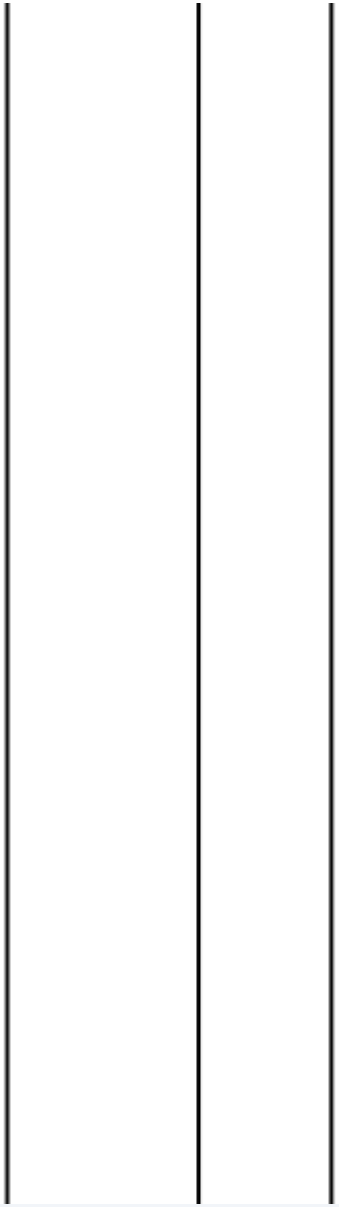
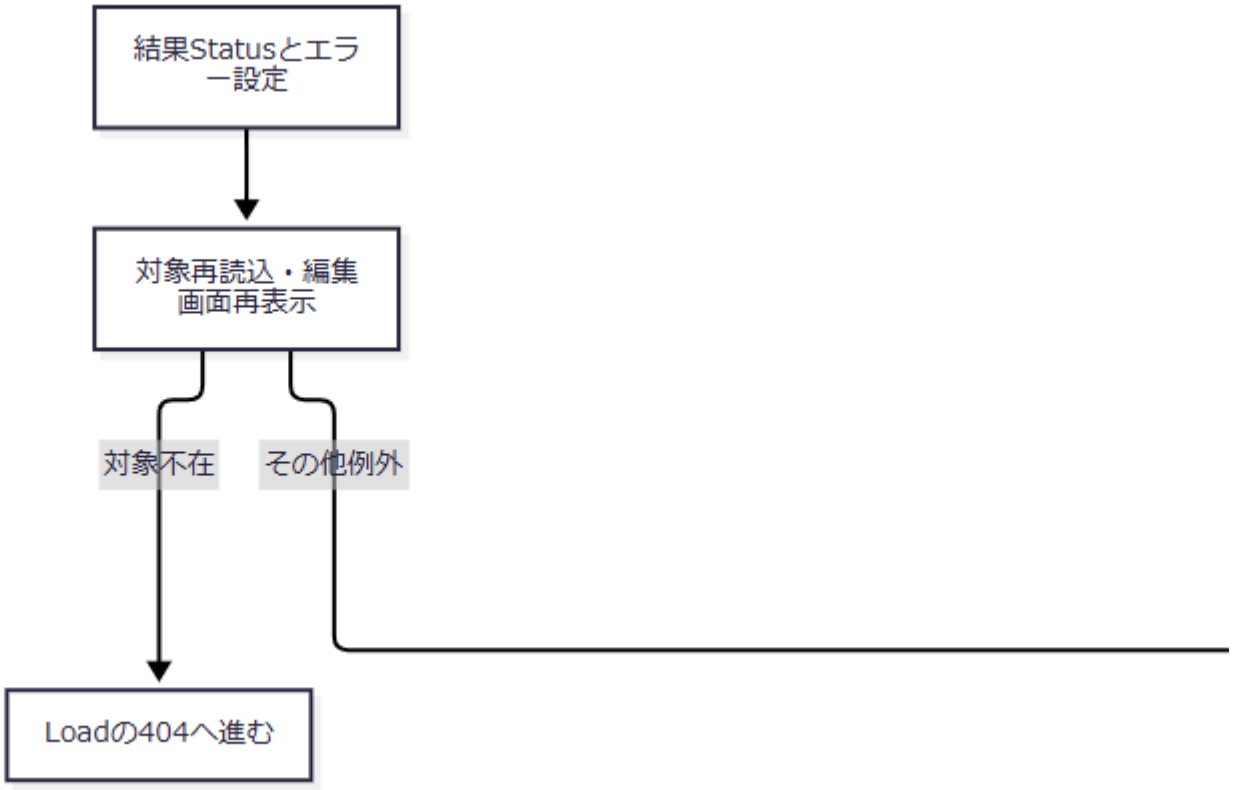

ck試行
書込開始済み、かつ、commit試行済みまたはrollback不成功
はい
いいえ
Unknown
SystemError
版プラス1保持
復帰先が非空
はい
いいえ
既存ID・snapshotのReturnRevision
新規状態ID・その版
通知格納・303 Location

diagram-010 12/15




結果Statusとエラー設定
対象再読込・編集画面再表示
対象不在
その他例外
Loadの404へ進む

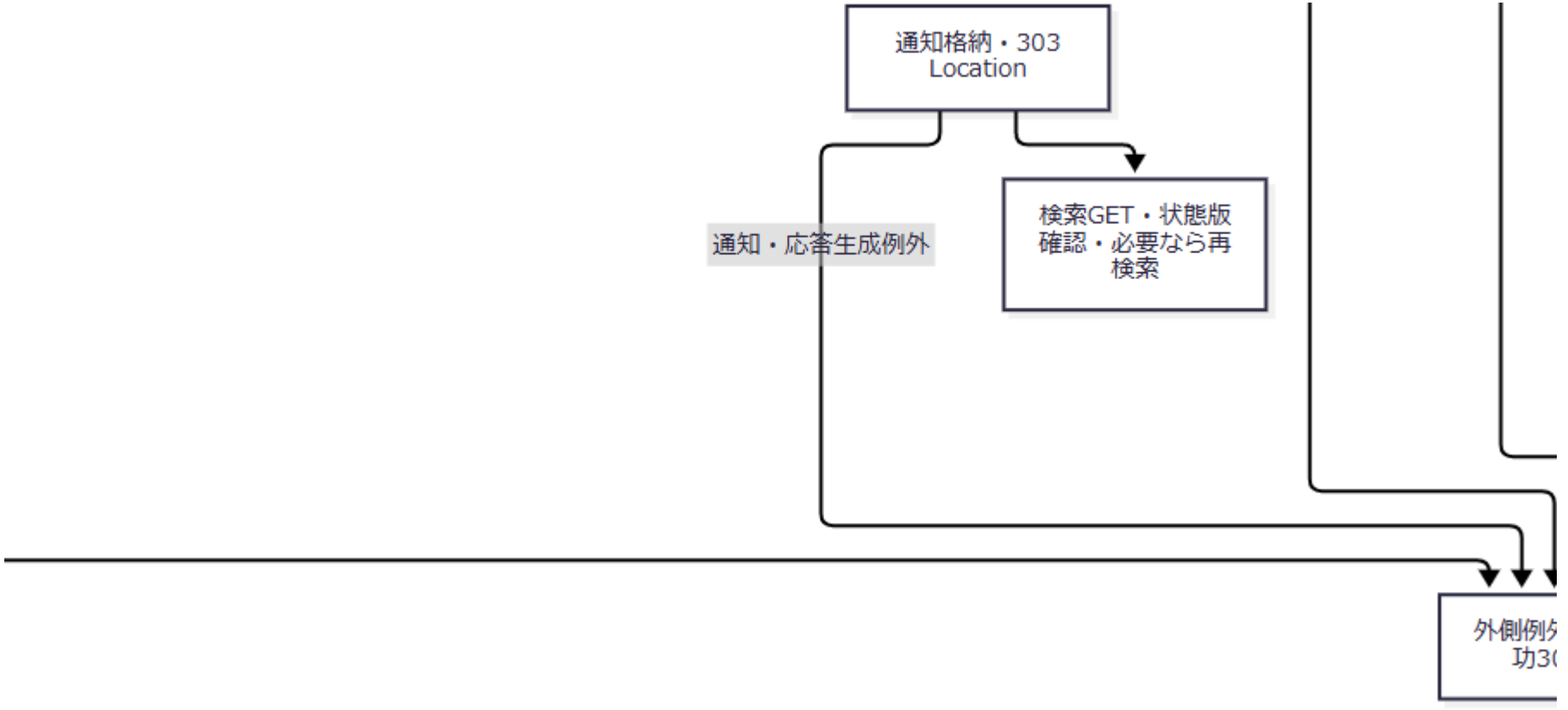

通知格納・303
Location
検索GET・状態版
確認・必要なら再
検索
通知・応答生成例外

diagram-010 15/15

